MASTER PACKET · GFIBER V8 TRANSFORMED TO AMAZON LEADERSHIP PRINCIPLES · STAR FORMAT · MATH, SOLUTION DESIGN AND PROBE QUESTIONS INTACT

Amazon — Supply Chain Manager III (L6) — the MASTER packet

You. Rajesh Rishiyur Chandrasekaran

What this document is. The GFiber MASTER packet v8 (Sep 13, 2026 — 255 pages) transformed for the Amazon L6 SCM loop: the same eighteen stories in the same uniform shape — **60-second pitch → gated STAR (five phases, each with entry · delivered · exit bar) → phase-by-phase probe Q&A → business-impact model (KPI tree, numbers table, math) → full step-by-step derivation → solution flow (the system & data map) → sensitivity (low/base/high) → the spoken total-impact statement → the full "if challenged" answers** — mapped to Amazon's sixteen Leadership Principles instead of a JD. §3 converts the twelve system-design prompts into design engagements told from the Elementum years, each keeping its stocks-and-flows map, first-principles invariants, solution design, system map, math and deep-dive probes. §4 is the numbers ledger — one canonical value per metric — and the never-say list. **This document is self-contained: nothing requires going back to the source packet.**

Relation to the loop packet. This is the depth book; *amazon-l6-scm-loop-packet.pdf* is the loop-mechanics book (interviewer tracker, AIQB 128-question bank, writing-exercise defense, video execution). The crosswalk — including the places where the two documents currently disagree — is in §1.3. Resolve those conflicts before the loop; never carry both versions of a number into the same building.

Voice, held everywhere. TPM / SAP integration program lead. "I" throughout. SAP teams and configurators built to the signed spec; I owned the data contract, the definitions, the gates, the signatures and the calls. Never claim to configure or deploy SAP. The only platform personally configured: Manhattan DOM. The MIT Supply Chain MicroMasters + Supply Chain Network Design certificate is an acquired toolkit, labeled wherever it appears.

Basis labels, used on every number. *record* — in the program record, say it flat · *user anchor* — a figure a named stakeholder stated, attributed · *industry benchmark* — a published rate, named · *modeled* — computed from stated inputs; say "modeled" in the same breath, never promote it to record.

0. How to use this packet in the loop

1. **Night before:** §1 LP map and roster → §4 never-say list → skim each story's pitch line.
2. **In the room, unprompted:** the 60-SECOND PITCH only. It ends on "Primary metric — X, before → after, $ impact." Stop there. The interviewer's probes are how they fill their STAR worksheet; silence after the pitch works for you.
3. **"Walk me through it":** STAR. The Action is five gated phases — Discovery → Plan & Design → Build & Test → Cutover/Go-Live → Hypercare — each spoken as entry bar · what I delivered · exit bar.
4. **"How did you actually run it":** the IF PROBED — PHASE Q&A block. Answer the phase asked, then stop.
5. **"How would you design…":** §3. Open with the stocks-and-flows map, state the invariants, then the design, then the metric. Program methodology lives only in §3's shared section — never inside an employer story.
6. **Numbers:** §4. A record dollar is said flat. Where the record holds no dollar, say the operational metric and then the modeled figure with the word "modeled" in the same breath — each story's *Business impact model* carries the inputs, the math and the low / base / high range. Never promote a modeled number to record.
7. **Amazon loop mechanics:** five 55-minute video interviews, each interviewer assigned 2–3 Leadership Principles, STAR worksheet, notes compared afterward. One question, one story — never the same story twice across the loop. The LP map below is the routing table.

1. The Amazon map — Leadership Principles → stories, the roster, and the crosswalk

1.1 Leadership Principle → story map

Primary = the opener when that LP's generic question lands. Backup/third = same LP, different interviewer. E-numbers are the §3 design engagements — use them when the question is "design…" or "how would you approach…", not for behavioral asks. Four stories open two LPs each (S10, S12, S13, S15): if both of a pair's LPs land in one loop, the second occurrence takes that LP's backup — never the same story twice.

LEADERSHIP PRINCIPLE	PRIMARY	BACKUP	THIRD
Customer Obsession	**S13** Cyber Monday — promise-at-risk allocation	S4 the promise rule fixed for customers	S11 graduation criteria per wave
Ownership	**S16** bus factor 1 → pipeline; owned the security miss	S1 took the definition nobody owned	S14 the seam between OMS and WMS
Invent and Simplify	**S17** lane master beats a solver	S16 four systems → one pipeline	S3 de-aggregation design; any §3 E-design
Are Right, A Lot	**S10** "noise said carriers; the data said measurement"	S8 measured lanes vs brochure	S4 wrong rule, right correction
Learn and Be Curious	**S7** learned CM data by math, not meetings	S16 learned the stack to build the pipeline	MIT toolkit (labeled), E7 forecasting
Hire and Develop the Best	**S15** 20-person team to a 4-week cadence; QA elevated	S11 wave teams grown into owners	—
Insist on the Highest Standards	**S2** robotics held at 94 vs 98 gate	S18 0.5% tolerance published before mock one	S9 ≥95% valid-ASN cohort gate held
Think Big	**S12** flipped the 18-month plan; ~$5M avoided	S9 1,000-lane vision, staged honestly	S1 book-of-record migration
Bias for Action	**S13** 4 AM allocation call	S18 rehearsed fallback fired, date held	S14 two days to the executive review
Frugality	**S12** ~$5M avoided by sequencing	S17 the bid — analyst weeks returned	S16 >$100K/yr priced before build
Earn Trust	**S14** $2.5M discrepancy — transparency over a quiet fix	S1 one signed "available", S3 Finance+Ops on one number	S5 the council; S6 two-signal release
Dive Deep	**S8** 200 lanes ranked; promise at p99	S10 Austria — measured the measurement	S3 aggregates de-aggregated; S7 phantom math
Have Backbone; Disagree & Commit	**S9** 597 vs 1,000, two levels up	S14 refused the quiet fix	S5 council without owning a system; S2 throttle
Deliver Results	**S11** 7 → 68 behind graduation gates	S18 8 months, both hubs, gate cleared	S2 the comingle launch; S6 3 months to live
Strive to be Earth's Best Employer	**S15** QA moved from afterthought to gatekeeper	S2 the floor protected during the ramp	—

LEADERSHIP PRINCIPLE	PRIMARY	BACKUP	THIRD
Success & Scale Bring Broad Responsibility	**S10** one site's fix rolled to 25 sites / ~$8M	S1 oversell is a customer harm, not a ledger error	S9 small carriers kept inside the standard

1.2 Story roster (speak in this order)

#	TITLE	PRIMARY METRIC (BEFORE → AFTER)	$ IMPACT
S1	Sephora S/4HANA ↔ ISM — one signed "available"	Oversell incidents recurring → 0	≈$2.5M/yr: $1M MCS run cost (user anchor) + ≈$1.5M modeled
S2	Sephora UDC — one comingle pool + robotics gate	Processing efficiency → +15%	≈$1.6M/yr modeled (record: none)
S3	Sephora SNB de-aggregation — one Financial On-Hand	FOH truths 2 → 1	≈$0.6M/yr modeled (record: none)
S4	Sephora in-transit promise rule (the failure story)	Starved lanes → restored; oversell +0	≈$0.5M/yr modeled (record: none)
S5	Sephora architecture council / signed FSD	Surprise escalations → 0	≈$0.5M one-time modeled (record: none)
S6	Elementum — real-time supply/demand view	Forecast accuracy 70 → 85%	$2M safety stock released (record)
S7	Elementum — contract-manufacturer BOM phantoms / revisions	Sites with planning-grade BOM 0 → 6	≈$0.3M/yr modeled; the $1.5M stays with S8
S8	Elementum — p99 lane promise	Lead-time accuracy → 99% on 200 lanes	580K units / ~$1.5M released (record)
S9	Elementum — 597-lane EDI Pareto	Completeness 60 → 95%	Freight −10% ≈ $2M (record)
S10	Elementum — Austria routing guide DQ/OTD	Data quality ~55 → ~99%	$500K/yr Austria → ~$8M across 25 sites (record roll-up; ~$320K/site avg)
S11	Flexport — Shipment Visibility launch (the wave machine)	CTS $0.66 → $0.03 (−95.5%) across 7 → 68 customers	CTS record; ~$1.5M pre-prod separate
S12	Flexport — FF App export launch (the roadmap flip)	Effort: 18-month plan → 66% less; value month 6	~$5M cost avoidance (record, said once)
S13	Manhattan — Cyber Monday 4 AM allocation	99% fill vs 95% goal	~$6M at risk → ~$8M protected (record)
S14	Manhattan — the $2.5M OMS↔WMS discrepancy	~200 orders reconciled before the review	$2.5M record; $10M initiative won (not claimed as savings)
S15	Sephora — 20-person team to a 4-week cadence	Critical defects −50%	≈$0.13M/yr modeled
S16	Flexport — four systems into one Snowflake pipeline	Accuracy 60 → 90%; parity within 1%	>$100K/yr priced (record of the pricing)
S17	Elementum — the semi-annual carrier bid	Cycle 6 weeks → ~3 (~50%)	≈$0.19M/yr modeled analyst hours; never award savings

#	TITLE	PRIMARY METRIC (BEFORE → AFTER)	$ IMPACT
S18	Sephora — RP → JDA WMS dual-DC migration	Mocks 7% → 1.8% → 0.3% vs 0.5% tolerance	$20M pipeline shipped on time; ~3% under budget

1.3 Crosswalk to the loop packet — and the conflicts to resolve before the loop

The loop packet (amazon-l6-scm-loop-packet.pdf) tells many of the same programs under its own story codes. Safe mappings first; then the four places the two documents **currently disagree**. Pick one version of each before the loop — interviewers compare notes, and so do documents.

THIS PACKET	LOOP PACKET	STATUS
S1 one "available"	S2	Same program, consistent.
S2 UDC comingle	S11 + S24	Consistent (robots beat = loop S11; log and pacing = loop S24).
S6 supply/demand view	S5	Consistent (70→85%, $2M, two-signal).
S7 CM BOM	S6	Consistent (six sites, 3 waves, 5 months, snapshot-diffing).
S8 p99 lanes	S3 (+S4 math)	Consistent (18d brochure, p99, 580K/$1.5M).
S9 EDI Pareto	S7	Consistent (597/1,000, ≥95% gate, −10% ≈ $2M).
S13 Cyber Monday	S14	Consistent (99% fill, ~$8M).
S17 carrier bid	S8	Consistent (6→~3 wks, <5% gate, no award dollar).
S18 WMS migration	S15	Consistent (0.5%, 7→1.8→0.3, paper-pick fallback).
S3, S4, S5, S10, S14, S15	—	New here; not in the loop packet. Safe to use anywhere.
S11 wave machine	MAIN + S18 + S26	One program: the loop packet splits it into the WBR (MAIN), the pilot launch (its S18) and the lever map (its S26). Consistent apart from the two conflicts below.
S16 Snowflake pipeline	MAIN (BRD/pipeline beat)	Consistent — the security-review miss appears in both. Note: the loop packet's S16 is a *different* story (the Elementum UAT crash).
S11 automation baseline	MAIN	**CONFLICT.** This packet (GFiber v8 ledger): automation 40% → 90%. Loop packet: 60% → 97%. Same program, two baselines. Decide which is the record and correct the other document.
S12 roadmap flip employer	S12	**CONFLICT.** This packet: Flexport FF App export launch (operator-first vs partner-first). Loop packet: told as a Sephora roadmap. Same numbers (−66% effort, ~$5M, month 6 vs 18) under two employers — the most dangerous inconsistency in the set.
S11 WBR hand-run hours	MAIN	**CONFLICT.** This packet: 1.5–2 → ~4 hrs/wk by hand. Loop packet: WBR prep 30 → 6 h/wk ≈ $103,680/yr. Different claims about the same review's manual effort.
S1 retirement dollars	S2	**DIVERGENT MODEL.** This packet: ≈$2.5M/yr decomposed ($1M MCS run cost, user anchor + ≈$1.5M modeled). Loop packet: ≈$1M/yr support cost. The $1M is the shared core; say the decomposition only from this packet's model.

THIS PACKET	LOOP PACKET	STATUS
S18 migration budget	S15	**CONFLICT.** This packet (GFiber v8 ledger): ~$1.8M budget, ~3% under (~$54K — consistent arithmetic). Loop packet (from the SAP playbook): ~$800K budget, ≈$54K under. $54K is 3% of $1.8M, not of $800K — this packet's pair is internally consistent; fix the loop packet.
S18 slip exposure	S15	**CONFLICT.** This packet: $20M pipeline ≈ ~$2.5M/*month* of slip. Loop packet's math box: ~$2.5M/*week* ($20M ÷ 8 weeks). Monthly is the record's framing; fix the loop packet.

> **Rule:** inside this packet, every number follows the §4 ledger (carried from the GFiber v8 record, kept as-is per the transform instruction). Where §1.3 marks a conflict, do not speak either version until you have decided which document is right — then fix the other one. **Code collision warning:** the two documents number stories independently — "S18" here is the WMS migration, "S18" in the loop packet is the pilot launch; always say which packet's code you mean.

2. The eighteen stories — pitch, gated STAR, phase probes, and the impact math

Every story keeps the GFiber v8 shape: **60-second pitch** (say first, unprompted; ends on the primary metric) → **STAR** with the Action as five gated phases (entry · delivered · exit) → **IF PROBED phase Q&A** (answer the phase asked, then stop) → **business-impact model** (KPI tree, numbers table, the math line by line, sensitivity, and what to say when a number is challenged). The GFiber fit line is replaced by the Amazon LP mapping and a "where this lands at Amazon" sentence. Dates and counts inside each story are the program record; anything labeled illustrative exists only to make the math complete.

S1 — Sephora S/4HANA ↔ ISM: one signed meaning of "available" across Store, Retail DC and Dotcom DC — policy before configuration

Amazon LPs: Earn Trust (backup) · Ownership (backup) · Think Big (third — the book-of-record migration) · Broad Responsibility (oversell is a customer harm). **Crosswalk:** loop packet S2 — same program, consistent. **Recency:** current role.

Honesty label. Integration program lead — no system reported to me; every configurator sat in another org. I owned the definitions, the asymmetry, the decrement and reconciliation contract, the signature chain and the council ask; engineers configured and coded to the FSD; planners executed the business acts.

60-SECOND PITCH

"**Situation.** Sephora was moving the inventory book of record from the in-house MCS to SAP S/4HANA, with ISM stood up beside it for the real-time view; Store, Retail DC and Dotcom DC each meant something different by 'available,' and most oversell bugs are vocabulary bugs. **Task.** As the integration program lead for inventory supply — no system reported to me — I had to define every bucket, its owning system, and how available-for-sale is computed and decremented in real time, and get Planning, Finance and operations to sign it before configuration. **Action.** Discovery: a bucket inventory with one proposed owner per bucket; the bar was every bucket owned. Plan & Design: the lower-of-two basis, the AFS equation, the real-time decrement contract with idempotent events, daily reconciliation to S/4 — FSD v1.1 → v1.2 → signed 6/5/25 by three orgs. Build & Test: node × channel × bucket-transition matrix; E2E Inbound 11/22, Dotcom 12/20, Retail 1/30. Cutover: January planning go-live, then May execution; OMS reads published AFS, never computes; MCS retired as book of record. Hypercare: daily ISM-vs-MB52 drift; control owners on Negative AFS. Lesson: policy before configuration. **Result.** Primary metric: customer-facing oversell incidents — recurring across three node types → **zero through two go-lives**; one signed AFS per node per channel. $ impact: ≈$2.5M/yr — $1M measured MCS support cost retired, plus ≈$1.5M modeled (avoided oversell, dual-book reconciliation labor, legacy change premium); the record has no pre-state oversell dollar, so the $1.5M is my model and I say so."

S — Sephora moved the book of record from MCS to S/4HANA, with ISM beside it to see digital demand S/4 cannot see in real time — a web order placed, a BOPIS order packed, a POS sale rung but not yet posted. Three node types, three working definitions of "available." Merchandising wanted merchandise reserve visible, Finance wanted on-hand to tie to the book, dotcom wanted in-transit promisable. Nobody owned the definition; I owned no system.

T — Own the integration design and data contract of inventory supply: which buckets exist, which system owns each, how available-for-sale is computed per node, and how it feeds promising, replenishment and Finance.

A — Five gated phases (record dates: kickoff 3/29 → council 5/15 → LOE 7/2 → sign-off 7/31 → FSD 8/14 → peak freeze Oct–Dec; ISM FSD v1.1 3/5/25 → v1.2 5/16/25 → signed 6/5/25; E2E Inbound 11/22 · Dotcom 12/20 · Retail 1/30; issues log 836 rows / 85.4% closed. The dated gate chain, the issues log and the rollback triggers are *program-level artifacts shared with S2/S5 — one umbrella program*; if two of these stories go to one loop, say so):

1. **Discovery** — profiled both books (MCS and S/4) for SKU overlap and order profiles per node; node-by-node sessions with ~15 functions; built the bucket inventory with one proposed owning system per bucket. *Exit bar:* every bucket owned by exactly one system; business sessions closed 4/30; open questions carried with owner and date.
2. **Plan & Design** — wrote the lower-of-two basis, the AFS/FOH equations, the real-time decrement contract (event → bucket delta, idempotency key per event), the daily ISM-vs-MB52 reconciliation into FSD v1.1 → v1.2, the RAID log and the council pack (S5). *Exit bar:* FSD signed 6/5/25 by Supply Chain Strategy, Inventory Management and FSC Inventory Accounting with zero open "who owns this bucket" rows.
3. **Build & Test** — authored the node × channel × bucket-transition matrix with before/after assertions; SIT on the interface contracts before UAT; published the E2E chain dates. *Exit bar:* 0 open contract defects in SIT; E2E green 11/22 · 12/20 · 1/30; Blocker + Critical open = 0 at go/no-go.
4. **Cutover & Go-Live** — wrote the go/no-go recommendation from counted rows, the publish-switch plan (OMS reads ISM AFS, never computes) and the reserve-copy invariants; assigned control-screen roles. *Exit bar:* January go-live (reserve rules live against demand), May go-live (execution); no SKU with two published AFS values; Negative AFS clean at T+1.
5. **Hypercare / Close** — daily drift report (ISM vs MB52), Negative AFS / AFS Correction audit review, hypercare exit memo and FSD as-built note. *Exit bar:* drift = 0 SKUs on consecutive days; blockers/day under threshold; exited into the monthly release train.

R — Primary metric: recurring vocabulary-driven oversells → **zero through two go-lives**; one signed AFS per node per channel; MCS retired. $ impact: ≈$2.5M/yr ($1M user anchor + ≈$1.5M modeled — model below).

IF PROBED — PHASE Q&A (15). Answer the phase asked, then stop.

Discovery — How did you know discovery was done?

When every bucket on the table had exactly one proposed owning system and no open "who owns this" row. Sessions closed 4/30; the exit memo listed remaining questions with an owner and a date, not a hope.

Who wrote the success criteria and who signed?

I wrote them with the VP Supply Chain in the sponsor memo — one signed AFS per node per channel, one Financial On-Hand, zero customer-impacting incidents. Supply Chain Strategy, Inventory Management and FSC Inventory Accounting signed the FSD that carried them.

What did you profile before designing?

Both books — MCS and S/4 — for SKU overlap and order profiles per node type, so the design argument started from their own data, not opinions about it.

Plan & Design — What was in your risk register at design sign-off?

Three at the top: vocabulary drift (a bucket without an owner), feed latency from WMS/OMS/POS, and override abuse on AFS-changing screens. Each had a trigger, a mitigation and an owner — mine was drift.

How did you handle an objection at the design review?

Merchandising wanted merch reserve promisable. I priced it — unpickable stock sold — and offered visibility without promisability. The no went into the FSD as a row, not into a meeting minute.

Why three signatories, not one?

Each owned a different truth: Strategy owned policy, Inventory Management owned execution, Inventory Accounting owned the book. One signature would have left two orgs free to disagree later.

Build & Test — How did you sequence stories in Jira?

Epics per module — reserve module, AFS engine, SNB, control screens — slotted by business criticality × effort; a story could not enter QA without a matrix cell reference.

How were UAT entrance criteria set and who could fail a build?

Entrance: SIT contract defects closed, test data handed off, environments ready. QA owned the gate and could fail a build; planners confirmed in UAT because QA had already demoed each matrix family during build.

Describe your bug-bash equivalent.

The dated E2E days — Inbound 11/22, Dotcom 12/20, Retail 1/30. Business and engineering ran the full flow in one room; every finding got a severity and owner before we broke.

Cutover — What made you willing to say go?

Counted rows: Blocker + Critical at zero, E2E green, the monitor for the interim copy built and tested, rollback triggers armed. Never confidence in the room.

What were your rollback triggers and did any fire?

Sync below 99% for four consecutive hours, or order-flow p95 above twice budget for an hour — consumers revert to the prior publish. Armed for both go-lives; never fired.

Hypercare — How did you decide hypercare was over?

Daily ISM-vs-MB52 drift and blockers/day under threshold on consecutive days, and the control owners — Inventory Control, Internal Audit, Finance — running the Negative AFS report without me. Then the design entered the monthly release train.

What did hypercare hand over, and to whom?

The control-owner map: Negative AFS to Inventory Control, Internal Audit and Finance; AFS Correction to the ISM app team; the daily drift report to Inventory Control. Every AFS-changing act had a screen, a role and an audit row before I stepped back.

Cross-phase — You owned no system; how did you get sixty people to sign?

Pre-wired every row of the systems-impacted table with its owner's level-of-effort before the council, ran working sessions to one table instead of arbitrating in meetings, and asked the council for three decisions rather than a review. A no that lives in a document survives; a no that lives in a meeting gets re-litigated weekly.

Why lower-of-two and not an average? · What would you do differently?

Oversell is a customer defect; undersell is a planning variance Inventory Control chases through the Negative AFS and AFS Correction screens — the asymmetry is the policy. Differently: the in-transit rule (S4) — back-test a ruleset against history before publishing it.

Where this lands at Amazon: every fulfillment promise rests on one published availability number per node that downstream systems read and never recompute — the exact contract I held the OMS to. At Amazon scale the same discipline is the ATP/promise seam between IPC-style planning and the order pipeline: one definition, one owner, policy signed before code.

BUSINESS IMPACT MODEL — KPI TREE, NUMBERS, THE MATH

```
Gross margin + customer trust (primary outcome)
+- Oversell / cancel cost (flow, $/yr) -> 0 incidents
|    oversell = orders x vocab-driven oversell rate x cost/incident
|    (loop) oversell -> NPS down -> repeat rate down -> orders down
+- Dual-book reconciliation labor (stock: FTE) -> 1 FTE + drift report
|    (loop) more drift -> more manual recon -> slower fixes -> more drift
+- Legacy run cost (MCS support, $/yr) -> retired
+- Tech-debt change premium (changes/yr x extra hrs x rate) -> S/4 + ISM
+- Book accuracy -> unexplained shrink / audit-finding risk
+- Time-to-market for a new node or channel (weeks) -> a matrix row
```

DRIVER	BASELINE	AFTER	$/YR	BASIS
MCS run / support cost	≈$1.0M/yr	$0 — retired	$1.0M	user anchor
Oversell / cancel incidents	recurring, ~0.10% of orders	zero through two go-lives	≈$0.9M	modeled (rate benchmark 0.05–0.25%; record: → zero)
Dual-book reconciliation labor	≈4 FTE by hand	≈1 FTE reading the drift report	≈$0.33M	modeled (3 × $110K)
Tech-debt change premium	≈40 changes/yr × +60 h on MCS	platform cost	≈$0.3M	modeled (40 × 60 × $125)

DRIVER	BASELINE	AFTER	$/YR	BASIS
Book accuracy → shrink · time-to-market	—	—	≈$0.3M + $0.2M	modeled upside, not in headline

```
Oversell avoided = 30M orders x 0.10% = 30,000 incidents/yr
  (benchmark: e-com inventory cancels run 1.5-3%; only 0.05-0.25% attributed to the
   three-definitions problem)
Cost/incident = CS contact $6 + refund handling $4 + lost margin ~$19 + goodwill ~$1 = ~$30
Oversell $ = 30,000 x $30 = $0.9M/yr · Recon labor = (4-1) FTE x $110K = $0.33M/yr
Change premium = 40 x 60 h x $125 = $0.3M/yr
Headline (base) = $1.0M + $0.9M + $0.33M + $0.3M = ~$2.5M/yr
Sensitivity: low ~$1.7M · base ~$2.5M · high ~$5.2M (rate 0.05-0.25%, $25-40, FTE 2-4)
```

If challenged. *"How do you know it was a tenth of a percent?"* — "I don't, to the decimal. The record says incidents were recurring and went to zero; the rate is a benchmark set deliberately low — a twentieth of a typical e-commerce inventory-cancel rate. First thing I'd measure: the 'item unavailable after order' cancel-reason code for the twelve months before January." *"Does the $1M already include your change premium?"* — "It might. If enhancement labor sits inside it, I drop the row and the headline is about $2.2M. I'd split the MCS cost center into run vs change tickets for one fiscal year." *"Three FTE sounds like a guess."* — "It's a benchmark count. What's in the record is the mechanism: reconciliation moved from argument to a daily drift report with tolerance zero and a named owner. I'd size it with a two-week time study — how I sized the Austria effort at Elementum."

THE MATH — FULL DERIVATION, STEP BY STEP

Supply basis (asymmetric lower-of-two). For SKU i at node n: OOB_in = S4_Avail_in − WMS_OnHand_in (out-of-balance; + = book above floor, − = floor above book) Basis_in = min(S4_Avail_in, WMS_OnHand_in) — a positive OOB reduces supply; a negative OOB is never promised until S/4 books it.

Available-for-sale. AFS_in = Basis_in − PickRes_in − EcomPick_in − EcomPack_in − SNB_in , where every subtrahend is a committed-or-sold-not-posted bucket. Merchandise Reserve and Lost/Found are outside S4_Avail by construction, so they never appear in the subtraction — no double-subtract path exists.

Financial On-Hand (DC). FOH_n = Avail_n + MerchRes_n + LostFound_n (physical truth for Finance; AFS is the promising truth for channels).

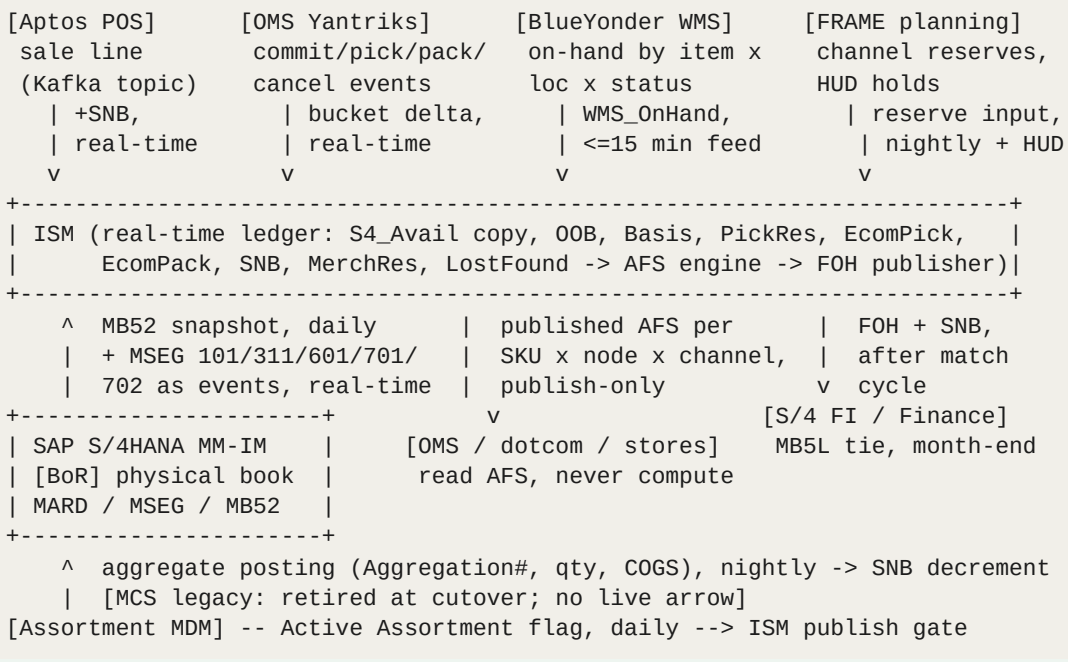
Real-time decrement (the ISM contract). Between two S/4 postings, ISM keeps AFS current from events, never from re-reading the book: AFS_in(t) = AFS_in(t_0) − Σ_{e ∈ (t_0, t]} Δ_e , where each event e (OMS commit / pick / pack / cancel, POS sale line) maps to exactly one bucket delta by the decision-rules table. Every event carries an idempotency key (source × OrderLine- or-Txn × line × event type); a replayed key is a no-op, a reversal must reference its original, so Σ Δ_e is the same whether an event arrives once or five times. Per-key ordering is per SKU × node, so a pick can never be applied before its commit.

Reconciliation (daily, tolerance 0). Drift_in = ISM.S4_Avail_in − MB52_in per SKU × SLoc at the daily snapshot; Drift ≠ 0 is a Sev, not a planning variance. Month-end: Σ_n FOH_n ties to MB5L quantity and value. SNB is the reconciling bucket between the two books: S4_Avail_in(after posting) = S4_Avail_in(before) − SNB_matched_in , so ISM decrements SNB on the matched aggregate posting only (S3) and SNB can never go negative.

Worked example (consistent with v105 Part 4 Ex. 1). S/4 Available 1,000 units; WMS on-hand 980 → OOB = +20 → Basis = 980. Picking Reserve 200, Ecom Pick 120, Ecom Pack 60, SNB 40 (Σ = 420) → AFS = 980 − 420 = 560. Flip the floor: WMS 1,020 → OOB = −20 → Basis = min(1,000, 1,020) = 1,000 → AFS = 580, not 600 — the 20-unit surplus stays unpromised. Decrement example (illustrative): a store rings 3 units at 10:02 → SNB 40 → 43, AFS 560 → 557 within the 60 s publish budget; the POS feed replays the same transaction at 10:04 → same key, no-op, AFS stays 557; the aggregate posting lands in S/4 overnight for 43 units → SNB 43 → 0, S4_Avail 1,000 → 957, and the next-morning drift check reads 957 − MB52 957 = 0 . Sanity: AFS ≤ Basis ≤ both books; never a negative published number (Negative AFS is a report, not a value channels read). The store-node walk is the same with Basis = S4_Avail (no WMS floor) and SNB as the only sold-not-posted bucket; the DC walks (Retail DC, Dotcom DC) carry the WMS floor and the OMS buckets. Comingle pool partitioning — the nine-reserve priority walk — is S2's math.

Oversell exposure (why asymmetry). Expected oversell units if surplus were promised: E[oversell] = Σ_i P(WMS surplus is phantom) × surplus_i . With no measured phantom rate at design time, the policy sets that probability to 1 (promise nothing unbooked) and lets the Negative AFS / AFS Correction screens chase the under-sell — a priced choice: under-sell is a planning variance; over-sell is a customer defect.

SOLUTION FLOW — THE SYSTEM & DATA MAP



SENSITIVITY — LOW / BASE / HIGH

INPUT	LOW	BASE	HIGH
Vocabulary-driven oversell rate	0.05%	0.10%	0.25%
Cost per oversell incident	$25	$30	$40
Oversell cost avoided	$0.4M	$0.9M	$3.0M
Reconciliation FTE released × loaded	2 × $95K = $0.19M	3 × $110K = $0.33M	4 × $130K = $0.52M
Change premium (changes × hrs × rate)	24 × 40 × $100 = $0.1M	40 × 60 × $125 = $0.3M	60 × 80 × $150 = $0.72M
MCS run cost (user anchor, fixed)	$1.0M	$1.0M	$1.0M
Total modeled $/yr (headline, excl. upside)	≈ $1.7M	≈ $2.5M	≈ $5.2M

TOTAL IMPACT STATEMENT — SAY IT LIKE THIS

"Conservatively, about two and a half million dollars a year. One million of that is measured: the MCS support cost that went away when S/4 became the book of record. About nine hundred thousand is avoided oversell: on roughly thirty million dotcom orders, a tenth of a percent going wrong at about thirty dollars each, and the record shows those incidents went to zero through both go-lives. About three hundred thousand is reconciliation labor, three analysts who stopped reconciling two books by hand, and about three hundred thousand is the change premium we stopped paying on an undocumented stack. The first number is measured; the other three are my model, and I can show you every input."

S2 — Sephora UDC: two warehouses in one building → one comingle pool, coupled with a BlueYonder upgrade and robotics behind a published gate

Amazon LPs: Insist on the Highest Standards (primary — the 98% gate) · Deliver Results · Have Backbone (the throttle) · Earth's Best Employer (the floor protected). **Crosswalk:** loop packet S11 + S24 — consistent. **Recency:** current role.

Honesty label. I owned discovery through FSD for the location and inventory model, the staging and the gates; BlueYonder, Locus and the platform teams built and configured; DC Ops ran the floor. The +15% is Sephora's signed number; every dollar is my model on an illustrative 100K-units/week DC.

60-SECOND PITCH

"**Situation.** Sephora's Utah DC ran as two systemic warehouses in one building, with a BlueYonder upgrade and Locus robotics on the same path and zero customer-facing incidents allowed. **Task.** I owned discovery through FSD for the location and inventory model, the staging, and the gates — a dozen core systems, 20+ touched, 60+ stakeholders. **Action.** Discovery: both books profiled; systems-impacted table with owners. Plan & Design: one comingle pool, three planning identities, two go-lives so each carried one defect class. Build & Test: node × channel × bucket matrix; robotics gate published at 98% throughput with a throttle built in advance. Cutover: six-step May choreography — clear negatives, transfer AFS to Uncommitted, scan-and-reserve, zero-then-publish so nothing was sellable in two places. Hypercare: week two hit 94; I throttled bots to ~30% of volume, root-caused zone/slotting logic, full volume week three. Lesson: a gate that cannot stop the program is decoration — mine did. **Result.** Primary metric: DC processing efficiency — baseline → **+15%** after one comingle pool replaced two systemic warehouses; customer-facing incidents zero through both go-lives. $ impact: ≈$1.6M/yr modeled (labor at +15%, pooling carry, fewer split shipments, self-transfer handling removed) — the +15% is the signed number; the dollars are my model."

S — The Utah DC ran as two systemic warehouses — Dotcom 801 (selling location 800) and Retail 901 — transferring stock to itself and paying duplicate handling for zero customer value. The ask: one omni/comingle facility, with a BlueYonder WMS 8.2 → 2023.1 upgrade and Locus robotics on the same critical path, a dozen core systems (20+ touched), an Oct–Dec peak freeze, zero customer-facing incidents allowed.

T — Discovery through FSD for the location and inventory model, the staging, and the acceptance gates; 60+ stakeholders on the steering path.

A — Five gated phases (record: kickoff 3/29 → sessions 4/30 → council 5/15 → LOE 7/2 → sign-off 7/31 → FSD 8/14 → peak freeze → E2E 11/22 · 12/20 · 1/30 → Locus RTL Feb–Mar, slipped 3 weeks, re-slotted openly → January go-live → May go-live; cutover plan 343 tasks / 48,885 task-minutes ≈ 815 hrs; issues log 836 rows / 85.4% closed, Blocker 164 + Critical 277 = 52.8%):

1. **Discovery (3/29 → 4/30)** — profiled 801/901 (SKU overlap, order profiles); sponsor objectives and success criteria (zero customer-facing incidents; one comingle facility); systems-impacted table. *Exit bar:* every impacted system named to an owner; success criteria signed by steering; open questions dated.
2. **Plan & Design (5/15 → 8/14)** — council pack (three decisions + out-of-scope fence, S5); the 900/901 location/inventory model with three planning identities; staged two-go-live timeline; RAID log; test-matrix skeleton; FSD to 8/14. *Exit bar:* council sign-off 7/31, FSD 8/14, peak-freeze boundary decided, LOE-backed plan across ~30 build tracks.
3. **Build & Test (Aug → Jan/Apr)** — node × channel × bucket-transition matrix as the regression asset; SIT contract-defect list; dated E2E chain; UAT entrance memo; Locus gate definition (98%) with throttle spec; three mock cutovers against the 0.5% tolerance — the S18 discipline, reused across the same Sephora program family. *Exit bar:* E2E green three dates; Blocker + Critical = 0 at each go/no-go; mock 3 inside tolerance; Locus RTL retested after the slip.
4. **Cutover & Go-Live (Jan; May 1 → Jul 1)** — cutover runbook (343 tasks, owner + duration + predecessor per task); six-step May choreography; go/no-go recommendation; rollback plan per interface. *Exit bar:* six invariants

held (Σ AFS conserved; no SKU sellable on 800 and 900 at once; negatives cleared first); rollback never fired.

5. **Hypercare / Close (Jun → Jul)** — daily burn-down read (271 → 65 → 21 → 14 → 4), Locus weekly gate read, hypercare exit memo, template handed to the next DC. *Exit bar:* blockers/day < 2 sustained → exit week 3 of 4; Locus at full volume week 3; +15% measured post-stabilization by DC Ops.

R — Primary metric: processing efficiency **+15%** (record, Sephora signed); zero customer-facing incidents through both go-lives; robotics gate held at 94 vs 98, full volume week three. $ impact: ≈$1.6M/yr modeled + ≈$2.2M one-time safety-stock release (model below).

IF PROBED — PHASE Q&A (15). Answer the phase asked, then stop.

Discovery — How did you know discovery was done?

When the systems-impacted table had a named owner and a written LOE for every row — a dozen core systems, 20+ touched — and the success criteria were signed by steering. The LOE landed 7/2.

Who wrote the success criteria and who signed?

I drafted them from the steering charter — one omni facility, self-transfers eliminated, zero customer-facing incidents, peak untouched — and steering (DC Ops, IT, Supply Chain Strategy) signed them in the council pack.

Plan & Design — What was in your risk register at design sign-off?

Two live truths during the interim, a robotics ramp with no stop condition, and a peak collision. Each had a trigger, a mitigation and an owner — the shadow monitor, the 98% gate with a throttle dial, and the freeze in the pack.

How did you handle a business owner who objected at the design review?

Digital worried a virtual Retail DC would blur their forecasting lens. I kept three planning identities over one physical location so each lens survived — the concern became the design, not an argument.

Why two go-lives?

So we never hunted two defect classes in the same week. January proved reserve rules against live demand while the floor was untouched; May left warehouse execution as the only remaining class.

Build & Test — How did you sequence work?

~30 build tracks under one log; stories slotted by business criticality × effort; per-environment workbooks (Dev 310, QA 255, Perf 200, Prod 221 rows) told each track where it could test. Blocker + Critical open = 0 was the gate.

How were UAT entrance criteria set and who could fail a build?

SIT contract defects closed, conversion build done, test data handed off, pre-prod and perf environments ready. QA owned the gate; DC Ops confirmed in UAT because QA had already demoed to them during build.

Describe a bug bash — what came out of it?

The dated E2E days: Inbound 11/22, Dotcom 12/20, Retail 1/30 — every function ran the full flow in one room, every finding logged with severity and owner before we left. The Locus 3-day workshop worked the same way for the robotics contract.

Cutover — What made you say go — or no-go?

The burn-down, not the room: Blocker + Critical at zero, mock 3 inside the 0.5% tolerance (the discipline S18 established, reused here), runbook rehearsed, rollback armed. I would have said no-go on one open Blocker; I never had to.

What were your rollback triggers and did any fire?

Sync < 99% for four consecutive hours, or order-flow p95 > 2× budget for one hour — per-interface rollback behind them. Armed for both go-lives; never fired.

How big was the cutover runbook?

343 tasks, 48,885 task-minutes — about 815 hours — May 1 to Jul 1, each task with an owner, duration and predecessor, and step 4 (zero-then-publish) as the one irreversible instant.

Hypercare — How did you decide hypercare was over?

Blockers per day under two, sustained, and the post-go-live spike decaying 271 → 65 → 21 → 14 → 4. Exited week 3 of 4. A calendar exit declares victory over open defects.

Which wave did you hold and how did the business take it?

The Locus ramp at week 2: 94 against a published 98. I throttled to ~30% for a week. DC Ops took it well because the gate and the dial were theirs before the ramp — the rule fired, I didn't improvise.

Cross-phase — Where did you trade scope for date? What did you escalate?

The comingle-reserve improvements — out of the release to protect zero incidents, priced as a later enhancement at steering. Escalated: the staged timeline and peak-freeze boundary as decisions; the Locus RTL slip the week it surfaced. I refused to compress UAT to absorb it.

Why 98%? · Why not one go-live? · Do differently?

98% was set from the pre-ramp baseline and the incident budget before the event — a gate that cannot stop the program is decoration; mine did, at 94. One go-live would mean hunting two defect classes in one week. Differently: staff the interim reporting workstream on day one, not month three — for four months every report had to declare which system of record it read.

Where this lands at Amazon: this is an FC launch/retrofit with automation on the critical path — publish the ramp gate and the throttle before the robots arrive, stage go-lives so each carries one defect class, and hold peak sacred. The zero-then-publish invariant is the same discipline as never letting one unit be promisable from two nodes at once.

BUSINESS IMPACT MODEL — KPI TREE, NUMBERS, THE MATH

```
DC cost-to-serve per unit (primary outcome)  [record: +15% efficiency]
+- Direct labor $ = units / (units per labor-hour) x loaded rate
|    +- units/labor-hour (+15%, record) <- self-transfers removed · one pick pool
|    +- Locus bot share of volume (+)
|         <- gate T >= 98% -> throttle 30% -> RCA -> ramp again (balancing loop)
+- Inventory carry $ = safety stock x cost of capital
|    +- two pools -> one pool (sqrt-n pooling) (-)
+- Split shipments $ = dotcom orders x split rate x cost per split (-)
+- Customer-facing incidents = 0 (constraint, record) -- the fence:
     every row above was allowed to move only inside this constraint
```

DRIVER	BASELINE	AFTER	$/YR	BASIS
Processing efficiency (units per labor-hour)	index 100	115	—	record (Sephora signed)
Direct labor hours for same volume	240K hrs	209K hrs	≈$0.9M	modeled (120 FTE × 2,000 h × $28 × 13%)
Safety stock, overlapping SKUs (2 pools → 1)	$7.5M	$5.3M	≈$0.2M carry	modeled (√2 pooling × 10% capital)
Split-shipment rate	~4.5M orders/yr	−1 pt	≈$0.45M	modeled (45K × $10)
Self-transfer handling	~260K units/yr	0	≈$0.1M	modeled (260K × $0.80 × 50%)
Total modeled	—	—	**≈$1.6M/yr**	+ $2.2M one-time working capital

```
hours saved = 240,000 - 240,000/1.15 = 31,300 h = 13.0% -> x $28/h = $0.88M/yr
poolable SS = $50M avg inv x 25% SS share x 60% overlap = $7.5M
pooling factor (2 equal pools -> 1) = 1 - 1/sqrt(2) = 0.293 -> release $2.2M one-time
carry = $2.2M x 10% = $0.22M/yr · splits = 45,000 x $10 = $0.45M/yr
self-transfer = 260K x $0.80 x 50% (rest already in labor row) = $0.1M/yr
total = 0.88 + 0.22 + 0.45 + 0.10 = ~$1.6M/yr
Sensitivity: low ~$1.1M · base ~$1.6M · high ~$2.4M (FTE 100-150, $24-32/h, capital 8-12%)
```

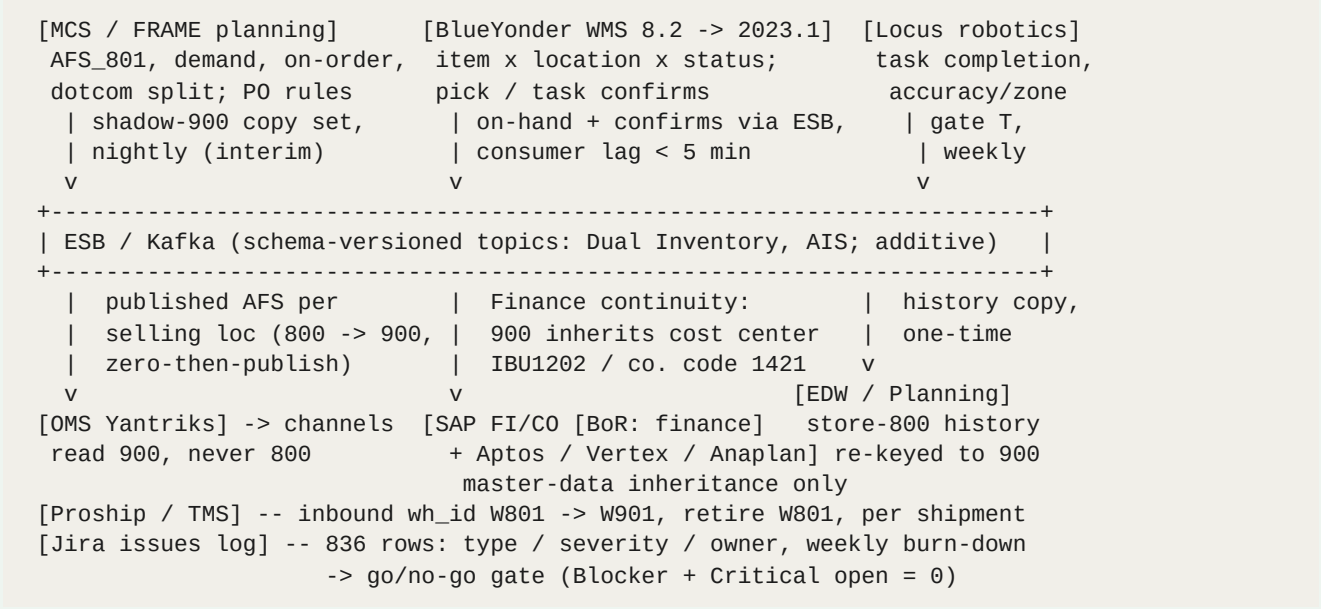
If challenged. *"The labor number is the biggest — how firm?"* — "The fifteen percent is measured; the headcount and the $28 loaded rate are my assumptions. Pull the DC's actual direct hours for the two quarters before and after and

multiply by payroll — that turns the model into a record in one afternoon." *"Pooling sounds like a textbook claim."* — "It is, and I labeled it. √2 pooling assumes two roughly equal independent pools on the same SKUs. Measure SKU-level on-hand on the last day of 801 vs the first stable month of 901; the release is whatever the difference actually is." *"Split shipments — evidence?"* — "The softest row. Parcels per dotcom order from the ship system before and after May, times the carrier's per-parcel cost. If the point didn't move, I drop the row and the total is about $1.1M."

THE MATH — FULL DERIVATION, STEP BY STEP

Throughput gate. T = pick-accurate completed tasks / tasks attempted per week. Gate published
before ramp: T ≥ 0.98 (stretch 0.99). Week 2 observed 0.94 → gate failed by 4 pts.
Throttle rule. Bot share of volume s drops from 100% to s_throttle until root cause is cleared:
s_throttle = 30% (dial built in advance). Manual capacity must cover (1 − s) × V_week ; with
V_week = weekly units, manual load = 0.70 × V. Illustrative: V = 100,000 units/week → bots 30,000, manual 70,000 — the floor staffing check is 70,000 / (units per labor-hour × hours available) ≤ 1 .
Incident budget. Zero customer-facing incidents allowed
any defect that could reach a customer must be caught at a gate: P(customer incident) ≈ Σ_gates
(1 − catch_rate_g) × P(defect_g) . Staging two go-lives halves the defect classes live at once, so the joint probability of two interacting classes in one week is removed by construction.
Interim monitor. |AFS_900 − AFS_801| > 0 on any SKU after the copy Sev. Tolerance is 0 units,
not a percentage, because 900 is defined as a copy of 801.
Rollback triggers. sync_rate = synced records / expected records ; fire if sync_rate < 0.99 for
4 consecutive hours, or order-flow p95 latency > 2 × budget for 1 hour. Illustrative: budget
p95 = 30 s trigger at > 60 s sustained 60 min. Comingle reserve partition (nine reserve types, strict priority — replicated 100% from Canada). The single physical pool P on 901 is
partitioned for both channels by a priority walk over reserve types r_k , k = 1..9 (Held first, then the DOT and Retail hard reserves, then the softer channel and launch reserves, then Uncommitted as the remainder): alloc_k = min(r_k, P − Σ_{j<k} alloc_j) , Uncommitted = P
− Σ_k alloc_k . The walk stops when the pool is exhausted, so the lowest tier is truncated first and no channel can starve another out of order. Worked example (v105 figures, split illustrative): pool 4,900 − Held 50 − DOT hard 1,550 − Retail hard 2,700 = Uncommitted 600.
Sanity: Σ alloc_k ≤ P , Uncommitted ≥ 0 . Why not tune it for the US: the Canada engine was
already proven; a second engine is a second defect class in the same release — the fence in
the council pack (S5). The comingle-reserve improvements were the scope I traded for zero incidents. S1 reads these reserves as a subtrahend input to AFS; the walk itself lives here.
Cutover invariants (six-step May choreography). Σ AFS_801 (before) = Σ Uncommitted_901 (after)
per SKU; negatives on 801 cleared first (AFS_801 < 0 ⇒ set 0 with audit) because a transferred negative is invented inventory; Yantriks zero-then- publish: AFS_pub(800) := 0 before AFS_pub(900) := AFS_901 , so at no instant is a SKU sellable on both.
Result metric. Processing efficiency +15% (canonical; definition per program = units processed
per labor-hour vs pre-UDC baseline).
Issues log: 836 rows, 85.4% closed; post-go-live spike 271 → 65 → 21 → 14 → 4 (weekly open counts) — a geometric burn-down with ratio ≈ 0.3–0.7 per week.

SOLUTION FLOW — THE SYSTEM & DATA MAP



SENSITIVITY — LOW / BASE / HIGH

INPUT	LOW	BASE	HIGH
Direct DC FTE (illustrative)	100	120	150
Loaded labor rate (benchmark)	$24/h	$28/h	$32/h
Labor $ saved	$0.63M	$0.88M	$1.25M
Cost of capital (benchmark)	8%	10%	12%
Carry $ saved on $2.2M release	$0.18M	$0.22M	$0.26M
Split rate reduction × cost per split	0.5 pt × $8	1 pt × $10	1.5 pt × $12
Split $ saved	$0.18M	$0.45M	$0.81M
Self-transfer handling (fixed share)	$0.1M	$0.1M	$0.1M
Total modeled $/yr	≈$1.1M	≈$1.6M	≈$2.4M

TOTAL IMPACT STATEMENT — SAY IT LIKE THIS

"Conservatively, about one point six million a year: roughly nine hundred thousand of labor, because the same volume takes thirteen percent fewer hours at plus fifteen percent efficiency; about two hundred thousand a year of carry on the two point two million of safety stock one pool releases versus two; about four hundred fifty thousand from fewer split shipments; and about a hundred thousand from the DC no longer shipping to itself. The plus fifteen percent and the zero incidents are measured and signed by Sephora; every dollar figure is my model on an illustrative hundred-thousand-units-a-week DC, and I will tell you which input each one rests on."

S3 — Sephora S/4HANA: Shipped-Not-Billed and de-aggregation — Finance and Operations on one Financial On-Hand number

Amazon LPs: Earn Trust (backup — Finance and Ops on one number) · Dive Deep (the grain problem) · Invent and Simplify. **Crosswalk:** not in the loop packet — safe anywhere. **Recency:** current role.

Honesty label. The flow, controls and ownership map were mine; S/4 and OMS engineers implemented. The record has no canonical dollar for the pre-state drift; the ≈$0.6M/yr is my model and I say so.

60-SECOND PITCH

"**Situation.** S/4 posts sales as aggregates while POS sends lines, so between sale and posting units were gone but the book was not decremented — Finance and operations reconciled by argument. **Task.** Define how the real-time layer tracks that gap so promising, replenishment and Finance agree, under a zero-incident constraint. **Action.** Discovery: profiled the posting-vs-line grain and named the phantom-availability window. Plan & Design: SNB incremented by POS/OMS/WMS events, decremented only on matched S/4 aggregate; de-aggregation pro-rates COGS per line; failure direction under-states, never over-states. Build & Test: Aggregation# join in the matrix; control owner per bucket. Cutover: publish Financial On-Hand and updated SNB back to S/4. Hypercare: unmatched postings age on a report, not as phantom AFS. Lesson: design the failure direction before the happy path. **Result.** Primary metric: Financial On-Hand definitions — **two (Finance book vs operations count) → one shared FOH**, availability decremented at sale/ship rather than at posting; zero incidents. $ impact: ≈$0.6M/yr modeled (phantom-availability cancels avoided + Finance reconciliation labor + month-end close time) — one FOH is the signed result; the record has no pre-state drift dollar and I am not claiming one."

S — S/4 posts sales as aggregates — one posting per Aggregation# with total quantity and COGS; the Aptos POS sends transaction lines with an Order#. Between the sale and the posting, units are gone but the book is not decremented — the phantom-availability window — and Finance and Operations read different on-hand numbers and reconcile by argument.

T — Define how ISM tracks the gap in real time and reconciles it so promising, replenishment and Finance agree, with the program's zero-incident constraint applying to the book as much as to the customer.

A — Five gated phases (SNB rode the ISM program's dated chain — FSD signed 6/5/25; E2E 11/22 · 12/20 · 1/30; May execution go-live):

1. **Discovery** — problem statement (the phantom-availability window), the event inventory (POS flash sale, omni pending, ship-to-home confirm, S/4 Aggregation# posting), the who-reads-which-number map. *Exit bar:* Finance and Ops agree on the description of the gap and which events move units before posting; Internal Audit named as reviewer.
2. **Plan & Design** — SNB bucket definition (increment on events, decrement only on matched posting); join spec Aggregation# ↔ Aptos header ↔ lines; per-line COGS pro-rata de-aggregation; the failure-direction statement (unmatched ⇒ SNB stays high); control-owner map into the FSD. *Exit bar:* signed inside the FSD 6/5/25; Finance accepts one FOH; WAC$ confirmed S/4-owned, ISM visibility-only.
3. **Build & Test** — de-aggregation test family (n lines; missing line; duplicate Aptos header); the SNB-never-negative assertion; unmatched-posting aging report spec. *Exit bar:* in SIT every posting matches or ages visibly; Σ per-line COGS = posting COGS to the cent in every cell; UAT = confirmation by Inventory Accounting.
4. **Cutover & Go-Live** — riding the May go-live: SNB opening balance reconciled to unposted sales at the snapshot; SNB publish back to S/4 enabled. *Exit bar:* aging report empty or explained at T+1; no negative SNB; Finance reads FOH from ISM.
5. **Hypercare / Close** — daily aging review with Inventory Accounting; control-owner handoff in the exit memo. *Exit bar:* aging report owned by Finance without me; month-end FOH ties to MB5L (mechanism canonical; days-to-tie not in record).

R — Primary metric: FOH truths **2** → **1**; AFS decremented at sale/ship; over-sell from unmatched postings = 0 by construction; zero incidents. $ impact: ≈$0.6M/yr modeled.

IF PROBED — PHASE Q&A (14). Answer the phase asked, then stop.

Discovery — How did you know discovery was done?

When Finance and Operations agreed on the description of the gap — units gone, book not decremented, between sale and posting — and on the exact events that move units before posting. Agreement on the description came before any design.

Who wrote the success criteria and who signed?

I wrote "one Financial On-Hand for Finance and Operations; AFS decremented at sale, not posting; safe failure direction." FSC Inventory Accounting signed it inside the FSD on 6/5/25 with Strategy and Inventory Management.

Plan & Design — What was in your risk register at sign-off?

A double-subtraction path, unmatched postings accumulating silently, and late or duplicate POS lines. Mitigations: designed out by construction, an aging report Finance owns, idempotency keys with a late-arrival flag.

How did you handle Finance objecting at the design review?

They wanted their own reconciliation. I showed the same number arriving from the physical side first and gave them the report to own. A parallel spreadsheet dies when its owner owns the real report.

Why decrement only on a matched posting?

An unmatched posting keeps SNB high, so AFS under-states and Replenishment over-orders visibly. Decrementing on an estimate would hide the gap as oversell — the failure direction was designed before the happy path.

Build & Test — How did you sequence the stories?

By event source: POS flash sales, then omni pending, then ship-to-home confirmations — each added to SIT only when its increments and matched decrements reconciled. The de-aggregation family ran on every source.

What was your definition of done?

Matrix cell green, the "SNB never negative" assertion, and Σ per-line COGS equal to the posting COGS to the cent. A story without the failure-direction assertion was not done.

Who could fail a build in UAT?

Inventory Accounting. Their entrance criterion was "every SIT case ties to the cent"; they confirmed rather than discovered because QA had demoed each case during build.

Cutover — What was the cutover risk specific to SNB?

The opening balance — sales made but not yet posted at the snapshot. I reconciled SNB's opening balance to unposted sales before the publish switch; the T+1 aging report was the go/no-go read for the matching cycle.

What were your rollback triggers?

Aging report growing without explanation, or any negative SNB at T+1 — rollback was cheap because S/4 stayed settlement authority; ISM republishes SNB from the event log. Neither fired.

Hypercare — How did you decide hypercare was over?

When Finance ran the aging and Negative AFS reports without me and month-end FOH tied to MB5L through the ISM number, not a bridge spreadsheet. I don't have a canonical days-to-tie figure and won't invent one.

What did the first weeks teach you?

That the aging report is the real product — the day Inventory Accounting asked me why a posting had aged before I noticed it, the control had an owner. Do-differently: Internal Audit in discovery, not review.

Cross-phase — Where did you trade scope for date?

Real-time per-line posting to S/4 — never in scope, and I held that line. The price was a matching cycle to maintain; the benefit was one book.

Why not post every line to S/4? · Why not let Finance derive SNB themselves?

Throughput and posting cadence — S/4 posts aggregates for a reason; ISM is the real-time ledger, S/4 stays the settlement authority. And a Finance-derived SNB is two numbers; the whole point was one.

Where this lands at Amazon: every fulfillment event moves units before Finance posts them — the same window at a vastly larger grain. The transferable design: one real-time ledger, the financial system as settlement authority, a safe failure direction chosen before the happy path, and a control report with a named owner instead of a bridge spreadsheet.

BUSINESS IMPACT MODEL — KPI TREE, NUMBERS, THE MATH

```
One Financial On-Hand for Finance and Operations (primary outcome)
+- Oversell / cancel cost = orders x phantom-cancel rate x cost/cancel
|    +- window W = t_post - t_sale -> SNB increments at sale/ship, AFS decremented now
|    +- failure direction: unmatched => under-sell, never over-sell
|         <- aging report -> Finance fixes match -> SNB falls (loop, -)
+- Finance reconciliation labor $ = analyst hrs/wk x 52 x loaded rate
|    +- bridge spreadsheet retired; de-aggregation ties COGS per line to the cent
+- Month-end close hours = people x days x hrs x 12 -> no bridge at close
+- Replenishment noise (stock: over-orders on under-stated AFS)
     <- match rate >= 99.5% in 24h keeps this stock near zero (loop)
```

DRIVER	BASELINE	AFTER	$/YR	BASIS
Financial On-Hand truths	2 (Finance vs Ops)	1 shared FOH	—	record (FSD 6/5/25)
AFS decrement point	at S/4 posting	at sale / ship	—	record
Phantom-availability cancels	~0.10 pt of orders	~0	≈$0.45M	modeled (30M × 0.10% × $15)
Finance reconciliation hours	32 hrs/wk bridge	~0	≈$0.14M	modeled (32 × 52 × $85)
Month-end close, inventory tie-out	+1 day bridge	ties to MB5L	≈$0.05M	modeled (6 × 8 h × $85 × 12)
Total modeled	—	—	**≈$0.6M/yr**	match rate ≥99.5% in 24h = design KPI

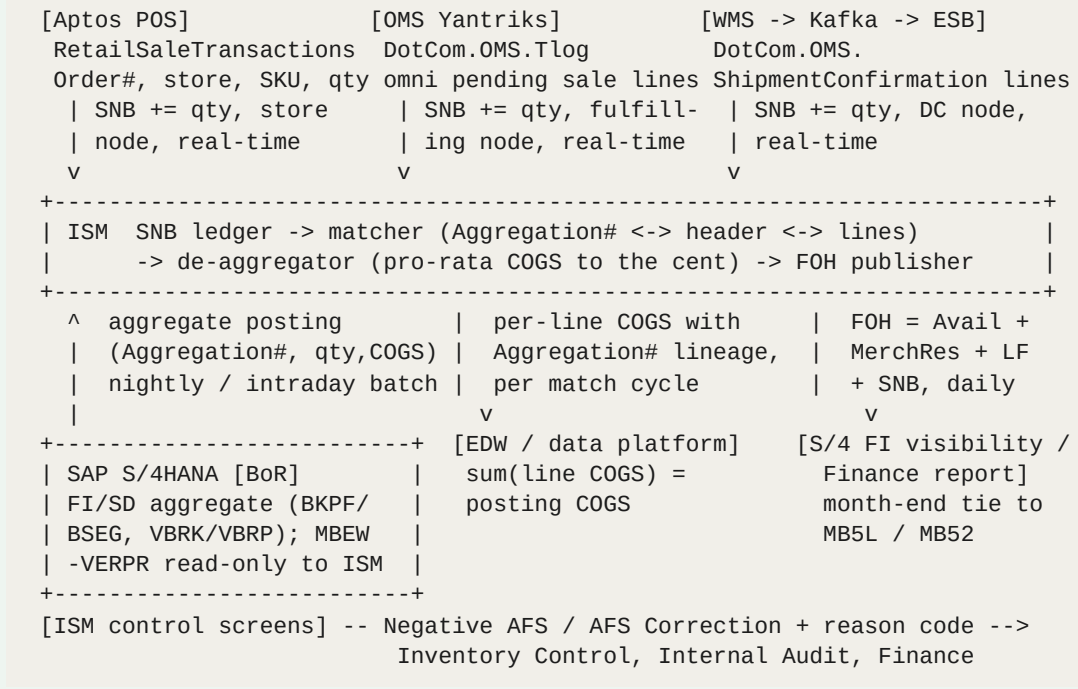
```
cancels avoided = 30M x 0.10% x $15 (CS $5-8 + refund + margin share; all-in $10-20) =
$0.45M
recon labor = 32 h/wk x 52 x $85/h = $0.14M · close = 6 people x 8 h x $85 x 12 = $0.05M
total = ~$0.6M/yr — one FOH is record; every dollar is modeled and labeled
```

If challenged. *"The cancel rate is borrowed from S1 — double-counted?"* — "Different window: S1's rate is the vocabulary problem across node types; this is the posting-lag window inside one node. If the room wants one number, I keep S1's and drop this row — the mechanism still stands on the record." *"32 hours of Finance time — source?"* — "Benchmark, not record. The record is the mechanism: the bridge spreadsheet died when its owner owned the real report. A two-week time study on Inventory Accounting prices it properly."

THE MATH — FULL DERIVATION, STEP BY STEP

Shipped-Not-Billed ledger. For SKU i at node n: SNB_in(t) = Σ sold-or-shipped-not-posted units −
  Σ matched-posting units . Increment on POS flash sale, OMS omni pending sale, WMS ship
  confirm; decrement only on a matched S/4 aggregate posting. De-aggregation (pro-rata COGS).
  Aggregate posting P with total qty q_P and total COGS C_P ; line i with q_i : COGS_i = C_P ×
  q_i / q_P , with Σ_i q_i = q_P and Σ_i COGS_i = C_P (allocation is exact; last line absorbs
  rounding so the sum ties to the cent).
Phantom-availability window. W_i = t_post − t_sale ; exposure per SKU ≈ sales rate × W units.
  The design makes the exposure zero by decrementing AFS at t_sale via SNB instead of at t_post
  .
Safe failure direction. If a posting never matches: SNB stays high AFS = Basis − … − SNB is
  lower than truth under-sell (visible over-ordering by Replenishment), never over-sell.
  Formally: AFS_reported ≤ AFS_true for all unmatched states.
Worked example (illustrative — no canonical drift $ exists; "illustrative, not program
    data").
  Aggregate posting P: 5 lines, q_P = 12 , C_P = $300.00 . Lines q = 3, 2, 4, 1, 2 → COGS =
  75.00, 50.00, 100.00, 25.00, 50.00 (Σ = 300.00). SNB before match = 12 (from POS events);
  after match SNB = 0. Suppose one line (q = 2) arrives late: SNB = 2 after matching 10; AFS
  under-states by 2 units until the line lands; the unmatched-aging report shows it at 24h.
  Sanity: SNB ≥ 0 always; a decrement beyond the matched quantity is rejected.
Reconciliation KPI. match_rate = matched postings / postings received per day (target ≥ 99.5%
  within 24h); SNB_aging_p95 in hours; FOH_ISM − FOH_S4 = 0 at month-end per SKU × node.

SOLUTION FLOW — THE SYSTEM & DATA MAP



SENSITIVITY — LOW / BASE / HIGH

INPUT	LOW	BASE	HIGH
Phantom-cancel rate avoided (modeled)	0.05 pt	0.10 pt	0.15 pt
Cost per cancel (benchmark)	$10	$15	$20
Cancel $ avoided	$0.15M	$0.45M	$0.90M
Bridge hours per week (illustrative)	20	32	48
Loaded analyst rate (benchmark)	$75/h	$85/h	$95/h
Reconciliation $	$0.08M	$0.14M	$0.24M
Close days saved per month	0.5	1	2
Close $	$0.02M	$0.05M	$0.11M
Total modeled $/yr	≈$0.25M	≈$0.6M	≈$1.2M

TOTAL IMPACT STATEMENT — SAY IT LIKE THIS

"Conservatively, about six hundred thousand a year: roughly four hundred fifty thousand of cancels that no longer happen because availability drops at the sale instead of at the posting; about a hundred forty thousand of Finance hours that were going into a bridge spreadsheet; and about fifty thousand a year of month-end close time. What is measured is that Finance and Operations went from two on-hand numbers to one, decremented at sale or ship, with zero incidents and the failure direction pointing at under-sell. Every dollar is my model on an illustrative thirty-million-order scale; the record has no pre-state drift dollar and I am not claiming one."

If challenged — the full answers.

The cancel number is three quarters of the total and you say the record has no drift figure.

Right, so it is the hardest number. It rests on a tenth of a point of orders being cancelled for phantom availability before the fix. What I would measure is the cancel-reason code for out-of-stock-after-order, per week, before and after the May go-live, times Digital's own cost per cancel; whatever that shows replaces my assumption.

Finance hours are soft, nobody logs spreadsheet time.

Agreed. Four people at eight hours a week is what Inventory Accounting described, not what a timesheet says. I would firm it up by counting the manual journal entries and bridge lines per month before and after; that is in the posting log and it is not opinion.

Isn't the safe failure direction just moving cost to Replenishment?

Yes, and I priced it that way on purpose: an unmatched posting under-states AFS and shows up as an aged row Finance clears the same day, so the over-order is small and visible. The measure is the aging report's p95 in hours and the match rate inside twenty-four hours; if match rate holds above ninety-nine and a half, the Replenishment side rounds to zero.

S4 — Sephora S/4HANA: the in-transit promise rule I got wrong — fixed through the signed policy, not a side configuration

Amazon LPs: Are Right, A Lot (the failure question — third) · Customer Obsession (backup) · Earn Trust (owning the miss) · Learn and Be Curious. **Crosswalk:** not in the loop packet — safe anywhere. The designated *policy-failure* story. **Recency:** current role.

Honesty label. The v1 rule was mine; the indictment came from my own program's reports. Lane σ ≈ 0.3 day and the ~0.49-day window are the record's promise math; the record carries no lane counts and no dollar for the conversion lost while I was wrong — anything of that kind is labeled illustrative, and I will not invent the loss figure.

60-SECOND PITCH

"**Situation.** My first ISM ruleset barred every channel from promising units in transit; Aging In-Transit showed dotcom starved of fast movers one day from the building. **Task.** Correct my own policy without weakening oversell protection, through the same three signatories. **Action.** Discovery: conversion-loss evidence from the aging and replenishment reports — that was the entry bar to reopen the policy. Plan & Design: promise math — lane σ ≈ 0.3 day, ~0.49-day window ≈ 95% coverage; Promisable = in-transit × 1[ETA ≤ window] × 1[on-time ≥ floor]; Aging In-Transit as kill switch. Build & Test: back-test against receiving history; regression through the node × channel matrix. Cutover: released on the normal monthly cadence as a signed FSD change, not a side config. Hypercare: kill switch armed; never fired. Lesson: both sides of a rule get a number before it ships. **Result.** Primary metric: sellable units on qualifying in-transit lanes — starved → **restored inside a per-lane promise window, oversell increase = zero**, kill switch never fired. $ impact: ≈$0.5M/yr modeled margin recovered — the record has no dollar for the conversion lost while I was wrong, and I will not invent one."

S — In the first ruleset I authored for the ISM availability model, no channel could promise against units in transit. It felt safe. The Aging In-Transit and Replenishment reports showed dotcom starved of fast-moving units reliably one day from the building, with conversion loss on those SKUs.

T — The policy was mine and I had written it conservatively; the task was to correct it without weakening oversell protection, and to take the miss back through the same signatories.

A — Five gated phases:

1. **Discovery** — built the evidence pack from the Aging In-Transit and Replenishment reports (fast movers reliably one day out, conversion loss), stated that the v1 rule was mine, reframed the problem as promise math rather than "make in-transit count." *Exit bar:* Dotcom, Inventory Planning and I agree the undersell is real and is the rule's doing.
2. **Plan & Design** — wrote the rule — Promisable = In-transit × 1[ETA ≤ window(lane)] × 1[on-time ≥ floor], window per lane from receiving history, Aging In-Transit as kill switch — rejected both blanket options in writing, carried it as an FSD change. *Exit bar:* signed by the same three signatories as v1.
3. **Build & Test** — owned the back-test of the ruleset against receiving history, four regression cells added to the matrix (qualifying lane / non-qualifying lane / aging breach / on-hold receipt), the release note. *Exit bar:* back-test shows no oversell on qualifying lanes; matrix green; QA gate passed.
4. **Cutover & Go-Live** — released on the normal cadence with the lane qualification list published and monitor thresholds set — no hotfix, no side configuration. *Exit bar:* sellable units restored on qualifying lanes; zero oversell traced to the window.
5. **Hypercare / Close** — kill-switch watch (aging per lane), oversell watch, close-out to the signatories with the miss named as mine plus the mechanism note. *Exit bar:* kill switch never fired; lesson institutionalized — the Locus 98% gate (S2) and the 0.5% tolerance (S18) were both set from data before the event.

R — Primary metric: starved lanes → restored inside the window; **oversell increase = 0**; kill switch never fired. $ impact: ≈$0.5M/yr modeled.

IF PROBED — PHASE Q&A (13). Answer the phase asked, then stop.

Discovery — How did you know this was a policy defect and not a data defect?

The reports showed fast movers reliably one day out with conversion loss — the lanes were on time; the rule forbade promising against them. Data was fine; my rule was the cause.

Who did you tell first?

The three signatories of the v1 rule, with my name on the miss, before proposing a fix. Owning it first is what made re-signing possible without a fight.

Plan & Design — What was in your risk register for the fix?

Oversell on an unreliable lane, aging shipments getting promised, on-hold receipts being counted. Each had an automatic mitigation — floor exclusion, kill switch, reserve logic — and a named owner.

How did you handle "don't touch a signed policy"?

By showing the undersell number and the promise math: σ about 0.3 day, a ~0.49-day window buys ~95% coverage. Then I changed the policy through the same signatures rather than a side configuration.

Why reject both blanket answers?

"Always counts" oversells unreliable lanes; "never counts" strands good units. Both sides of a rule get a number now — that was the lesson.

Build & Test — What was your definition of done?

Back-test against receiving history attached, regression cells green for all four cases, QA's gate passed. No back-test, no QA entry.

How did you test it without a bug bash?

A rule change doesn't need a room; it needs history. I back-tested against receiving history first — the step I had skipped in v1 — then ran the matrix.

Cutover — Why the normal release and not a hotfix?

A policy fix belongs in the governed artifact with regression. A hotfix would have made the fix as untraceable as the error.

What were your rollback triggers?

Any oversell traced to the window, or aging above threshold on a qualified lane — rollback is per lane: window to zero. Neither fired.

Hypercare — How did you decide it was closed?

One release cycle with the kill switch silent and no oversell increase, then a one-page close-out to the signatories. The lesson outlived the fix: the 98% robotics gate and the 0.5% tolerance were both set from data before the event.

What stayed on the RAID log after close?

One standing row: the kill-switch threshold per lane, owned by Inventory Control, reviewed on the normal cadence. A rule that qualifies lanes automatically needs one human who reads the aging report every week.

Cross-phase — Which wave did you hold? What did you refuse?

Lanes, not waves — a lane whose on-time rate sat below the floor stayed unpromisable; Dotcom accepted lane-by-lane because the alternative was all-or-nothing. Refused: shipping it as a side configuration. The error was in the policy; the fix had to be too.

Why didn't you catch it in design? · What changed in how you work?

I optimized for the defect I feared (oversell) and never priced the one I couldn't see (undersell). Now both sides of every rule get a number, and I test a ruleset against history before publishing — one week of measurement would have skipped the starvation.

Where this lands at Amazon: inventory in transit between FCs, or inbound from a vendor, is exactly this promising decision at network scale. And the failure answer every Bar Raiser wants: a real policy error, named as mine to the same

signatories, fixed with math, institutionalized as "both sides of a rule get a number."

BUSINESS IMPACT MODEL — KPI TREE, NUMBERS, THE MATH

```
Dotcom margin on fast movers (primary outcome)
+- Conversion recovered $ = starved units x conversion x lost-sale share x unit margin
|    +- Promisable = InTransit x 1[ETA <= w] x 1[OTR >= floor]
|    +- w = z x sigma_lane = 1.645 x 0.3 d ~ 0.49 d  [record]
+- Oversell cost = late units x P(late) x cancel/expedite cost
|    +- OTR floor excludes unreliable lanes · kill switch drops a lane
|         <- aging > k x p90 -> lane out -> planner reviews -> back in (balancing loop)
|    result: oversell increase = 0, kill switch never fired  [record]
+- Governance: FSD change via same 3 signatories, back-test + matrix regression,
     normal release train -> every later rule gets both sides priced (reinforcing loop)
```

DRIVER	BASELINE	AFTER	$/YR	BASIS
Sellable units on qualifying in-transit lanes	starved (excluded)	restored inside window	—	record
Lane σ and promise window	σ ≈ 0.3 d	w ≈ 0.49 d ≈ 95% cover	—	record (promise math)
Oversell increase from the window	—	0	$0	record (kill switch never fired)
Conversion recovered	$0	≈$1,440/day	≈$0.5M	modeled (300 × 0.6 × 0.4 × $20 × 365; inputs illustrative)
Under-sell cost while the v1 rule stood	unknown	—	—	record: no $ in the record; **not claimed**

```
recovered = 300 starved units/day x 60% conversion x 40% lost-sale share x $20 margin
          = $1,440/day -> ~$0.5M/yr (every input illustrative; the mechanism is record)
```

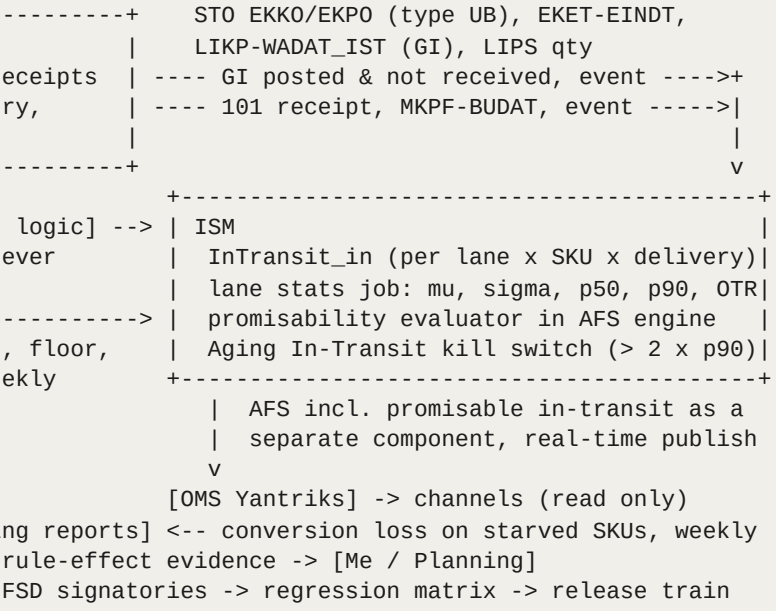
If challenged. *"What did the error cost?"* — "The record has no dollar for it and I won't invent one. What the record has: fast movers starved one day from the building, for weeks, by my rule. The $0.5M/yr is the recovery run-rate, modeled, labeled." *"Why believe the window is safe?"* — "Because the back-test ran before release — the step v1 skipped — and the kill switch plus on-time floor make the failure direction per-lane and reversible. Zero oversell traced to the window is record."

THE MATH — FULL DERIVATION, STEP BY STEP

Promise rule. Promisable_in = InTransit_in × 1[ETA_lane ≤ promise_window_lane] × 1[OTR_lane ≥ floor] , where OTR_lane = on-time rate over the trailing receiving history and floor is the per-lane admission threshold. Window from lane variability (MIT toolkit label: normal approximation on lane transit). For lane transit time L ~ (μ_L, σ_L) , a window that covers a fraction α of arrivals beyond the ETA is w = z_α × σ_L . Canonical: measured lane σ ≈ 0.3 day; z_0.95 = 1.645 → w = 1.645 × 0.3 ≈ 0.49 day ~95% coverage on qualifying lanes. Check: P(L ≤ ETA + 0.49) = Φ(0.49/0.3) = Φ(1.63) ≈ 0.95 . Units: days. What "unpromisable" claims. Setting Promisable = 0 for all in-transit is the limit σ_L → ∞ (or OTR → 0) — a claim of infinite variability. Measured σ contradicts it. Both errors priced (the lesson). Under-sell cost per lane-day ≈ starved units × conversion loss × margin ; over-sell cost ≈ late units × P(late) × (cancel/expedite cost) . Illustrative ("not program data"): a SKU with 100 units one day out, 60% would convert at $20 margin → $1,200/day of under-sell; with OTR 97% the over-sell exposure is 100 × 0.03 × $15 expedite = $45/day. The asymmetry is what the "safe" rule ignored.

Kill switch. Aging_lane = units in transit longer than k × p90_lane ; lane drops out of promisability if Aging_lane > threshold (e.g., any unit > 2 × p90 or OTR falls below floor for the trailing window). Sample-size sanity for OTR (toolkit). To estimate OTR ≈ 0.95 within ±5% at 95% CI: n = z² p(1−p)/e² = 3.84 × 0.0475 / 0.0025 ≈ 73 receipts per lane before the lane qualifies — illustrative.

SOLUTION FLOW — THE SYSTEM & DATA MAP



SENSITIVITY — LOW / BASE / HIGH

INPUT	LOW	BASE	HIGH
Starved fast-mover units/day (illustrative)	150	300	500
Conversion on a shown unit (illustrative)	60%	60%	60%
Lost-sale share (benchmark)	30%	40%	50%
Unit margin (illustrative)	$15	$20	$25
Oversell increase (record)	$0	$0	$0
Total modeled $/yr	≈$0.15M	≈$0.5M	≈$1.4M

TOTAL IMPACT STATEMENT — SAY IT LIKE THIS

"Conservatively, about half a million a year of margin recovered: roughly three hundred fast-mover units a day that were sitting one day from the building and unsellable, of which about sixty percent would have converted and about forty percent of those were truly lost rather than substituted, at about twenty dollars of margin each. What is measured is that those units became sellable inside a back-tested half-day window and that oversell did not move, the kill switch never fired. The dollars are my model on illustrative volumes, and the record carries no figure for what my first rule cost while it stood, so I do not claim one."

If challenged — the full answers.

Three hundred units a day is your number, not the report's.

Correct, it is the hardest input. It assumes the Aging In-Transit report's pattern held across the qualifying lanes every day. To firm it up I would pull the daily count of units within the window on qualifying lanes for the first quarter after the release; that count is in the report, and it replaces my three hundred.

How do you know those sales were lost and not delayed a day?

I do not, which is why I only take forty percent of them, the middle of the published range for a stockout turning into a lost sale. The measure is conversion on the affected SKUs the week before and the week after the release, from Digital's own funnel; if it moved less than that, the number shrinks with it.

You net oversell at zero, is that convenient?

It is the record: zero oversell incidents traced to the window and a kill switch that never fired. But I would keep measuring it, because a lane's on-time rate can decay. The standing check is the aging report per lane against two times its ninetieth percentile, owned by Inventory Control weekly.

S5 — Sephora: the architecture council without owning any system — three decisions and a fence

Amazon LPs: Have Backbone; Disagree & Commit (third) · Earn Trust (influence without authority) · Think Big (architecture governance). **Crosswalk:** not in the loop packet — safe anywhere. **Recency:** current role. *Same program family as S1/S2 — the governance story; don't pair with either for one interviewer.*

Honesty label. I owned none of the systems; the design decision was the deliverable. Zero escalations and the dated gate chain are the record; the dollars are my model of what governance avoided.

60-SECOND PITCH

"**Situation.** The UDC location and inventory model had to clear architecture council — a challenge gate — with ~80 named participants across ~15 functions, and I owned none of those systems. **Task.** Take the design from discovery through council to sign-off and FSD without a surprise escalation. **Action.** Discovery: both inventory books profiled before any design argument; business sessions closed 4/30. Plan & Design: every impacted row pre-wired with signed LOE (7/2); council asked for three separable decisions plus an out-of-scope fence. Build & Test: the 'build' was the pack — architecture, staged timeline, peak-freeze boundary. Cutover: 5/15 challenge → 7/31 sign-off → 8/14 FSD. Hypercare: fence held in writing for the whole program; a second live warehouse became a monitored shadow copy, not an argument. Lesson: a no that lives in a document survives. **Result.** Primary metric: surprise escalations on a 60+ stakeholder design — the norm on prior programs → **zero**; the dated gate chain held. $ impact: ≈$0.5M one-time modeled (avoided rework on ~3 escalations that never happened + six weeks of benefit not deferred + a fenced second reserve engine not built)."

S — The UDC location and inventory model had to clear the architecture council — a challenge gate, not a formality — with a dozen core systems (20+ touched) and ~80 named participants across ~15 functions on the impact table, and I owned none of those systems.

T — Take the design from discovery (kickoff 3/29, sessions 4/30) through the council (5/15) to sign-off (7/31) and FSD (8/14) without a surprise escalation.

A — Five gated phases — here the phases are governance milestones; the cutover is the council decision itself:

1. **Discovery (3/29 → 4/30)** — profiled both books (SKU overlap, order profiles); sessions per function with the profile on the table — "design" deliberately not argued yet; systems-impacted table with a provisional owner on every row; stakeholder map (~80 names / ~15 functions). *Exit bar:* every impacted team in a session, every row owned.
2. **Plan & Design → council (5/15)** — council pack as three separable decisions (three planning identities over one location; January planning / May execution timeline; peak-freeze boundary) plus the out-of-scope fence (reserve math untouched; MCS decommission belongs to Global ERP); every owner pre-briefed privately. *Exit bar:* council challenge held 5/15; decisions taken or dated; no surprise objection.
3. **Build & Test → LOE (7/2)** — one LOE template to one deadline across every impacted owner; each open concern converted into design or a dated RAID item (the second-live-warehouse concern → a monitored shadow copy). *Exit bar:* LOE signed 7/2 by every impacted owner; no owner learns their cost after this date.
4. **Cutover → sign-off (7/31), FSD (8/14)** — assembled the sign-off pack (decisions, fence, LOE, risks); published FSD 8/14 with discovery results attached. *Exit bar:* signed 7/31; FSD 8/14; fence lines in writing.
5. **Hypercare / Close** — enforced the fence (scope requests answered from the pack, in writing); applied the peak-freeze boundary Oct–Dec; handed the program into the build (S2). *Exit bar:* zero surprise escalations for the program's life; both scope lines held in writing.

R — Primary metric: surprise escalations → **0**; gate chain 5/15 → 7/31 → 8/14 held with zero re-litigation. $ impact: ≈$0.5M one-time modeled.

IF PROBED — PHASE Q&A (13). Answer the phase asked, then stop.

Discovery — How did you know discovery was done? Why make it a data exercise?

Every impacted team had been in a session on its own profiled data; every row had a provisional owner; sessions closed 4/30. Profiling is disarming — it is their numbers, not my opinion; SKU overlap and order profiles settled arguments before they started.

Plan & Design — What was in your risk register at the council?

Surprise LOE in the room, a second live warehouse becoming a second truth, and scope re-litigated after sign-off. Mitigations: private pre-wire with a 7/2 LOE deadline, the monitored shadow copy, the fence.

How did you handle a partner who objected at the review?

The "second truth" objection became the design. When a concern is legitimate, converting it costs less than winning it.

Why exactly three decisions?

Separability. Architecture, staged timeline, freeze boundary — a no on any one leaves the others standing. A single "approve the design" would have been all-or-nothing.

Build (LOE) — How did you get sixty-plus people to sign LOE you didn't control?

One template, one deadline, one private conversation per owner before the council so nobody learned their cost in the room. Signatures landed 7/2.

What was your definition of done for a row?

Owner named, LOE signed, concern converted to design or dated. A row with an open concern did not go into the sign-off pack.

Cutover — What would have made you delay the 7/31 sign-off?

An open row. There were none — which is what the pre-wire buys.

What was your rollback if the council said no?

Separability. A no on the timeline would have sent us back with the architecture intact; nothing had been configured yet by design — policy before configuration.

Hypercare — How did you keep scope from creeping back?

The fence lived in the signed pack; every later request was answered from it in writing. A no in a document survives; a no in a meeting gets re-litigated weekly.

What did "hypercare" mean for a governance program?

Answering every scope request from the signed pack, in writing, for the program's life — and applying the peak-freeze boundary Oct–Dec as the ruling. Zero surprise escalations was the exit metric, and it held.

Cross-phase — What did you escalate? What did you refuse?

Escalated nothing — the pre-wire is the escalation-prevention mechanism; the freeze boundary went to the council as a decision. Refused: retuning reserve math and folding in the MCS decommission. Both refusals written, signed, held.

What if the council had said no?

The three decisions were separable — a no on the timeline would not have unwound the architecture. That's why I asked for three, not one.

Where this lands at Amazon: influence without authority is the L6 job description. The mechanism transfers whole — pre-wire every owner's cost before the review, ask the forum for separable decisions rather than a blessing, fence what you will not do, and answer scope creep from the signed document.

BUSINESS IMPACT MODEL — KPI TREE, NUMBERS, THE MATH

```
Program delivered on the dated gate chain, zero surprise escalations
  (record: council 5/15 -> sign-off 7/31 -> FSD 8/14)
+- Avoided rework $ = escalations avoided x rework weeks x team $/wk
|    +- pre-wire coverage = rows with signed LOE / rows = 100%
|    +- 3 separable decisions: a no on one cannot unwind all
+- Schedule protection $ = slip weeks avoided x monthly benefit
+- Fenced scope $ = second reserve engine not built
     <- requests answered from the pack in writing -> fewer requests next cycle (loop)
```

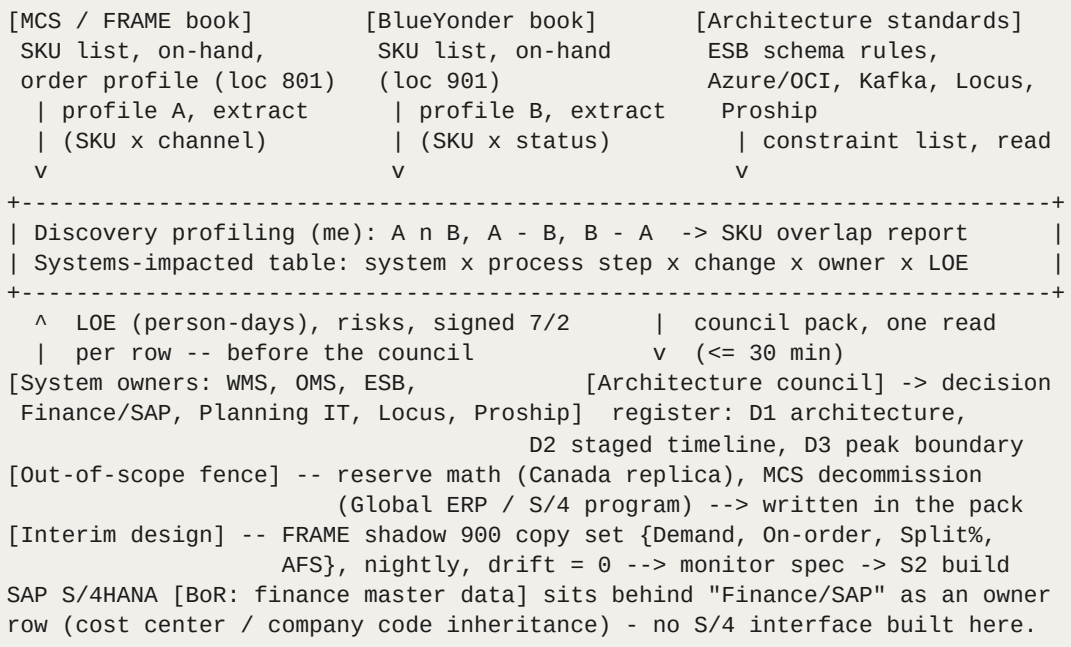
DRIVER	BASELINE	AFTER	$	BASIS
Surprise escalations (60+ stakeholder design)	prior norm ~3	0	—	record; ~3 is industry benchmark (2–4)
Pre-wire coverage before council	partial	100% by 7/2	—	record
Avoided rework	—	—	≈$0.3M	modeled (3 × 960 h × $110)
Slip weeks avoided	~6 wks	0	≈$0.1M	modeled (6/52 × S2's ≈$0.9M labor value)
Second reserve engine (fenced, not built)	proposed	not built	≈$0.1M	modeled (2 eng × 12 wks × 40 h × $110)
Total modeled	—	—	**≈$0.5M one-time**	fence breaches in the log: 0 (record)

If challenged. *"You're pricing escalations that never happened."* — "Correct, and labeled: the ~3 is a benchmark for designs this size, the 960 hours per escalation is illustrative. The record is zero escalations against a norm everyone in that building recognized, and a gate chain that held to the day. If the room won't price prevention, keep the record and drop the dollars — the mechanism is the point."

THE MATH — FULL DERIVATION, STEP BY STEP

```
Decision decomposition. A council "review" is one binary outcome on a bundle; three
    separable
  decisions D = {architecture, timeline, peak boundary} give 2³ = 8 outcomes, of which any
  partial approval preserves progress. Expected recoverable progress: with independent
approval
  probabilities p_a, p_t, p_b , E[decisions approved] = p_a + p_t + p_b vs a bundle's p_a ×
    p_t
  × p_b . Illustrative: 0.9 each → 2.7 of 3 expected vs 0.73 chance of the whole bundle.
Pre-wiring coverage. coverage = rows with signed LOE before council / total impacted rows ;
  target 100% (LOE signed 7/2 ahead of 7/31 sign-off). Surprise escalations = rows whose
owner
  first learned cost in the room ; result 0.
Gate chain. Kickoff 3/29 → business sessions 4/30 → council challenge 5/15 → LOE 7/2 → sign-
off
  7/31 → FSD 8/14: 138 days discovery-to-spec; council-to-sign-off 77 days with zero
  re-litigation.
Impact table size. ~80 named participants across ~15 functions, a dozen core systems (20+
  touched)  impacted-row grain = system × process step; LOE per row in person-days, summed
per
  system owner for the council pack. Two-truths risk (design, not argument). During the
interim,
  two live books create disagreement probability per SKU per day; the shadow-900 copy set
with
  |AFS_900 − AFS_801| = 0 monitor makes disagreement a detectable Sev rather than a latent
truth
  conflict.
```

SOLUTION FLOW — THE SYSTEM & DATA MAP



SENSITIVITY — LOW / BASE / HIGH

INPUT	LOW	BASE	HIGH
Escalations avoided (benchmark)	2	3	4

INPUT	LOW	BASE	HIGH
Rework weeks per escalation (illustrative)	1	2	3
Loaded blended rate (benchmark)	$90/h	$110/h	$130/h
Avoided rework $	$0.09M	$0.32M	$0.75M
Slip weeks avoided	4	6	8
Schedule protection $	$0.07M	$0.10M	$0.14M
Fenced engine build $	$0.09M	$0.11M	$0.12M
Total modeled $/yr	≈$0.25M one-time	≈$0.5M one-time	≈$1.0M one-time

TOTAL IMPACT STATEMENT — SAY IT LIKE THIS

"Conservatively, about half a million, one time, of cost this governance avoided: roughly three hundred thousand of rework on the two or three escalations that never happened because every owner had signed their effort before the council; about a hundred thousand of the comingle benefit arriving six weeks sooner than it would have after those re-litigations; and about a hundred thousand for the second reserve engine the fence kept out of the release. What is measured is zero surprise escalations and the dated chain holding, council to sign-off to spec. The dollars are my model of what did not happen, so I hold them loosely and I say so."

If challenged — the full answers.

You are pricing things that did not happen.

Yes, and that is the hardest part of any governance number. It rests on a norm of two to four re-litigations for a design with sixty-plus stakeholders and two weeks of core-team rework each. To firm it up I would take the prior two programs of this size at Sephora and count the scope re-openings and the weeks each cost from their own RAID logs; that replaces my norm with theirs.

A hundred thousand of schedule protection borrows S2's number.

It does, deliberately, and it is labelled that way. The assumption is six weeks of slip avoided against a benefit worth about seventeen thousand a week. The measure is simple: the FSD landed on the fourteenth of August against the plan, so slip avoided is the plan minus the actual, and the weekly value comes from the DC's measured efficiency once it is priced.

The fence saving is just work you chose not to do.

Right, so I count it once and small. The assumption is two engineers for twelve weeks. What would firm it up is the actual estimate the engineering team gave when they proposed the US-tuned engine; if their number was different, mine changes, but the point stands that a second engine was a second defect class in the same release.

S6 — Elementum: eight SAP data domains into one real-time supply/demand view — a shortage visible a lead-time early

Amazon LPs: Deliver Results (third) · Earn Trust (the two-signal release rule) · Dive Deep · Think Big (one view over eight domains). **Crosswalk:** loop packet S5 — consistent (70→85%, $2M, both-errors rule). *Different program from S7 (the six-site ingest) and S8 (the $1.5M lanes) — never merge the dollars or the timelines.*

Honesty label. I owned the definitions, the data-integrity program and the delivery; the client's SAP teams and our integration engineers built the feeds; planners owned the forecast — I owned its measurement and the release rule. I have not owned a demand forecast end-to-end and won't claim it.

60-SECOND PITCH

"**Situation.** A six-site manufacturer learned about component shortages when the line stopped; eight SAP domains never met, so 'which orders are at risk' had no data path. **Task.** I took the definitions, the data-integrity program, and the delivery. **Action.** Discovery: profiled all eight domains; signed gap findings. Plan & Design: supply = on-hand + production plan + released planned orders only; grain SKU × location × date; shortage alert = days-of-cover vs lane lead-time p90. Build & Test: inventory and production first — numbers planners already believed; daily reconciliation to MD04/COOIS/VA05/MB52. Cutover: shortage-alert product live in 3 months behind data-quality gates; nobody mandated off a spreadsheet. Hypercare: safety-stock release only where forecast error AND lead-time error both improved, signed with planning and Finance. Lesson: define supply honestly before any screen. **Result.** Primary metric: forecast accuracy on scoped class-A SKUs — **70% → 85%**. $ impact: $2M of safety stock released under the both-errors-improved rule (record, one-time working capital), with ≈$0.8M/yr modeled recurring value around it — separate from the $1.5M lane program; never add them."

S — A strategic manufacturing customer with six sites (mix of make-to-order and make-to-stock): planners learned about a component shortage when the line stopped; S&OP ran off private spreadsheets; shipment data was 60% complete; supplier, material, BOM, inventory, production, transportation, sales-order and planning data all lived in SAP and site systems but never met. Leadership's question — "which customer orders are at risk this quarter?" — had no data path.

T — Nobody owned the seam; there was no spec, no agreed grain, no definition of "supply," no data-quality bar. I chose all four, as owner of the definitions, the data-integrity program and the delivery.

A — Five gated phases (canonical plan: Discovery 4–6 wks → master-data foundation 4–6 wks parallel → domain waves 8–12 wks → views live 4 wks → scale & hypercare 4 wks; the 3-month figure is the shortage-alert product; the six-site ingest is S7's separate 5-month program):

1. **Discovery (4–6 wks)** — profiled every SAP domain (completeness, latency, joinability) to my own gap templates; wrote the definitions memo (grain SKU × location × date; supply = on-hand + production plan + *released* planned orders only) and the metrics tree. When Planning asked to count unfirmed MRP orders as supply because it looked better, I refused: unfirmed orders breathe every run. *Exit bar:* signed gap findings per domain; scope frozen in writing; planners agree the supply definition before any screen.
2. **Plan & Design (parallel)** — master-data foundation plan; MPN mapping worklist with its coverage gate; BOM extract and phantom clean-up plan; domain-wave sequence by trust; DQ scorecard spec; the 10-step onboarding playbook. *Exit bar:* master-data gates green; unmapped-MPN list under threshold with a named owner; wave order signed by domain owners.
3. **Build & Test (8–12 wks)** — domain waves behind published DQ gates; shortage-alert rule specified (days-of-cover vs lane LT p90); daily reconciliation to MD04/COOIS/VA05/MB52; weekly root-cause loop with false alarms worked as defects. *Exit bar:* per-domain completeness ≥ gate; reconciliation passing daily; false alarms trending down; pilot planners using the alert.

4. **Cutover & Go-Live (4 wks)** — views live for pilot planners with the at-risk list replacing the Monday spreadsheet; go/no-go read from the scorecards. *Exit bar:* planners answer "where is my product" from the platform in the live S&OP meeting — not from a spreadsheet.
5. **Hypercare / Close (4 wks)** — exception workbench staffed; weekly root-cause reviews; the safety-stock release rule signed with Planning and Finance. *Exit bar:* blockers/day under threshold two consecutive weeks; release applied only where forecast AND lead-time error both improved.

R — Primary metric: forecast accuracy **70%** → **85%** on scoped class-A SKUs (record). $2M safety stock released under the both-errors rule (record, one-time); planner effort −50%; shortage alert live in 3 months; adoption 8×.

IF PROBED — PHASE Q&A (14). Answer the phase asked, then stop.

Discovery — How did you know discovery was done?

Every domain had a signed gap finding on its own profiled data and the metrics tree was agreed. Four to six weeks; scope frozen in writing at the exit.

Who wrote the success criteria and who signed?

I wrote the metrics tree — OTIF, inventory dollars, MAPE/bias, lane lead-time accuracy, completeness, latency, adoption — and the domain owners with sign-off authority signed their gates.

What did you define before any screen?

Grain — SKU × location × date — and supply: on-hand plus production plan plus released planned orders only. Planners accepted the definition first; the view was built to it.

Plan & Design — What was in your risk register at sign-off?

MPN mapping never done, carrier EDI capability spread, planner trust, and SAP extract load — carried openly with a mitigation and owner each. MPN became a standing metric rather than a task.

How did you sequence the waves?

By trust: inventory and production first because planners already believed those numbers, then purchasing/ASN, then transportation with the best lanes first. Class-A SKUs before the tail.

Build & Test — What was your definition of done for a feed?

Its DQ scorecard — completeness, latency, accuracy vs source, joinability — above the published gate, and daily reconciliation passing. Below the gate, the feed got fixed, not automated.

How did you treat false alarms?

As defects: classified weekly to a root cause — mapping gap, master-data defect, carrier capability, pipeline bug — owned and burned down. Adoption followed planners watching their reported defects die.

Who could fail a wave?

The domain owner — they held sign-off authority over their gate. I designed it so the business could say no to its own data.

Cutover — What was the go-live gate for the views?

Planners answering "where is my product" from the platform in the live S&OP meeting rather than from a spreadsheet. The shortage alert was live in three months.

What was your rollback?

The spreadsheet — still there, never banned. Reversibility was free because I never mandated anyone off it.

Hypercare — How did you decide hypercare was over? What did it hand over?

Blockers/day under threshold two consecutive weeks, adoption targets met, the exception workbench staffed. Handover: the weekly root-cause review to domain owners, the onboarding playbook (profile → map → test file → parallel run → scorecard → live), and the MPN exception queue as a standing metric.

Cross-phase — Where did you trade scope for date? What did you escalate?

The lane tail and the MPN tail — deferred behind the same gates, priced in coverage, not count. Escalated the MPN worklist gating longer than planned — to the sponsor as a re-sequenced date with the coverage number attached.

Who owned the forecast?

> The planners. I owned the accuracy measurement (MAPE/bias per SKU family), input integrity and the release rule. I have not owned a demand forecast end-to-end and won't claim it.
>
> **Why released planned orders only? · Do differently?**
>
> Unfirmed MRP orders change every run; planning against them amplifies noise and inflates supply. Differently: start the MPN-mapping worklist on day one — it is never "done" and gated more than I planned.

Where this lands at Amazon: this is supply/demand visibility built honestly — define supply before the screen, measure MAPE and bias at the decision grain, alert a lead-time early, and release buffer only when both error signals improve. The two-signal release rule is the part most rooms have never seen.

BUSINESS IMPACT MODEL — KPI TREE, NUMBERS, THE MATH

```
Customer orders at risk this quarter -> answered a lead-time early
+- Inventory $ (stock): $2M safety stock released (record)
|    +- forecast error 30% -> 15% (accuracy 70 -> 85%, record)
|    +- lead-time error (lane p90 measured, not brochure)
|    +- release rule: BOTH errors improved -> SS = z*sigma*sqrt(L) falls
+- Shortage response (flow): alert when DoC <= lane LT p90
|    +- fix window = LT p90 - DoC: days to re-plan instead of fly
+- Planner effort -50% (record) -> hours to root cause, not reconciliation
+- Data-integrity loop: daily recon -> defects burned down -> planner trust (+)
     -> adoption 8x -> more domains feed the peg -> errors fall further (+)
```

DRIVER	BASELINE	AFTER	$/YR	BASIS
Forecast accuracy, class-A SKUs	70%	85%	—	record
Safety stock released (one-time)	—	$2M out	—	record
Carry on released $2M	tied up	free	$0.2M	modeled ($2M × 10% WACC; benchmark 8–12%)
Expedites avoided via the alert	~150 events/yr	~60 pre-empted	$0.3M	modeled (60 × $5K; premium benchmark $3–8K)
Planner effort on reconciliation	12 × 40%	12 × 20%	$0.3M	modeled (2.4 FTE × $120K; −50% is record)
Total modeled recurring	—	—	**≈$0.8M**	around the $2M one-time record; never added to S8's $1.5M

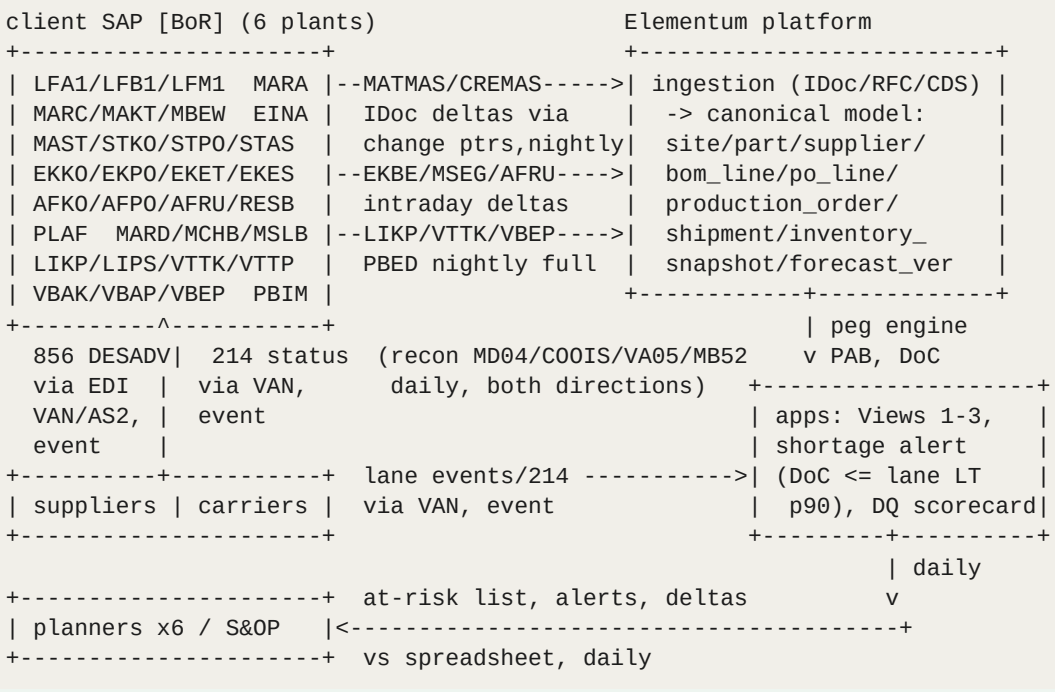
```
carry = $2M x 10% = $0.2M/yr · expedites = 150 events x 40% caught early x $5K = $0.3M/yr
planner capacity = 12 planners x 40% recon share x -50% = 2.4 FTE x $120K = $0.3M/yr
  (capacity redeployed to root cause, not headcount removed)
total modeled recurring = ~$0.8M/yr; range $0.5-1.2M (WACC 8-12%, premium $3-8K)
```

If challenged. *"The $2M — measured how?"* — "It's the record: the release Finance signed, applied only on SKU families where forecast error and lead-time error had both demonstrably improved. That rule is why the release survived audit — capital came out on evidence, not on a better-looking average." *"150 shortage events?"* — "Illustrative volume — six sites at roughly 25 escalations a year. The record is the mechanism: the alert fires at days-of-cover ≤ lane p90, which is the last moment a re-plan is cheaper than a flight."

THE MATH — FULL DERIVATION, STEP BY STEP

Supply and demand at grain SKU × location × date. supply(t) = on_hand + Σ in-transit arriving ≤
t + Σ production finishing ≤ t + Σ PO lines landing ≤ t + Σ RELEASED planned orders finishing
≤ t (unfirmed MRP planned orders excluded). demand(t) = Σ sales-order schedule lines due ≤ t +
Σ DC replenishment due ≤ t + forecast net of orders (outside the frozen horizon only) .
PAB(t)
= supply(t) − demand(t) ; a negative PAB is a dated shortage.
Multi-level BOM explosion. demand(c) = Σ_p demand(p) × qty_per(p,c) / (1 − scrap_pc) , recursed
by low-level code; where-used is the inverse map c → {p} .
Shortage alert (rule, not report). DoC_c = PAB-positive horizon in days (days until PAB(c) < 0);
alert if DoC_c ≤ LT_p90(lane_c) — a lead-time early. Illustrative: component with on-hand
1,200, consumption 100/day → DoC = 12 days; lane p90 = 14 days alert now (fix window 12 − 14
< 0: expedite or re-promise); with p90 = 9 days no alert (3-day slack).
Forecast accuracy. MAPE = (1/n) Σ |A_t − F_t| / A_t over periods with A_t > 0 ; WMAPE = Σ|A−F| /
ΣA ; bias = Σ(F − A) / ΣA . Canonical: accuracy (1 − MAPE) 70% → 85% on scoped class-A SKUs
(error 30% → 15%, halved). Illustrative check: 4 weeks A = 100, 120, 80, 100; F = 130, 110,
100, 90 → |e| = 30, 10, 20, 10 → MAPE = (0.30 + 0.083 + 0.25 + 0.10)/4 = 18.3%; WMAPE = 70/400
= 17.5%; bias = (430 − 400)/400 = +7.5% (over-forecasting).
Release rule (safety stock). Release buffer for SKU i only if MAPE_i(after) < MAPE_i(before) and
LTacc_i(after) > LTacc_i(before) ; released amount from SS = z·σ_d·√L (demand-driven) or
z·d·σ_L (lead-time-driven), toolkit form. Illustrative: d = 100/day, σ_L 4 → 2 days, z = 2.33
(99%) SS 932 → 466 units released 466. Canonical program total: $2M released under the rule
(never added to S8's $1.5M).
Worked peg (v105 §5.12, illustrative). ROUTER-100 at DC-East: on-hand 400; in-transit 600 lands
day 5; production 800 day 9. Demand 500 day 3, 700 day 7, 400 day 10. PAB(3) = 400 − 500 =
−100; PAB(7) = 1,000 − 1,200 = −200 (day-9 supply is too late; fix window 2 days); PAB(10) =
1,800 − 1,600 = +200 .

SOLUTION FLOW — THE SYSTEM & DATA MAP



SENSITIVITY — LOW / BASE / HIGH

INPUT	LOW	BASE	HIGH
Cost of capital (benchmark)	8% → $0.16M	10% → $0.20M	12% → $0.24M
Expedites avoided (illustrative)	45	60	75
Expedite premium (benchmark)	$3K → $0.14M	$5K → $0.30M	$8K → $0.60M
Loaded planner cost (benchmark)	$100K → $0.24M	$120K → $0.29M	$140K → $0.34M
Total modeled $/yr	≈$0.5M	≈$0.8M	≈$1.2M

TOTAL IMPACT STATEMENT — SAY IT LIKE THIS

"The measured number is two million dollars of safety stock released, and that is record, signed with planning and Finance under the both-errors-improved rule. Around it I model roughly eight hundred thousand a year of recurring value: about two hundred thousand in carry cost on the capital that came out, about three hundred thousand in expedite premium the shortage alert pre-empts because it fires a lead-time early instead of at line-down, and about three hundred thousand of planner capacity from the fifty percent effort cut. The two million and the fifty percent are measured; the three recurring pieces are my model on benchmark rates, and I keep them separate from the one-and-a-half million in the lane program."

If challenged — the full answers.

The expedite figure is the softest

It rests on about sixty expedites a year pre-empted at a five thousand dollar premium, both illustrative; to firm it I would pull the expedited-freight ledger for the six sites for the twelve months before and after the alert went live and count the premium directly.

Planner capacity is not cash

Agreed, it is two point four FTE of time redeployed, not headcount removed; the fifty percent is measured, the forty percent share of the week is my assumption, and a two-week time study on the S&OP prep cycle would replace it.

Why not claim the whole two million every year

Because a release is a one-time working-capital event; the recurring value is the carry at eight to twelve percent, and I would rather say two hundred thousand a year honestly than two million a year wrongly.

S7 — Elementum: making a contract manufacturer's BOM safe to plan on — phantoms, revision overlaps, one supply equation

Amazon LPs: Learn and Be Curious (primary — learned CM data by math) · Dive Deep · Earn Trust (external partner).
Crosswalk: loop packet S6 — consistent. *Guardrails: "six sites," never "6 NA sites"; no "500+ product families"; 60→95% belongs to S9; no tolerance value — the record doesn't carry one.*

Honesty label. I owned the ingest requirements, the reconciliation gate, the wave plan and the weekly Flextronics call; engineers built the normalization layer and quarantine to the spec; the commercial relationship belonged to the customer's procurement — my leverage was evidence. The $1.5M/580K belongs to the p99 program this backbone enabled (S8).

60-SECOND PITCH

"**Situation.** Flextronics supplied the BOM our component-demand visibility depended on; phantoms double-time demand if ingested as stocked, and overlapping revisions rewrite history if you auto-load one. **Task.** I owned ingest requirements, the reconciliation gate, the wave plan, and the weekly Flextronics call. **Action.** Discovery: sampled finished goods, exploded BOMs, compared component counts to Flextronics MRP — phantoms found by math. Plan & Design: phantom flatten with an explicit marker; effective-date quarantine, human disposition, never auto-load; supply equation signed. Build & Test: mapping tables owned by named site super-users; per-site reconciliation is the go-live gate. Cutover: three waves / six sites / 5 months; wave 3's hostile ERP used snapshot-diffing — 6 weeks vs a 3-week average, priced up front. Hypercare: the weekly call became standing, framed as their protection. Lesson: reconcile to the source's math, not its documentation. **Result.** Primary metric: contract-manufacturer sites with planning-grade BOM/supply visibility — **0 → 6 in 3 waves / 5 months, zero auto-loaded revision overlaps**. $ impact: ≈$0.3M/yr modeled — the six sites and zero overlaps are measured; the dollars are my model; the $1.5M belongs to S8."

S — Same manufacturer; Flextronics supplied the BOM and planner data that component-demand visibility depended on. Two traps documentation never confesses: phantom assemblies (logical groupings that pass demand through without stocking — ingest them as stocked and component need-dates get double-timed) and revision overlaps (two BOM revisions partially true for weeks — auto-load one and you rewrite demand history). Six site systems carried six definitions of yield, lead time and order status; MTO and MTS had different ATP semantics.

T — Own the ingest requirements, the reconciliation gate, the wave plan — and the chair of the weekly Flextronics call.

A — Five gated phases (canonical: wave 1 = two cleanest high-volume sites; wave 2 = three sites on parameterized maps; wave 3 = the hostile legacy ERP, flat files, no change-data-capture):

1. **Discovery** — sampled finished goods and exploded them against Flextronics MRP (the phantom signature found by math); catalogued the six-site definition inventory (yield, lead time, order-status semantics); wrote the MTO/MTS tagging rule and a discovery memo naming the two traps. *Exit bar:* Flextronics engineering confirms the phantom signature; six definitions catalogued; wave order proposed.
2. **Plan & Design** — supply equation (on-hand + production plan + released planned orders); ingest requirements (phantom flatten with marker; effective-date quarantine queue with human disposition — never auto-load); the reconciliation report as the per-site gate; the 2/3/1 wave plan; mapping-table ownership model; weekly call charter. *Exit bar:* requirements signed by customer planning; Flextronics staffs the weekly call; wave economics accepted.
3. **Build & Test** — held engineers to the spec for normalization and quarantine; reconciliation report per site; parameterized mapping templates for wave 2; snapshot-diff design for wave 3 (consecutive full extracts, checksums, a completeness flag on every extract). *Exit bar:* wave-1 reconciliation within tolerance; zero auto-loaded overlaps in test; completeness flag on every extract.

4. **Cutover & Go-Live (per site)** — each site's go-live ran off its matched reconciliation report, mapping tables handed to a named super-user, disposition queue live. The wave-3 duration went to customer leadership up front as a priced wait. *Exit bar:* the reconciliation report was the gate, not a date on a slide — six sites live over three waves; hardest site 6 weeks vs the 3-week average, absorbed in plan.
5. **Hypercare / Close** — weekly Flextronics DQ/disposition call left standing (it had become theirs — the forum that protected their feed's reputation); the wave-2 yield gap fixed in the mapping row, model untouched; completeness flag written as a rule in the ingest layer. *Exit bar:* zero auto-loaded revision overlaps; phantom double-timing eliminated at source; backbone handed to the 200-lane program (S8).

R — Primary metric: CM sites with planning-grade BOM **0** → **6** in 3 waves / 5 months; zero auto-loaded overlaps; hardest site in plan. $ impact: ≈$0.3M/yr modeled.

IF PROBED — PHASE Q&A (13). Answer the phase asked, then stop.

Discovery — How did you find the phantoms?

By math, not documentation: sampled finished goods, exploded their BOMs in our model, compared component counts to Flextronics' own MRP. The mismatches were the phantom signature; their engineers confirmed.

How did you know discovery was done?

The two traps — phantoms and revision overlaps — named with evidence, the six sites' definitions catalogued, and Flextronics confirming the signature.

Plan & Design — What was in your risk register at sign-off?

Phantoms ingested as stocked, revision overlaps auto-loaded, and partial extracts consumed. The first two had mechanisms from day one — marker and quarantine; the third got its completeness flag properly at wave 3, my do-differently.

How did you get an external supplier to staff a standing call?

By making the call theirs: it protected Flextronics from blame for misses their feed didn't cause. Evidence was my only leverage — I owned no commercial relationship.

What was the entry bar for a site?

Extracts flowing, a named super-user for every mapping table, and the reconciliation report built for that site. No super-user, no wave slot.

Build & Test — What was your definition of done for a site?

Reconciliation report matched within the published tolerance — the report was the gate, not a date on a slide. I won't quote the tolerance value; the record doesn't carry it.

Describe your bug-bash equivalent.

The wave-2 yield gap: several points between their report and ours. Planners, Flextronics and engineers in one session traced it to engineering-sample pulls netted on their side versus gross started-units on ours — fixed in one mapping row.

Cutover — Which wave did you hold and how did the business take it?

Wave 3 ran six weeks against a three-week average because the no-CDC ERP needed snapshot-diffing. I had priced it up front, so leadership took it as the plan, not a slip.

What did each wave deliver?

W1: the normalization layer, quarantine, and the first two sites. W2: parameterized mapping tables and three more sites. W3: snapshot-diffing and the hardest site. Each wave's mapping tables stayed with a named super-user.

Hypercare — How did you decide it was closed? What did you leave behind?

Six sites live behind matched reports, zero auto-loaded overlaps, the weekly call running without me. Left behind: a named super-user per mapping table, the quarantine queue with human disposition, the completeness-flag contract, the call as their forum. The model stayed standard; the mappings stayed theirs.

Cross-phase — Where did you trade scope for date? What did you refuse?

Nowhere on quality; I traded speed — three waves over five months instead of a big bang, so one surprise could not spend six sites. Refused: auto-loading revisions, forcing one lead-time semantics, going live on a date instead of a matched report.

> **Why not force one lead-time semantics across sites?**
> ATP and lead-time meaning genuinely differ between MTO and MTS; one forced semantics lies about both.
>
> **Do differently?**
> Put the completeness flag on the extract from wave 1 — the unflagged partial extract is the failure that looks like thousands of phantom deletions.

Where this lands at Amazon: vendor and manufacturer master data at scale carries the same two traps — a phantom level in a kit BOM and an effectivity overlap on an engineering change both manufacture phantom demand. The transferable rules: detect by the source's own math, quarantine with a human disposition, and let the reconciliation report — not a date — open each wave.

BUSINESS IMPACT MODEL — KPI TREE, NUMBERS, THE MATH

```
Planning-grade component demand at six CM sites (0 -> 6, record)
+- Demand-timing integrity (stock of trust)
|    +- phantom double-timing: components pulled early AND twice
|    |     -> false shortages -> expedites acted on (modeled $)
|    +- revision overlaps auto-loaded: 0 (record) -> no rewritten demand history
+- Reconciliation labor (flow): manual BOM-vs-MRP per site -> disposition queue only
+- Backbone enabled: one sigma per lane -> S8 p99 program ($1.5M is S8's)
+- Gate loop: recon in tolerance -> site live -> super-user owns mapping row ->
     defects fixed as data -> next wave mostly configuration -> 5 months for 6 sites
```

DRIVER	BASELINE	AFTER	$/YR	BASIS
CM sites with planning-grade BOM	0	6 (+6 in 5 months)	—	record
Auto-loaded revision overlaps	uncontrolled	0	—	record
Manual BOM reconciliation labor	6 hrs/wk × 6 sites	1.5 hrs/wk × 6	$0.08M	modeled (1,300 hrs × $60)
Expedites acted on false shortages	~72/yr	~0	$0.2M	modeled (72 × $3K premium)
Overlap rework avoided	~24 overlaps/yr	dispositioned	$0.02M	modeled (24 × 16 h × $60)
Total modeled	—	—	**≈$0.3M**	record has no $ for S7; $1.5M is S8's

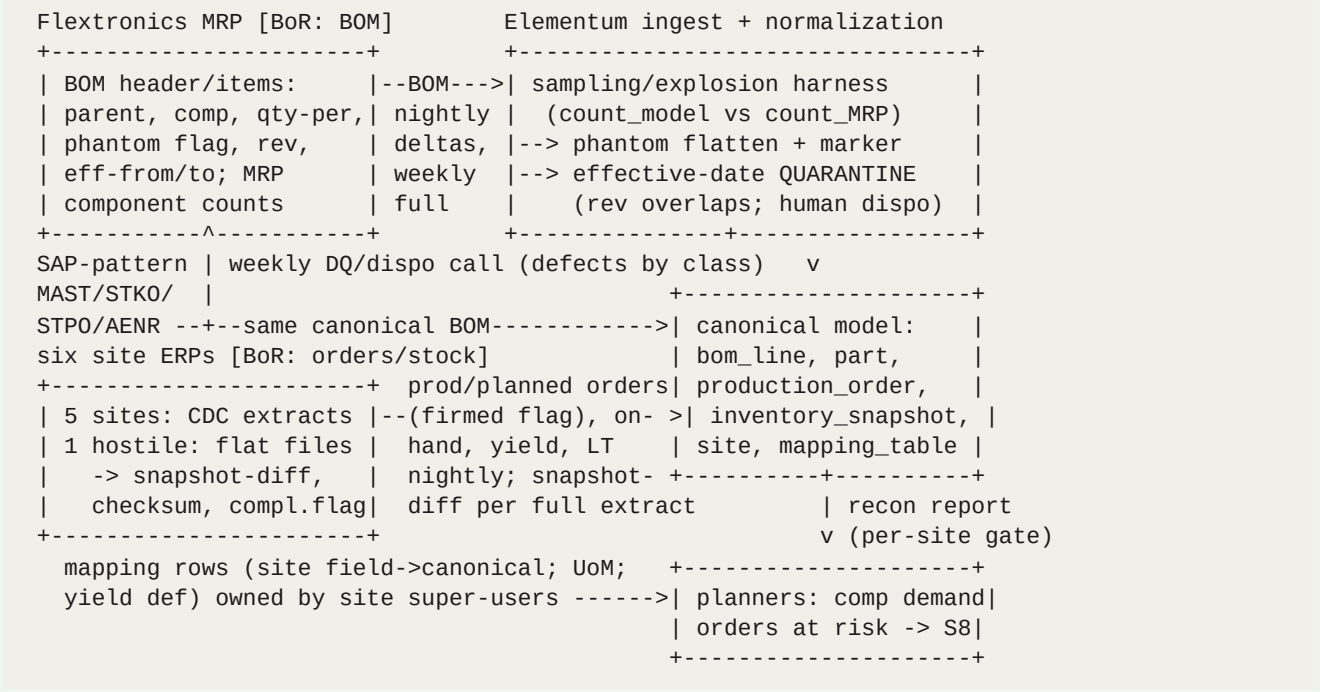
```
recon labor = 6 sites x (6 - 1.5) h/wk x 48 wk = ~1,300 h x $60 = $0.08M/yr
false shortages = 6 sites x ~5/month x 12 = 360/yr; acted on before recheck 20% = 72
expedites = 72 x $3K = $0.2M/yr (premium benchmark $2-5K)
overlap rework = 24 x 16 h x $60 = $0.02M/yr (quarantine makes the error 0 by construction)
```

If challenged. *"360 false shortages — source?"* — "Illustrative volume at the 'phantom shows a shortage of 100' pattern, scaled. The record is the mechanism and the zero: phantom double-timing eliminated at source, zero auto-loaded overlaps ever. The dollars are deliberately small because the big dollar — the $1.5M release — belongs to the program this backbone enabled, and I won't double-count it."

THE MATH — FULL DERIVATION, STEP BY STEP

Phantom double-timing. For finished good FG with lead time LT_FG and phantom sub-assembly SA (special procurement 50, never stocked) with nominal LT_SA : correct component need-date = due_FG − LT_FG (pass-through). Naive ingest (SA as stocked) yields due_FG − LT_FG − LT_SA and a second requirement at the SA level
demand offset twice and counted twice. Detection: count_model(c | FG) ≠ count_MRP(c | FG) systematically for components under SA.
Worked example (Rune §7 tree, illustrative quantities). FG-1000 → SA-200 (phantom) → COMP-A ×2, COMP-B ×1; FG-1000 → COMP-C ×4. Order: 100 FG due day 20; LT_FG = 5, nominal LT_SA = 3. Correct: COMP-A 200 due day 15; COMP-B 100 due day 15; COMP-C 400 due day 15. Naive: COMP-A 200 due day 12 plus an SA-200 requirement of 100 at day 15 that MRP tries to "stock" — components appear 3 days early and SA shows a phantom shortage of 100. Flattened with marker:
qty-per(FG, COMP-A) = 1 × 2 = 2, need-date offset = LT_FG only .
Revision overlap. Revisions R1 (valid-to d1) and R2 (valid-from d0 < d1): for t ∈ [d0, d1] both are partially true. Auto-load rule pick(R2) rewrites demand for orders released under R1 history error = Σ_orders∈[d0,d1] |qty_per_R2 − qty_per_R1| × qty . Quarantine: hold both; human dispositions per order/effectivity; error = 0 by construction. Yield gross vs net (wave-2 gap). yield_site = good / started_net where started_net excludes engineering-sample pulls; yield_model = good / started_gross . Gap = yield_site − yield_model = good × (1/started_net − 1/started_gross) — "several points"; fixed in the mapping row (define started = net of sample pulls), model untouched. Snapshot-diffing (no CDC). Δ_t = extract_t − extract_{t−1} (row-level by key); accept only if checksum_t valid and completeness_flag_t = rows_t / expected_t ≥ 1 − ε . A partial file with 40% of rows looks like 60% deletions — the flag prevents consumption.
Wave economics. Waves W1 = 2 sites, W2 = 3, W3 = 1 hostile; average 3 weeks/site, hostile 6 weeks 2×3 + 3×3 + 6 = 21 weeks ≈ 5 months (sequential upper bound; parallelism inside waves absorbed the hostile site "in plan"). Failure in W1 costs 2 sites of credibility, not 6.
Reconciliation gate. Per site: |component_count_model − component_count_MRP| / count_MRP ≤ τ on the sampled FG set, and yield/LT vs the site's own reports within tolerance — the report is the gate.

SOLUTION FLOW — THE SYSTEM & DATA MAP



SENSITIVITY — LOW / BASE / HIGH

INPUT	LOW	BASE	HIGH
Loaded analyst rate (benchmark)	$55 → $0.07M	$60 → $0.08M	$70 → $0.09M
Share of false shortages expedited (illustrative)	10% → 36	20% → 72	30% → 108
Expedite premium (benchmark)	$2K → $0.07M	$3K → $0.22M	$5K → $0.54M
Overlap rework (illustrative)	$0.01M	$0.02M	$0.03M
Total modeled $/yr	≈$0.2M	≈$0.3M	≈$0.7M

TOTAL IMPACT STATEMENT — SAY IT LIKE THIS

"I do not claim a dollar figure from the record on this one; the million and a half of safety stock belongs to the lane program this backbone made possible. What I can model is about three hundred thousand a year: roughly eighty thousand in manual BOM reconciliation the quarantine queue replaced across six sites, about two hundred thousand in expedites that used to be triggered by phantom shortages that were never real, and a small piece, twenty thousand or so, in plans that no longer get reworked because a revision overlap was auto-loaded. The six sites, the five months and the zero auto-loaded overlaps are measured; every dollar here is my model on benchmark rates."

If challenged — the full answers.

The expedite line is the hardest

It assumes about seventy expedites a year were being fired at phantom shortages, one in five of the false signals; to firm it I would join the expedite ledger to the shortage alerts for the year before ingest and count how many alerts had no real component gap behind them.

Where is the record dollar

There is none for this story, on purpose; the inventory release was measured downstream in the p99 program and I will not book it twice, so what I offer here is the labor and expedite model, labelled as such.

> **Six hours a week per site is a guess**
>
> It is; the honest replacement is the disposition-queue log we left behind, which timestamps every human decision, so a month of that log against the pre-ingest planner timesheets would give you the real before and after.

S8 — Elementum: replacing brochure lead times with measured lane distributions — promise at p99, safety stock released by real variability

Amazon LPs: Dive Deep (primary) · Are Right, A Lot (backup — measured vs brochure) · Frugality (the release).
Crosswalk: loop packet S3 (+S4's math) — consistent. *A different program from S6's $2M — never add them.*

Honesty label. I owned the requirement (measured lanes, not brochures), the p99 promise policy and the release rule; the data team built the distributions; planners sized the buffers; Finance signed the release. The safety-stock formula is how I explain it now with the MIT inventory-theory toolkit — an illustration, not a program artifact.

60-SECOND PITCH

"**Situation.** Planning ran on carrier-brochure lead times — buffers sized on a variability nobody had measured. **Task.** I owned the requirement, the promise policy, and the release rule; the data team built the distributions; planners sized the buffers. **Action.** Discovery: the ask was a 'variability report'; I reframed it as a promise-and-buffer policy. Plan & Design: promise at p99 per lane — the smallest d with P(L≤d) ≥ 0.99 — not average-plus-padding; release rule as a signed planning act with Finance. Build & Test: actual transit and production cycle measured on 200 lanes; drift flag and refresh cadence. Cutover: I would not endorse the cut until lead-time accuracy hit 99% on that set. Hypercare: post-release stockout-and-expedite watch; fat-tail lanes keep buffer until the source process moves. Lesson: fat tails keep their buffer; I owned the rule, not the model as a day job. **Result.** Primary metric: lead-time accuracy on 200 lanes — brochure lead times → **99% at the p99 promise**. $ impact: 580K units / ~$1.5M of safety stock released without a stockout (record, one-time working capital), with ≈$0.6M/yr modeled around it — a different program from S6's $2M; never add them."

S — The six-site manufacturer's logistics manager asked for a "lead-time variability report." The real problem: inventory planning ran on carrier-brochure lead times — every buffer sized on a variability nobody had measured; too much buffer on reliable lanes, too little on fat-tailed ones.

T — Nobody owned the seam between transportation reality and planning parameters; I took it.

A — Five gated phases (dependency: the six-site canonical model (S7) — six definitions cannot produce one σ; lane-cohort counts and week markers are not in the record):

1. **Discovery** — reframed the report request as a parameter problem; scoped 200 lanes by site/lane/mode with success criteria (measured lanes replace catalog values; buffer released only on evidence); showed one right-skewed lane where the average hid the tail. *Exit bar:* planning and logistics agree the problem is the parameter, not the report.
2. **Plan & Design** — wrote the requirement to the data team (mean, spread, tail per lane; production cycle + inbound transit vs actual), the p99 promise policy, the release rule as a signed planning act (fat-tail lanes keep buffer until the source process moves — refused as a blanket release, agreed as a rule), the stockout-and-expedite watch, the drift flag and refresh cadence. On p95 vs p99 I showed the asymmetry per lane and left the percentile choice to planning — their choice, my frame. *Exit bar:* policy signed by planning; Finance agrees the release rule in the room; data team accepts the spec.
3. **Build & Test** — data team built the lane distributions and tracked lead-time accuracy per lane; I showed the asymmetry per lane (line-down cost vs carrying cost). *Exit bar:* lead-time accuracy 99% on the 200-lane set — the bar I would not endorse a cut below.
4. **Cutover & Go-Live** — planners executed the buffer release under the signed rule, fat-tail lanes excluded until their source process moved, the watch armed first. *Exit bar:* 580K units / ~$1.5M released; no stockout in the watch window.
5. **Hypercare / Close** — read the stockout-and-expedite watch, set drift flags per lane, handed the refresh cadence to planning. *Exit bar:* watch clean; the promise stays measured rather than decaying into a new brochure.

R — Primary metric: lead-time accuracy → **99% on 200 lanes** at the p99 promise. $ impact: **580K units / ~$1.5M released without a stockout** (record, one-time); ≈$0.6M/yr modeled recurring around it.

IF PROBED — PHASE Q&A (13). Answer the phase asked, then stop.

Discovery — How did you know discovery was done?

When planning and logistics agreed the problem was the parameter, not the report — brochure lead times sizing every buffer — and the 200-lane scope by site, lane and mode was written. The canonical model from the six-site program made one σ possible.

Who wrote the success criteria and who signed?

I wrote them — measured lanes replace catalog values; buffer released only on evidence; fat tails keep buffer. Planning signed the policy; Finance agreed the release rule in the same room.

Plan & Design — What was in your risk register at design sign-off?

The average hiding the tail, a buffer cut before evidence, and the promise decaying into a new brochure. Mitigations: cumulative p99 per lane, a 99% accuracy bar before endorsement, drift flags with a refresh cadence.

How did you handle Finance pushing for a broader release?

With the rule they had signed — fat-tail lanes keep buffer until the source process moves. I showed the lanes and the asymmetry; the rule did the refusing.

Why p99 and not p95?

Per lane, the cost of a line-down on a fat-tail lane dwarfed the carrying cost of the extra day. I showed the asymmetry and let planning choose; they chose p99.

Build & Test — What was your definition of done for a lane?

Distribution built from actuals — mean, spread, tail — accuracy tracked, p99 published, drift flag set. Below 99% accuracy on the set, I did not endorse a release.

Who built the distributions?

The data team, to my requirement. I owned the requirement, the promise policy and the release rule; planners sized the buffers; Finance signed. The formula is how I explain it now with the MIT toolkit — not a program artifact.

Cutover — What made you willing to endorse the release?

Lead-time accuracy at 99% on the 200-lane set and the watch armed. Then planners released under the rule, lane by lane, with fat tails excluded.

What was your rollback trigger?

Any stockout or expedite on a released lane during the watch — buffer restored. It never triggered.

Hypercare — How did you know it was closed?

The watch stayed clean, 580K units and about $1.5M were out without a stockout, and planning owned the refresh cadence and drift flags. The promise stayed measured.

What did you hand to planning at close?

The drift flag per lane with its refresh cadence, the p99 promise per lane, and the release rule as a signed act they could re-apply. The stockout-and-expedite watch became a standing report.

Cross-phase — Which lanes did you hold and how did the business take it?

The fat-tail lanes — buffer kept until the source process moved. Finance accepted because the exception was in the rule they signed, not a judgment I made afterwards.

What did you refuse?

Averaging away a tail, and endorsing a cut below 99% accuracy. Both were written into the act before anyone asked.

Where this lands at Amazon: inbound lead-time variability is half of every buffer and every promise. The transferable discipline: measure the lane's own distribution, promise at the tail you can defend, release capital only on evidence with the exception written into the rule, and give every parameter a drift flag so the promise never decays into a new brochure.

BUSINESS IMPACT MODEL — KPI TREE, NUMBERS, THE MATH

```
Service held at lower inventory: 580K units / ~$1.5M released (record)
+- Safety stock (stock) = z * d * sigma_L per lane
|    +- sigma_L: brochure folklore -> measured per lane (200 lanes)
|    +- release rule: 99% LT-accuracy bar + fat-tail lanes keep buffer; Finance
|         signed each lane (S6's two-signal rule is that program's own control)
|    +- fat-tail lanes: sigma_L worse than brochure -> buffer KEPT
+- Promise integrity (flow): p99 per lane, LT accuracy 99% (record)
|    +- cost asymmetry: line-down >> one day of holding -> p99 not p95
+- Drift loop: refresh cadence -> p99 recomputed -> drift flag ->
     buffer decision re-opened -> promise stays measured, never a new brochure
```

DRIVER	BASELINE	AFTER	$/YR	BASIS
Lead-time accuracy, 200 lanes	brochure	99% at p99	—	record
Safety stock released (one-time)	brochure-sized	measured-sized	—	record: 580K units / ~$1.5M
Stockouts on released lanes	—	0 in watch	—	record
Carry on released $1.5M	held	free	$0.3M	modeled ($1.5M × 20% all-in carry; benchmark 15–25%)
Expedites on late arrivals	~1,000 late × 10%	~100 × 10%	$0.3M	modeled (90 × $3K)
Total modeled recurring	—	—	**≈$0.6M**	around the ~$1.5M record; never added to S6's $2M

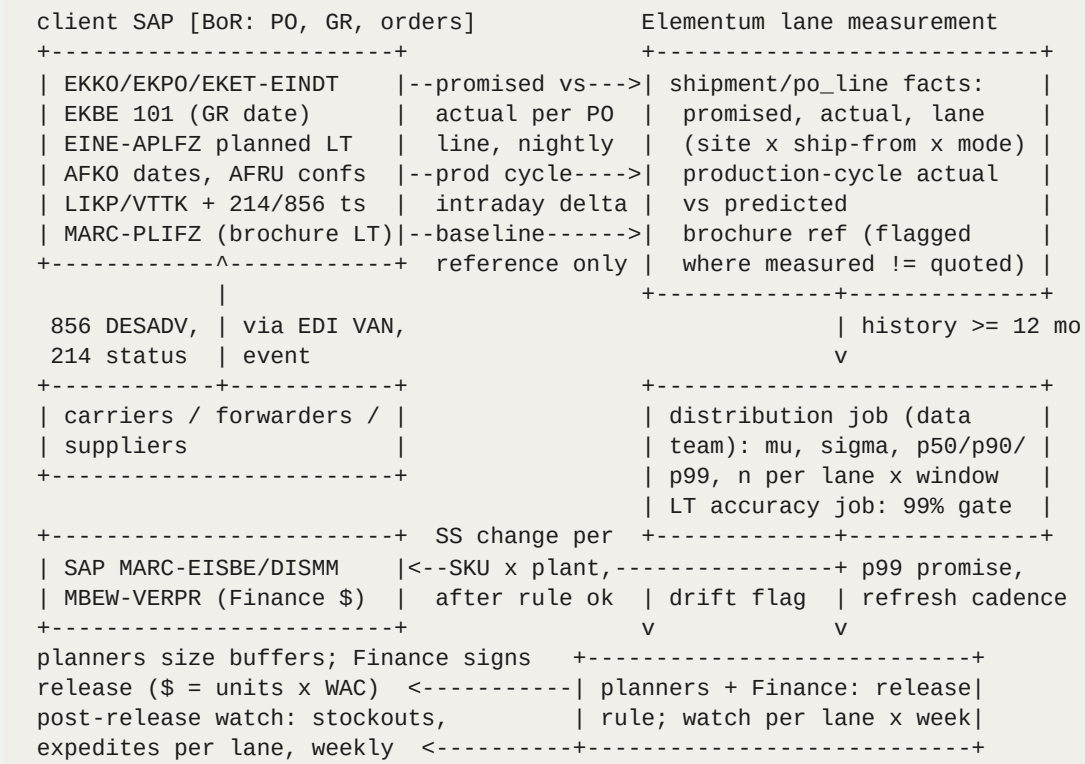
```
released = 580K units x ~$2.59/unit = ~$1.5M (record pair; low-value components)
carry = $1.5M x 20% all-in (capital ~10% + storage + insurance + obsolescence) = $0.3M/yr
shipments = 200 lanes x ~50/yr = ~10,000 (illustrative)
late before = 10,000 x 10% = ~1,000; late after = 10,000 x 1% = ~100 (99% is record)
expedites avoided = 900 x 10% share x $3K = $0.27M ~ $0.3M/yr
Sensitivity: carry 15-25% -> $0.23-0.38M; late share 5-15%; premium $2-5K
```

If challenged. *"Is the $1.5M measured?"* — "The release is record: 580K units out, no stockout in the watch window, Finance signed each lane. The ~$2.59/unit is the derived average on low-value components. What's modeled is the recurring value around it — carry and expedites — and each input is labeled." *"Why did you own the percentile choice?"* — "I didn't — I owned the frame. I showed each lane's asymmetry, line-down cost against carrying cost, and planning chose p99. Their choice surviving an audit is worth more than my choice winning a meeting."

THE MATH — FULL DERIVATION, STEP BY STEP

(Toolkit label: MIT Supply Chain MicroMasters inventory theory — how I explain the requirement
now; the data team built the distributions, planners sized buffers, Finance signed the release.)
Measured lane distribution. For lane l, actual transit L_l from shipment history (production cycle + inbound transit vs predicted).
Statistics: μ_L, σ_L, p50, p90, p99 . p99 defined cumulatively. $p99_l = \min\{ d : P(L_l \le d) \ge 0.99 \}$ — the smallest d with 99% of shipments arriving by d. Not $\mu + 2.33\sigma$ unless the lane is
normal; a right-skewed lane's normal shortcut understates the tail (illustrative from the record: by 3.4 days).
Lead-time accuracy. $LTacc_l = 1 - |\text{actual} - \text{promised}| / \text{promised}$ per shipment, aggregated per
lane; gate 99% on the 200-lane set before endorsing any cut.
Safety stock (toolkit forms). Demand-driven: $SS = z \times \sigma_d \times \sqrt{L}$. Lead-time-driven: $SS = z \times d \times \sigma_L$. Combined: $SS = z \times \sqrt{(L\,\sigma_d^2 + d^2\,\sigma_L^2)}$. $z = 2.33$ at 99% service, 1.65 at 95%.
Illustrative worked example ("illustrative, not program data"). Lane with d = 500 units/day,
brochure σ_L assumed 5 days (folklore), measured σ_L = 1.2 days; z = 2.33. Brochure SS = 2.33 × 500 × 5 = 5,825 units; measured SS = 2.33 × 500 × 1.2 = 1,398 units release 4,427 units on
this lane if forecast error also improved (release rule). Fat-tail lane: measured σ_L = 6.5
days (worse than brochure) SS rises to 7,573 — buffer kept until the source process moves.
Sanity: units = units/day × days ; SS decreases with σ_L, never with wishful averaging.
Canonical program result. 580K units / ~$1.5M released across 200 lanes at 99% lead-time
accuracy, service held (implied ≈ $2.59/unit — derived; low-value components). Different
program from S6's $2M — never add.
Drift flag. Recompute p99 per lane on a refresh cadence; flag if $|p99_{new} - p99_{old}| > \tau$ or if
LTacc falls below floor buffer decision re-opened; the promise stays measured, never decays
into a new brochure.
Cost asymmetry (why p99). $Cost_{line_down} \times P(L > \text{promise})$ vs holding cost × extra days × d ; on
fat-tail lanes the first term dominates planners pick p99 per lane after seeing both numbers.

SOLUTION FLOW — THE SYSTEM & DATA MAP



SENSITIVITY — LOW / BASE / HIGH

INPUT	LOW	BASE	HIGH
Carrying rate (benchmark)	15% → $0.23M	20% → $0.30M	25% → $0.38M
Late share before p99 (illustrative)	5% → 400 fewer	10% → 900 fewer	15% → 1,400 fewer
Share of late arrivals expedited (illustrative)	5%	10%	15%
Expedite premium (benchmark)	$2K → $0.04M	$3K → $0.27M	$5K → $1.05M
Total modeled $/yr	≈$0.3M	≈$0.6M	≈$1.4M

TOTAL IMPACT STATEMENT — SAY IT LIKE THIS

"The measured result is five hundred eighty thousand units, about a million and a half dollars of safety stock, released at ninety-nine percent lead-time accuracy with zero stockouts on the watch; that is record and Finance signed it. What recurs around it I model at roughly six hundred thousand a year: about three hundred thousand in carrying cost on capital we no longer hold, and about three hundred thousand in expedites we no longer pay because the promise sits at the ninety-ninth percentile instead of a brochure number. The release and the accuracy are measured; the two recurring pieces are my model on benchmark carrying and expedite rates, and I never add this to the two million from the forecast program."

If challenged — the full answers.

The expedite number is the softest

It assumes ten thousand shipments a year on the two hundred lanes and that one late arrival in ten forced an expedite before p99; to firm it I would pull the expedite ledger by lane for the year before and after the release and compare

directly.

Twenty percent carrying cost is high

Fifteen to twenty-five is the textbook range and I use the middle; capital alone is eight to twelve, so if Finance wants capital only the carry is about a hundred and fifty thousand and I would say that instead.

Why is the million and a half not an annual saving

Because it is a one-time working-capital release, booked once as a signed planning act; what recurs is the carry and the avoided expedites, and I would rather be right at six hundred thousand a year than wrong at a million and a half.

S9 — Elementum: 1,000 supplier/carrier routes asked, 597 built first — X12 EDI onboarding behind a ≥95% ASN cohort gate

Amazon LPs: Have Backbone; Disagree & Commit (primary — 597 vs 1,000, two levels up) · Deliver Results · Insist on the Highest Standards (the gate held twice) · Think Big (the staged 1,000-lane vision) · Broad Responsibility (the portal floor kept small carriers inside the standard). **Crosswalk:** loop packet S7 — consistent. *Cohort sizes and the number of cohorts are not in the record — do not quote them.*

Honesty label. I owned the scope analysis and the onboarding standard, influencing an executive two levels above my sponsor; engineers built the maps and harness to the standard; Transportation owned the carrier relationships.

60-SECOND PITCH

"**Situation.** The Global Head of Supply Chain wanted inbound visibility on 1,000 lanes in two months; shipment-data completeness was 60% and high-value lanes were blind. **Task.** I owned the scope analysis and the onboarding standard, influencing an executive two levels above my sponsor. **Action.** Discovery: spend Pareto — 597 lanes = 80% of freight spend — pre-validated with Transportation and Procurement. Plan & Design: X12 850/855/856/940/945/214/997 with an XML/portal floor; cohort gate ≥95% valid ASN. Build & Test: mapping-and-test harness; scorecards per lane. Cutover: 597 live in month one; one cohort failed twice on a timestamp format — I held it. Hypercare: completeness scorecards to 95%; ~200 next wave behind the same gate. Lesson: automating dirty data manufactures confident wrong answers — the gate is the product. **Result.** Primary metric: inbound shipment-data completeness — **60% → 95%**. $ impact: freight spend −10% ≈ $2M in six months (record); my model splits it into routing-guide compliance, mode discipline and expedites avoided, and adds ≈$0.4M/yr of modeled document-handling labor on top; 597 lanes = 80% of spend live in month one; adoption 8×."

S — The customer's Global Head of Supply Chain wanted end-to-end inbound visibility on 1,000 global lanes in two months — scope, schedule and quality all fixed. Lanes differed by orders of magnitude in spend, volume and criticality; carriers differed in EDI capability; completeness was 60% and high-value lanes were effectively blind.

T — Own the scope analysis and the onboarding standard, influencing an executive two levels above my sponsor.

A — Five gated phases:

1. **Discovery** — built the spend Pareto and lane scoring model (spend × criticality × mode × volume) and the carrier EDI-capability survey; pre-validated the 597 / ~200 / remainder scope with Transportation and Procurement before taking it up. *Exit bar:* lane owners cannot contradict the cut; the executive accepts the phased plan in spend share, not lane count.
2. **Plan & Design** — wrote the onboarding standard (document set + XML/portal floor — "the standard was the data, not the transport"), the ≥95% valid-ASN cohort gate, the per-lane scorecard spec (events received / expected; 997 rate; tracking-number joinability), the harness design. *Exit bar:* standard signed by Transportation; gate published to carriers before cohort 1; harness templates exist.
3. **Build & Test** — held engineers to the harness (per-partner mapping templates, test scripts, DQ checklist), test files per carrier, a parallel run per lane, scorecards live. *Exit bar:* each cohort's lanes clear ≥95% valid ASN in test before activation.
4. **Cutover & Go-Live (per cohort)** — activated cohorts only off the published gate; published the scorecards; recorded the hold decision when a cohort failed. *Exit bar:* cohort active only after the gate; a failing cohort held — twice, on a timestamp format.
5. **Hypercare / Close** — weekly completeness scorecards; harness reused for later waves; method written up as the sourcing SOP (spend file in → phased plan out). *Exit bar:* completeness 60% → 95%; 100% visibility on high-value lanes; remaining waves behind the same gate.

R — Primary metric: completeness **60% → 95%**; 597 lanes = 80% of spend live in month one; freight −10% ≈ $2M in six months (record); adoption 8×.

IF PROBED — PHASE Q&A (13). Answer the phase asked, then stop.

Discovery — How did you know discovery was done?

When the spend Pareto was pre-validated by Transportation and Procurement and the scope proposal — 597 first, ~200 next, remainder after — could not be contradicted by any lane owner in the leadership meeting.

Who signed the scope?

The Global Head of Supply Chain accepted it after seeing the curve; the lane owners had already validated it. I priced the deferral in spend share, not lane count.

Plan & Design — What was in your risk register at design sign-off?

Carriers unable to produce valid ASNs, the scope being contradicted in the room, and bespoke maps per partner. Mitigations: XML/portal floor plus a hold rule, pre-validation, and a parameterized harness.

What was the entry bar for a cohort?

Standard and gate published to its carriers, mapping templates instantiated, test files in hand, a scorecard per lane. Then ≥95% valid ASN in test before activation.

How did you set the standard?

850/855/856/940/945/214/997 on X12, with XML or portal templates as the floor for carriers without EDI. The standard was the data, not the transport.

Build & Test — What was your definition of done for a lane?

Valid-ASN rate ≥95%, 997 acknowledgments flowing, tracking numbers joinable, completeness on its scorecard. Below that, the lane was fixed, not automated.

Describe your bug-bash equivalent.

The cohort test day: every carrier's file through the harness in one sitting; defects classified as format, timestamp, missing segment or join key, each with an owner. That is where the timestamp defect surfaced.

Cutover — Which cohort did you hold and how did the business take it?

One cohort failed twice on a carrier timestamp format; I held it both times. Transportation accepted because the 95% gate was published before cohort one — a gate that only holds in private is a suggestion.

What did each wave deliver?

Wave 1: 597 lanes, 80% of spend, live in month one. Wave 2: ~200 lanes on a harness that made the work mostly configuration. Wave 3: the tail, including non-EDI carriers on templates.

Hypercare — How did you know you were done? What did you watch?

Completeness 60% → 95%, 100% visibility on high-value lanes, the harness reused without me, the method adopted as the sourcing SOP. Watched weekly: per-lane completeness, the 997 rate, tracking-number joinability. A lane that slipped below its scorecard went back to "fix," not "live."

Cross-phase — Where did you trade scope for date? What did you escalate?

Deferred ~400 lanes carrying ~20% of spend so the 597 could go live at the highest data quality in month one — priced, and delivered later behind the same gate. Escalated the re-scope to the Global Head of Supply Chain — two levels up — with the spend curve and the lane owners' validation attached. Refused: activating a cohort that failed the gate.

Did you deliver the remaining 400? · Do differently?

Yes — later waves, same gate; the first 597 carried the spend and the decision. Differently: the harness from cohort one, not cohort two.

Where this lands at Amazon: vendor and carrier onboarding at scale is a Pareto plus a gate: spend share first, a published data-quality bar a partner must clear before activation, and a floor (portal/templates) so the long tail is kept inside the standard rather than priced out of it. The sentence that travels: automating dirty data manufactures confident wrong answers.

BUSINESS IMPACT MODEL — KPI TREE, NUMBERS, THE MATH

```
Inbound freight spend -10% ~= $2M in 6 months (record)
+- Visibility (stock): completeness 60% -> 95%, high-value lanes 100%
|    +- routing-guide compliance: off-guide carrier spend caught
|    +- mode discipline: air -> ocean/ground when ETA is trusted
|    +- expedites + detention: chase calls replaced by 214 events
+- Document handling (flow): 850/855/856/940/945/214 by X12, not by hand
+- DQ gate loop: >=95% valid ASN -> cohort activates -> scorecard per lane ->
     defects fixed as data -> harness parameterized -> cohort 3 mostly config -> adoption 8x
+- Scope loop: 597 = 80% spend first -> $ lands month one -> executive funds the next wave
```

DRIVER	BASELINE	AFTER	$	BASIS
Shipment-data completeness	60%	95%	—	record
Lanes live month one	0	597 = 80% of spend	—	record
Freight spend on onboarded lanes	~$20M/yr (packet-derived)	−10%	≈$2M / 6 mo	record ($2M)
— routing-guide compliance	off-guide unseen	caught by 856/214	~$0.8M	modeled split of the record $2M
— mode discipline	premium modes by habit	ETA trusted	~$0.6M	modeled split
— expedite + detention avoided	blind lanes chased	events replace calls	~$0.6M	modeled split
Manual document handling	~90K docs/yr keyed	X12/XML validated	$0.4M/yr	modeled (90K × 6 min × $45/hr)

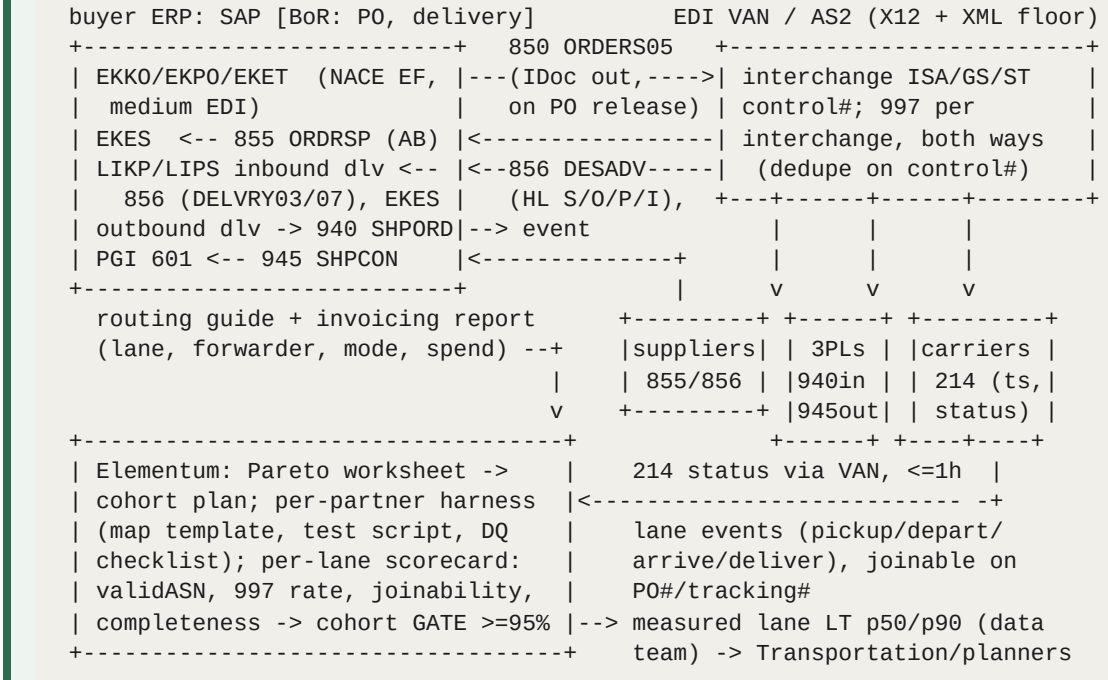
```
The three slices (0.8 + 0.6 + 0.6) SUM TO the record $2M -- they decompose it, never add to
it
docs = 597 lanes x ~50 shipments x 3 documents = ~90,000/yr (illustrative)
doc labor = 90,000 x 6 min / 60 x $45/hr = ~$0.4M/yr modeled add-on
```

If challenged. *"Is the $2M annualized?"* — "No — it's the record as measured: minus ten percent on the onboarded lanes' spend in six months. I keep it as stated rather than doubling it into a run-rate." *"How do you know the split?"* — "I don't, to the dollar — the split is my model of where the ten percent came from, and the three slices sum to the record, never on top of it."

THE MATH — FULL DERIVATION, STEP BY STEP

Spend Pareto. Sort lanes by annual freight spend s_l descending; cumulative share S(k) = Σ_{l≤k} s_l / Σ_all s_l .
Canonical: 597 of 1,000 lanes S(597) = 80% of spend. Priority score per lane (spreadsheet): score_l = f(forwarder) × spend_l × criticality_l × mode_weight × volume_l — a weighted ranking, not an optimizer.
Cohort gate. validASN_rate = ASNs passing all validation rules / ASNs received over the test window; activate the next cohort only if validASN_rate ≥ 0.95 . One cohort failed twice on a carrier timestamp format (invalid excluded from valid count, so the rate fell below 0.95) — held.
Completeness. completeness_l = events received / events expected per lane (expected = shipments × milestones per mode). Canonical: 60% → 95%. Illustrative: 200 shipments × 4 milestones = 800 expected; 480 received 60%; 760 95%. 997 acknowledgment rate. ack_rate = 997s received / documents sent per partner (pipe-alive proof; target ~100%; a drop is a transport failure, not a data failure).
Joinability. join_rate = 856/214 events with a matching PO#/tracking# in our data / events received ; unjoinable events are received but useless.
Freight result. Spend −10% ≈ $2M in 6 months implied addressable base ≈ $20M/yr on the onboarded lanes (derived from the canonical pair, stated as such). Adoption 8×; high-value lanes 100% visible.
Sanity: S(k) is monotone non-decreasing; a gate on rate (not count) prevents a large cohort from hiding bad partners; completeness ≤ 100% (over-reporting = duplicate events dedupe rule).

SOLUTION FLOW — THE SYSTEM & DATA MAP



SENSITIVITY — LOW / BASE / HIGH

INPUT	LOW	BASE	HIGH
Shipments/yr on 597 lanes (illustrative)	20,000	30,000	40,000
Minutes per manual document (benchmark)	4	6	8
Loaded coordinator rate (benchmark)	$40	$45	$55
Total modeled $/yr	≈$0.2M	≈$0.4M	≈$0.9M

TOTAL IMPACT STATEMENT — SAY IT LIKE THIS

"The measured number is freight spend down ten percent, about two million dollars in six months on the five hundred ninety-seven lanes that carried eighty percent of spend, with completeness sixty to ninety-five percent and adoption eight x. Inside that two million I model roughly eight hundred thousand from catching off-guide carrier spend, six hundred thousand from mode discipline once the ETA was trusted, and six hundred thousand from expedites and detention that chase calls used to cause; those three are my split, the two million is the record. On top of it I model about four hundred thousand a year of document-handling labor that X12 replaced, and that piece is entirely mine."

If challenged — the full answers.

The split of the two million is the softest

It is a judgment on how the ten points divide; to firm it I would tag every invoice line on the onboarded lanes by cause, off-guide carrier, mode change, or expedite, for the six months and let the ledger set the shares.

Ninety thousand documents is a guess

Fifty shipments a lane and three touched documents each is illustrative; the ISA and GS control-number counts on the VAN for the first full quarter would give the exact document volume in an afternoon.

Labor saved is not cash

Correct, nine thousand hours is coordinator capacity moved from keying to exceptions; I would measure it as headcount held flat while lanes went from six hundred to a thousand, which is the number Finance can see.

S10 — Elementum: Austria — "the noise said carriers; the data said measurement" (routing-guide rules engine, 25 sites, ~$8M)

Amazon LPs: Are Right, A Lot (primary) · Dive Deep (backup) · Broad Responsibility (primary — one site's fix rolled to 25 sites) · Earn Trust (the Finance-signed method). **Crosswalk:** not in the loop packet — safe anywhere. *Guardrails: say $8M, never "$1M"; "~99% in 3 months," never "two months"; 1–2 hrs/wk is the after, never the before.*

Honesty label. I owned the analysis and the rules design and pulled the data myself; the data platform team built the engine; Finance signed the savings method before any dollar was claimed. Site-wave counts and dates beyond "Austria first, then 25 sites" are not in the record.

60-SECOND PITCH

"**Situation.** Austria transportation could not measure on-time delivery; reports said data quality was 40%; the team spent 8–10 hours a week reconciling by hand; everyone believed the carriers were late. **Task.** I owned the analysis and the rules design. **Action.** Discovery: three years of shipment history in SQL — true DQ ~55%, not 40; defect classes named. Plan & Design: promised-delivery-date as a routing-guide rules engine with Incoterm in the key (~8% of lanes); Finance-signed savings method before any dollar — contracted guide rate minus paid rate, executed only. Build & Test: alerts by reason code; automated OTD vs a manual sample per cohort before any scorecard. Cutover: Austria scorecard released; manual 8–10 → 1–2 hrs/wk. Hypercare: super-users per region, a one-page runbook; 25 EU sites. Lesson: measure before you blame; sign the method before the number. **Result.** Primary metric: routing-guide data quality — **~55% true (40% reported) → ~99% in 3 months**. $ impact: $500K/yr at Austria (10% of a ~$5M base, Finance-signed executed-only method), then ~$8M across 25 EU sites / 400+ lanes — both record; plus ≈$0.5M/yr of modeled analyst capacity."

S — Transportation at the customer's Austria site could not measure on-time delivery. Reports said data quality was 40%; the team spent 8–10 hours a week reconciling promised vs actual dates by hand; the freight base was ≈$5M/yr with no enforceable OTD. Everyone — the site, the carriers' account teams, my own sponsor — believed the carriers were late.

T — I owned the analysis and the rules design.

A — Five gated phases:

1. **Discovery** — pulled three years of shipment history in SQL myself; ran the defect census (missing rates, wrong-grain lanes, five spellings of one carrier name, brochure transit times); established true DQ ~55% vs 40% reported; reframed the problem as measurement, not carriers. *Exit bar:* sponsor accepts the reframe on the evidence; defect classes named and counted.
2. **Plan & Design** — rules-engine spec (lookup by ship-from/to, mode, volume, Incoterm, SLA; a no-hit pages a human, never a default carrier); reason-code alert design; **the savings method signed with Finance before any dollar was claimed** (contracted guide rate − paid rate, per shipment, executed only); the scorecard DQ gate (automated OTD = manual sample per lane cohort). *Exit bar:* spec signed by transportation; Finance method signed; scorecard gate published.
3. **Build & Test** — held engineering to the spec; weekly defect burn-down by reason code; manual-sample comparison per lane cohort. *Exit bar:* DQ trending to ~99%; automated OTD matches the manual sample per cohort.
4. **Cutover & Go-Live (Austria)** — released the carrier scorecard only after the gate passed; retired the manual reconciliation (8–10 → 1–2 hrs/wk); stood up the savings ledger per shipment. *Exit bar:* a scorecard the carriers could not dispute; $500K/yr on the Finance-signed method.
5. **Hypercare / Close → scale** — super-user model per region; one-page runbook (add a lane · read a no-hit · when to escalate); two shadowed rule reviews per site; rolled out site by site. *Exit bar:* 25 EU sites / 400+ lanes ≈ $8M; ~99% DQ; adoption cost per site = a rules file + one DQ cycle, in days.

R — Primary metric: DQ **~55% → ~99% in 3 months**; $500K/yr Austria → ~$8M across 25 sites (record); manual reconciliation 8–10 → 1–2 hrs/wk.

IF PROBED — PHASE Q&A (13). Answer the phase asked, then stop.

Discovery — How did you know discovery was done?

When the three-year pull had every blank classified — missing rate, wrong grain, name variant, brochure time — and the sponsor accepted that true quality was ~55%, not 40%, and that the problem was measurement, not carriers.

Why pull the data yourself?

Because the room already had an explanation. Three years of history in SQL was faster than a request queue and could not be argued with.

Plan & Design — What was in your risk register at design sign-off?

Incoterm changing who owns the clock, a silent default carrier on a no-hit, and savings disputed after the fact. Mitigations: Incoterm in the key, page a human, Finance-signed method before any claim.

How did you handle the sponsor objecting to the delay?

I showed the defect classes trending down weekly and the manual-sample gate the scorecard had to pass. He was unhappy for weeks and right to want speed; the carriers never disputed the result.

Who signed the savings method and when?

Finance, before the first dollar was claimed: contracted rate minus paid rate, per shipment, per lane, executed only. Negotiated-but-not-executed never counted.

Build & Test — What was your definition of done for a lane rule?

It resolves without no-hits across the sample window and its automated OTD matches the manual sample for its cohort. Until then the manual reconciliation kept running.

How did you treat data defects?

As classes, not one-offs: alerts by reason code, trended weekly, burned down — root cause at scale. That is how 55% became ~99% in three months.

Cutover — What made you say go on the scorecard?

Automated OTD matching the manual sample per lane cohort. Anything less would have shipped a scorecard the carriers could dispute — and they would have won.

What was your rollback?

The manual reconciliation stayed alive until the match held. Reversibility was free because I didn't retire the old process early.

Hypercare — How did you decide Austria was closed and scale could start?

DQ ~99%, manual hours at 1–2 a week, the scorecard undisputed, and a regional super-user running the rules file after two shadowed reviews. Then 25 sites at a rules file plus one DQ cycle each.

What did you hand to the regional super-users?

Their rules file, the one-page runbook — add a lane, read a no-hit, when to escalate — and two shadowed rule reviews each. After that, adding a site cost days.

Cross-phase — Where did you trade scope for date? What did you refuse?

I traded date for evidence — delayed the scorecard weeks rather than ship it on bad data; scope was never cut. Refused: a scorecard on 55% data, a silent default carrier, and any savings number without a Finance-signed method.

How do you know the $8M is real? · Do differently?

Finance-signed, executed-only method applied across 25 sites; the $500K pilot is 10% of a ~$5M base; anything merely negotiated is excluded. Differently: start with the industry benchmark for lane transit times instead of proving the brochure wrong lane by lane — same answer, faster.

Where this lands at Amazon: the cleanest "used data to overturn the room's explanation" story — and the model for how a savings claim should be built: method signed with Finance before the number, executed-only, then scaled site by site with the adoption cost measured in days. Broad Responsibility angle: one site's fix became 25 sites' instrument.

BUSINESS IMPACT MODEL — KPI TREE, NUMBERS, THE MATH

```
Enforceable OTD + executed freight savings: $500K Austria -> ~$8M (record)
+- Measurement integrity (stock): true DQ ~55% (40% reported) -> ~99%
|    +- promised date = ship + transit(lane_key), Incoterm in key (~8% of lanes)
|    +- no-hit pages a human, never a default carrier
+- Savings ledger (flow): (guide rate - paid rate) x executed shipments
|    +- Austria: ~$5M base x 10% = $500K/yr (record)
|    +- 25 EU sites / 400+ lanes: rules file + one DQ cycle each -> ~$8M
+- Manual reconciliation: 8-10 hrs/wk -> 1-2 hrs/wk per site (modeled $)
+- Defect loop: alerts by reason code -> weekly burn-down -> DQ rises ->
     fewer no-hits -> scorecard trusted -> carriers fix at source (+)
```

DRIVER	BASELINE	AFTER	$/YR	BASIS
Routing-guide data quality	~55% true (40% reported)	~99% in 3 months	—	record
Austria savings (guide − paid, executed)	$0	10% of ~$5M base	$500K	record
Network roll-up, 25 EU sites / 400+ lanes	$0	~$8M	~$8M	record
— implied avg per site · implied base	—	~$320K/site · ~$80M at 10%	—	modeled (derived; Austria above average — a larger site, consistent)
Manual reconciliation, 25 sites	8–10 hrs/wk each	1–2 hrs/wk	≈$0.5M	modeled (~9,400 hrs × $55; per-site $20K)

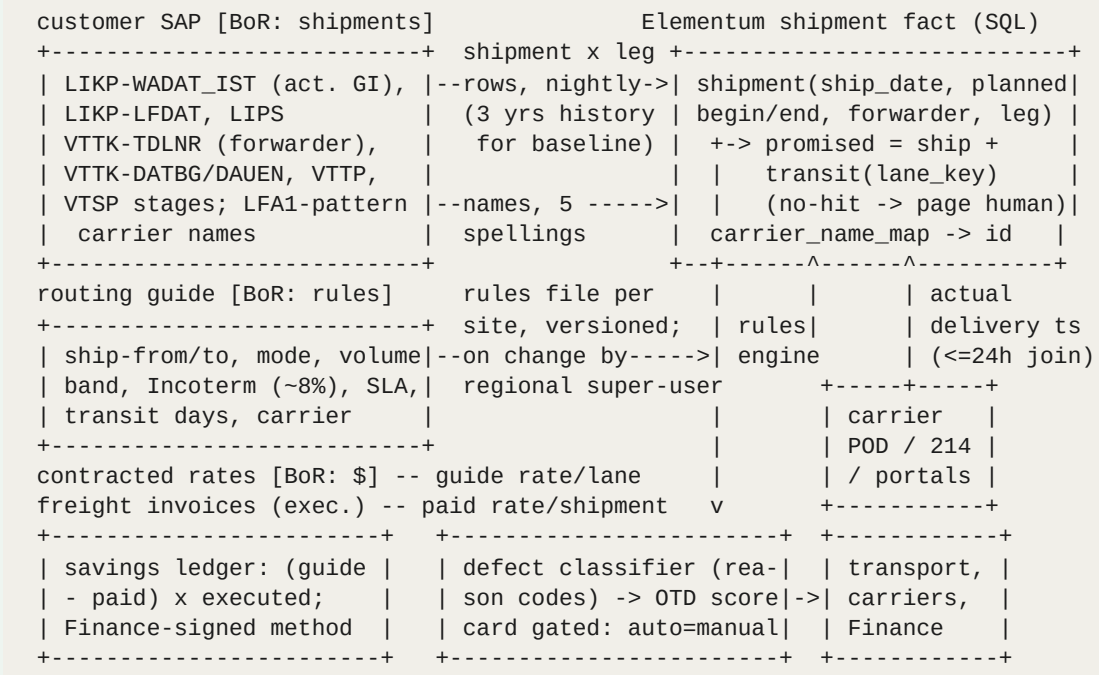
```
austria = ~$5M x 10% = $500K/yr (record; Finance-signed, executed-only)
network = ~$8M (record; say $8M, never $1M) -> avg $320K/site; implied base ~$80M at 10%
recon capacity = 25 sites x ~7.5 h/wk x 50 wk x $55/h = ~$0.5M/yr modeled add-on
```

If challenged. *"How is the $8M real?"* — "Because the method was signed before the number existed: contracted guide rate minus paid rate, per shipment, executed only. Negotiated-but-not-executed never counted. The $500K Austria pilot is the method's proof; the $8M is the same ledger across 25 sites."

THE MATH — FULL DERIVATION, STEP BY STEP

```
Data quality (true vs reported). DQ = shipments with a valid promised date AND a valid
actual
  date AND a resolvable
lane / all shipments . Reported 40% (a field-populated proxy); SQL-measured ~55% (populated
and
  valid). Defect classes: missing rate, wrong-grain lane, carrier name variant (five
spellings),
  brochure transit time posing as a promise. Target ~99% in 3 months.
On-time delivery. OTD = deliveries with actual ≤ promised / deliveries with a valid promise
;
  only valid-promise rows count — measuring OTD on brochure promises penalizes carriers for
our
  error. OTD accuracy ~97% post-fix (automated vs manual sample agreement per lane cohort).
Promised-date rule. promised = ship_date + transit(lane_key) with lane_key = (ship-from,
  ship-to, mode, volume band, Incoterm, SLA) . Incoterm changes who owns the clock: DDP
promise
  runs to the consignee's door; EXW to the carrier pickup — ~8% of lanes needed Incoterm in
    the
  key. No-hit  page a human, never a default.
Savings method (Finance-signed). Per shipment saving_s = (contracted guide rate_s −
  counterfactual paid rate_s) for executed shipments only (never "negotiated"); annual
    = Σ_s
  saving_s . Pilot: freight base ≈ $5M/yr × 10% = $500K/yr; network: 25 EU sites / 400+
lanes ≈
  $8M (say $8M, never $1M).
Manual effort. 8–10 hrs/wk → 1–2 hrs/wk (−80%); across a team of illustrative 4 people  ~32
  hrs/wk recovered ("illustrative, not program data").
Sanity. 55% true < 40% reported? No — 55% > 40%: the report under-counted usable rows
    because it
  keyed on a single field; the
SQL measure was both broader and stricter. DQ ≤ 100%; OTD computed only on the valid-promise
  denominator; savings only on executed rows.
```

SOLUTION FLOW — THE SYSTEM & DATA MAP



SENSITIVITY — LOW / BASE / HIGH

INPUT	LOW	BASE	HIGH
Hours saved per site per wk (record range)	6 (8→2)	7.5 (9→1.5)	8.5 (10→1.5)
Loaded analyst rate (benchmark)	$50	$55	$65
Sites with the same manual burden (illustrative)	15	25	25
Total modeled $/yr	≈$0.2M	≈$0.5M	≈$0.7M

TOTAL IMPACT STATEMENT — SAY IT LIKE THIS

"The measured numbers are five hundred thousand a year at Austria, ten percent of a five million freight base on a method Finance signed before I claimed a dollar, contracted rate minus paid rate on executed shipments only, and about eight million when the same rules file and one data-quality cycle went to twenty-five sites and four hundred plus lanes. The roll-up is simple: eight million over twenty-five sites is about three hundred twenty thousand a site, which implies roughly eighty million of freight at ten percent, and Austria at five hundred thousand sits above that average because it was a bigger site. On top I model about half a million a year of analyst capacity from cutting manual reconciliation from eight to ten hours a week to one or two at each site; the eight million is record, the half million is my model."

If challenged — the full answers.

Is the eight million really measured

It is the Finance-signed executed-only method applied at each site, not a projection; the assumption underneath is that the ten percent Austria rate held elsewhere, and the thing that firms it is the per-shipment ledger at each site, which is what Finance read.

The half million of labor is the softest

> It assumes all twenty-five sites carried Austria's eight to ten hours a week, which I cannot prove; a one-week time log at three sites before their rules file went live would replace that assumption with data.
>
> **Why not count negotiated rate cuts too**
>
> Because negotiated but not executed is a promise, not a saving; the method counts only shipments that moved at the guide rate, and that discipline is why the carriers never disputed the scorecard or the dollars.

S11 — Flexport: Shipment Visibility launch — 7 → 68 behind graduation gates, the WBR, and pre-prod measurement

Amazon LPs: Deliver Results (primary) · Ownership · Customer Obsession (graduation criteria per wave) · Earn Trust (the discarded 99.4% week) · Hire and Develop (backup — wave teams grown into owners). **Crosswalk:** loop packet MAIN + S18 + S26 — **carries the §1.3 conflicts** (automation 40→90 here vs 60→97 there; WBR hours 1.5–2 → ~4 here vs 30→6 there). Resolve before the loop; inside this packet, this page is canon. *Never "10×" — say 7 → 68 (9.7×). Never "$1.00 → $0.03" or "70% baseline."*

Honesty label. Program owner across nine teams / ~100 people; the WBR format was copied from the LSA review, not invented; week counts for the build are illustrative; the ~$1.5M pre-prod saving is a separate canonical figure, never merged with CTS.

60-SECOND PITCH

"**Situation.** Flexport's visibility product had public targets — 68 monthly active customers, 170 MAU, 32K shipments — off seven pilots at $0.66 a container; Sales wanted the whole pool boarded at once, and leadership said the program was not showing value. **Task.** As program owner across nine teams / ~100 people, I owned the boarding path, the measurement system, and the pre-prod proof. **Action.** Discovery: ~50-container calibration entry bar per client; WBR input/output OS copied from an existing review, seven metrics frozen. Plan & Design: six graduation criteria published to Sales before wave 1; launch gate 100 real containers × 7 checks; sample size 300 containers, ±1.6% at 95% confidence. Build & Test: weekly ops audit; WBR by hand 1.5–2 then ~4 hrs/wk; pre-prod ingest, per-hop monitors, kill switches. Cutover: wave 1 (2 clients) → wave 2 (4 of 7) → wave 3 (20+) held in public on an ETA-accuracy miss from one bad source → full rollout. Hypercare: discarded a flattering 99.4% three-touch week; the VP unlocked the 68-client rollout two months early off the WBR. Lesson: board no faster than automation matures; a gate that only holds in private is a suggestion. **Result.** Primary metric: cost-to-serve per container — **$0.66 → $0.03 (−95.5%)** across customers 7 → 68 (9.7×) with zero rollbacks and no added ops headcount; automation 40% → 90%; the ~$1.5M pre-prod saving is a separate record figure."

S — Senior leadership told me the visibility program wasn't demonstrating value at the anticipated rate — it would drop from #3 to #14 of 16 initiatives.

T — I had been reporting activity, not evidence. Funding and the 68-customer rollout hung on whether value was legible.

A — Five gated phases (canonical: WBR copied from the LSA review — Mon–Sun week, five-week window, WoW = (B−A)/A, ±5% green/red, every input tagged to an output; seven frozen metrics; run by hand 1.5–2 hrs/wk then ~4 for over a year; wave roster W1 Q4'21 (2 clients) → W2 Q1'22 (4 of a 7-pool) → W3 Q2'22 (20+, held on ETA accuracy) → full H2'22):

1. **Discovery** — wrote the honest diagnosis (reporting activity, not evidence); inventoried what leadership actually decides on (rollout pace, funding); chose the LSA WBR as the pattern to copy rather than invent. *Exit bar:* leadership agrees on the questions the review must answer weekly.
2. **Plan & Design** — froze seven metric definitions (coverage auto, coverage w/ human, tickets, tickets per active container, 4PL SLA %, 3PL SLA %, delta); input→output tagging; the ±5% WoW rule; the 15-day inactive rule as the honest denominator; the four-system pull procedure; six graduation criteria published to Sales before wave 1. *Exit bar:* definitions signed by the owners of each input; first Monday deck dry-run.
3. **Build & Test** — built the deck manually every week (1.5–2 hrs → ~4); pre-decomposed every red with an owner and a dated fix; enforced the data-honesty rule (a truncated week is a data miss, not a result); pre-prod: 300-container measurement, per-hop monitors, kill switches. *Exit bar:* two consecutive Mondays where every red arrives with an owner and the number verifies the following week.

4. **Cutover & Go-Live** — the Monday WBR became the operating forum, with the rollout decision taken in the review; waves boarded only through the six graduation criteria; wave 3 held in public on ETA accuracy traced to one bad source. *Exit bar:* VP unlocks the 68-client rollout two months early.
5. **Hypercare / Close** — handed the manual pull to the Snowflake pipeline (S16) with the definitions in code; left the critic steering by the numbers. *Exit bar:* automation 40% → 90%; CTS $0.66 → $0.03; the same critic uses the review to steer.

R — Primary metric: CTS **$0.66 → $0.03 (−95.5%)**; customers 7 → 68 (9.7×), zero rollbacks; accuracy 60% → 90%; both public targets two months early.

IF PROBED — PHASE Q&A (13). Answer the phase asked, then stop.

Discovery — How did you know discovery was done?

When leadership had told me what they actually decide on — rollout pace and funding — and agreed the questions the review must answer weekly. Their critique was about evidence, not effort, and I accepted that first.

Why copy a format instead of designing one?

The LSA review already had leadership's trust — five-week window, WoW at ±5%, inputs tagged to outputs. Inventing a format would have spent credibility I didn't have.

Plan & Design — Who wrote the metric definitions and who signed?

I wrote the seven; the owner of each input — Data Services, Support, GTM — signed the definition they would be red against. Frozen after that; new metrics out of scope.

What was in your risk register?

Measurement-caused reds, a flattering wrong number, and bus factor one. Mitigations: decompose before the meeting, print data misses honestly, automate the pull — which became S16.

How did you handle the objection to freezing the list?

Stability over coverage — a number that changes definition can't steer a rollout. I offered a parking lot for new metrics and kept the seven.

Build & Test — What was your definition of done for a red week?

Decomposed before the meeting, an owner named, a dated fix, and the same number verified the following Monday. A red without an owner was my miss, not theirs.

Describe a hard call in the build.

Discarding a truncated 99.4% week. It flattered the program and would have taught the VP a number that couldn't survive the next Monday. I printed a data miss.

Cutover — When did the review become "live"?

The Monday the VP took the rollout decision in the room — unlocking the 68-client rollout two months early off the numbers, not a narrative.

Which wave did the review hold and how did Sales take it?

Wave 3, on ETA accuracy traced to one bad source. Sales had the six graduation criteria before wave 1, so the hold was the rule firing — and the customers we held became the cleanest wave.

Hypercare — How did you know the review would outlive your hands?

When the pipeline (S16) produced the same seven numbers within 1% for consecutive weeks and the critic who started this steered by them.

What did you keep doing after the review was credible?

Running it by hand every Monday for over a year — 1.5–2 hours, then about 4 — until the pipeline matched within 1%. A review that skips a week teaches leadership it can be skipped.

Cross-phase — Where did you trade scope for date? What did you escalate?

New metrics for stability, and boarding speed for a rollout that never rolled back — held wave 3 publicly rather than take a bad first impression at scale. Escalated nothing outside the review — the review was the escalation forum, with the VP deciding in it every Monday.

> **Why discard 99.4%? · Do differently?**
> Three touches is not a week; printing it would have taught the VP a number that couldn't survive the next Monday. Differently: start the WBR at kickoff, not after the critique.

Where this lands at Amazon: this is a WBR culture story told to the company that invented the WBR — the respect move is that I copied a trusted format rather than inventing one, froze definitions, tagged inputs to outputs, and let the review make the rollout decision. The graduation criteria are Amazon-shaped launch gates: published before the wave, held in public when they fired.

BUSINESS IMPACT MODEL — KPI TREE, THE WAVE-MACHINE MATH

```
Program value legible to leadership (funding + 68-client rollout)
+- Cost-to-serve per container $0.66 -> $0.03 (record)
|    +- Human interactions/container 3.7 -> 0.17 (record cross-walk: $0.18/interaction)
|    +- Automation coverage 40% -> 90% (record)
|         (loop) fewer touches -> more calibration data -> better ETA/DQ ->
|                fewer exceptions -> fewer touches
+- Customers boarded 7 -> 68 (9.7x), zero rollbacks (record)
|    +- Six graduation criteria published before W1 (gate)
|    +- W3 held publicly on ETA accuracy -> cleanest wave
+- Time-to-value: rollout unlocked 2 months early off the WBR (record)
+- Pre-prod measurement path ~$1.5M saved (record, SEPARATE canonical)
```

```
Wave-machine math: entry bar ~50 containers calibration per client (not a wave gate)
Six graduation criteria (ALL must pass): DQ >= 90% vs source of truth ·
  ETA/milestone accuracy >= 80% · verification SLA 90% (review <24h, max 48h) ·
  engagement >= 100 containers + 5 key features · stability · CSAT >= consignee average
Launch gate before first paying customer: 100 real containers x 7 checks;
  load proof 500 containers / 20 min / 99.6%
CTS cross-walk: $0.66/$0.18 ~ 3.7 residual interactions -> $0.03/$0.18 ~ 0.17
Pre-prod: n = 300 containers, expected accuracy 98% -> +/-1.6% at 95% CI;
  E2E accuracy 85% -> ~98%; rollout 1.5 months vs 6; ~$1.5M saved (separate canonical)
WBR instrument: 10,289 events; event-level human 19.4%; WoW = (B-A)/A; +/-5% red/green
At the 32K-shipment target: -95.5% CTS ~ $50K/yr modeled direct (volume illustrative) --
  the value is scale without headcount, not the direct dollars
```

If challenged. *"The CTS saving is only $50K?"* — "Directly, yes — and I say that out loud. The point of −95.5% was never the direct dollars; it's that 7 customers became 68 with zero added ops headcount. The modeled figure is honest because the volume is; the scaling is the record." *"Why ±1.6%?"* — "n = 300 at an expected 98% accuracy gives a 95% confidence interval of about ±1.6 points — big enough to certify the gate, small enough to afford. That measurement path cut rollout from 6 months to 1.5 and carries the separate ~$1.5M record."

THE MATH — FULL DERIVATION, STEP BY STEP

```
WBR arithmetic (copied from the LSA WBR pattern, not invented). Week = Mon–Sun; show the
most
  recent 5 weeks plus monthly roll-ups. Week-over-week WoW = (B − A) / A where A = prior
week, B
  = current; green/red at ±5% relative; every input metric tagged to the output line it may
move
  ("RollUp").
Seven frozen metrics. coverage_auto = containers tracked with no human touch / active
  containers; coverage_with_human = tracked incl. human review / active; tickets = support
  tickets; tickets_per_active_container = tickets / active containers; 4PL_SLA% and 3PL_SLA%
=
  milestones within SLA / milestones due; delta = 4PL − 3PL.
Honest denominators. Active container = touched within 15 days (the inactive rule) — a
container
  that stops moving stops inflating coverage.
Cost-to-serve per container. CTS = (human interactions per container × loaded $/interaction)
;
  canonical $0.66 → $0.03  (0.03 − 0.66)/0.66 = −95.5% . Cross-walk to the platform's $0.18
    per
  eliminated interaction: $0.66/$0.18 ≈ 3.7 residual human interactions per container
before;
  $0.03/$0.18 ≈ 0.17 after.
Automation grains (all real, different instruments). Event grain: 1,991/10,288 = 19.4%
human;
  weekly stock automation 40.2% (13 Dec) · 40.0% (17 Jan) · 21.9% (31 Jan red week) · 69.2%
(21
  Feb, active 287 → 546); ticket grain 538 auto / 456 human = 54.1%. Headline: automation
40% →
  90%; accuracy 60% → 90%. Why 99.4% was discarded. Three touches in a week  automation = 1
–
  h/N with N tiny; a 95% CI on N = 3 is wider than the metric ("Wilson interval ≈ [0.44,
    1.0]" —
  illustrative math on the real decision). A number that cannot survive the next Monday
teaches
  the VP the wrong thing. Illustrative WoW ("not program data" for these two values). A
    = 40.0%,
  B = 21.9%  WoW = −45%  red; root cause a carrier format change (measurement), not
operators.
Sanity. Rates ≤ 100%; WoW undefined if A = 0 (rule: show "n/a"); CTS reduction and
    automation
  rise move together.
```

SENSITIVITY — LOW / BASE / HIGH

INPUT	LOW	BASE	HIGH
Containers per shipment (benchmark)	1.5	2.5	4.0
Containers/yr at 32K shipments	48K	80K	128K
Direct CTS saving ($0.63 × containers)	$30K	$50K	$81K
ARR per customer (benchmark, illustrative)	$10K	$25K	$50K
Revenue pull-forward (one-time)	$102K	$254K	$508K
WBR labor released ($/hr 80–120)	$17K	$21K	$25K
Revenue pull-forward is one-time; $1.5M record excluded	—	—	—

INPUT	LOW	BASE	HIGH
Total modeled $/yr	≈$47K	≈$71K	≈$106K

TOTAL IMPACT STATEMENT — SAY IT LIKE THIS

"Honestly, the cost-to-serve number is a unit-economics result, not a big dollar number: ninety-five percent off per container is measured, and at the thirty-two-thousand-shipment target that is only about fifty thousand dollars a year of direct cost, which is my model on an illustrative container count. What it bought is scale: sixty-eight customers boarded on the same ops team with zero rollbacks, about two and a half people we never had to hire, and a rollout unlocked two months early, which I model at roughly a quarter million of one-time revenue pull-forward. The million and a half on the pre-prod measurement path is a separate record figure and I keep it separate. So: the ninety-five percent and the seven-to-sixty-eight are record; every dollar figure except the one-point-five is my model."

If challenged — the full answers.

Your containers-per-shipment is a guess.

It is, and I said so. The record gives me thirty-two thousand shipments, not containers; I used two and a half FCL containers per shipment, and at one and a half to four the saving runs thirty to eighty thousand. I would firm it up by pulling the active-container count straight from the fifteen-day active rule in the WBR, which is the same denominator the review already used.

Fifty thousand a year is small — why lead with it?

Because it is honest, and because the point was never the dollars, it was boarding nine point seven times the customers without nine point seven times the ops headcount. The hardest part of that claim is the one-minute-per-interaction assumption behind the two-and-a-half FTE; I would measure it from the ticket-grain automation data, five hundred thirty-eight auto versus four hundred fifty-six human.

Where does the million and a half come from, then?

That is the pre-prod measurement path, a separate canonical: three hundred containers at plus or minus one point six percent let us prove the product in six weeks instead of six months. It is a record figure, it is not cost-to-serve, and I never add it to the fifty thousand. To firm it up I would want the finance sign-off memo that priced the four and a half months saved.

S12 — Flexport: FF App export launch — operator-first vs partner-first, CTS −25% vs −50%, the 8-week rebuild/reuse rule

Amazon LPs: Think Big (primary — the flip) · Frugality (primary — ~$5M avoided) · Have Backbone (changing the manager's plan) · Earn Trust (privately first). **Crosswalk:** loop packet S12 — **§1.3 CONFLICT: the loop packet tells this as a Sephora roadmap; this packet's record is Flexport FF App.** Resolve before the loop. *Say the ~$5M once — never "avoidance" and "annual reduction" as two results.*

Honesty label. The roadmap was my manager's; I owned the sequencing recommendation. Engineering produced the re-estimates; the ~$5M is the record's cost-avoidance figure, said once; the run-rate decomposition is my model.

60-SECOND PITCH

"**Situation.** The Freight Forwarding App export roadmap was operator-first, 18 months, already socialized, and my manager owned it; baselines 339.5 min/shipment and 2,200 operator interactions per departed shipment. **Task.** With no authority over the roadmap, re-sequence it on cost-to-serve evidence. **Action.** Discovery: time study × loaded rates. Plan & Design: operator-first −25% CTS in 9 months vs partner-first −50% in 6; the 8-week rebuild/reuse rule; the reversal trigger written. Build & Test: engineering re-estimated with confidence buffers; I showed my manager privately first. Cutover: the Bellevue onsite ratified the flip; burn-up 55.8% → 83.2%. Hypercare: operators lived in old screens two more quarters — priced and published. Lesson: bring the model, name the owner, write the reversal trigger. **Result.** Primary metric: engineering effort to first value — the 18-month operator-first plan → **66% less effort on the partner-first path, value in 6 months instead of 18**. $ impact: ~$5M cost avoidance (record, said once); CTS −50% vs −25% at the work-item grain; 48 of 53 partners migrated within a month."

S — The inherited launch plan for the freight-forwarding app was operator-persona first, 18 months, already socialized, and my manager owned it. Baselines: 339.5 min/shipment time-to-serve, 2,200 operator interactions per departed shipment.

T — I owned the sequencing recommendation — and changing a plan with my manager's name on it.

A — Five gated phases:

1. **Discovery** — baseline memo (339.5 min/shipment; 2,200 interactions); bare-minimum scope per persona cut with the business; the question restated as time-to-value and unit cost, not persona preference. *Exit bar:* business agrees the bare minimums; engineering agrees to re-estimate with confidence levels.
2. **Plan & Design** — re-estimates (9 months operator vs 6 months partner) with confidence buffers (×1.0/×1.4 + 15%); the 20%-still-in-old-app finding; work-item-level cost-to-serve translation (−25% vs −50%); the rebuild-vs-reuse rule per module (>8 dev-weeks → reuse); read privately to my manager first; engineering got two days to break it. *Exit bar:* my manager puts his name on the flip at the Bellevue onsite; sequencing signed.
3. **Build & Test** — 15 domains × 6 modules (Booking Validated → Customs Declaration → …); Jira burn-up with the scope tripwire ("new work estimated and swapped, never absorbed"); operator-interactions-eliminated as the success metric; the tenet "no go-live without every work item assigned to one operator." *Exit bar:* burn-up 55.8% → 83.2% across Q3–Q4; every work item owned.
4. **Cutover & Go-Live (waves)** — MVP with 11 partners in 10 countries; partner migration plan published; operators' two further quarters on old screens priced and published. *Exit bar:* 48 of 53 import partners migrated within a month; MVP live.
5. **Hypercare / Close** — tracked interactions eliminated; sequenced FCL 2022 → Air/LCL 2023; close-out with ~$5M cost avoidance and 66% less effort. *Exit bar:* value in 6 months instead of 18; operators moved onto the shared chassis on the published schedule.

R — Primary metric: effort to first value **−66%; value month 6 instead of 18**. ~$5M cost avoidance (record, said once); CTS −50% vs −25%; 48/53 partners in a month.

IF PROBED — PHASE Q&A (13). Answer the phase asked, then stop.

Discovery — How did you know discovery was done?

When each persona had a bare-minimum scope the business had signed and engineering had agreed to re-estimate with confidence levels. Baselines — 339.5 minutes and 2,200 interactions per shipment — were on paper first.

Who wrote the success criteria?

I did: time-to-value and cost-to-serve at work-item level, with monthly operator interactions eliminated as the metric. The business signed the minimums; my manager signed the sequencing.

Plan & Design — How did you handle your manager's plan being wrong?

Privately first, with the numbers — 9 versus 6 months, 20% still in the old app, −25% versus −50%. He argued two points, I adjusted one, and the onsite ratified it with his name on it.

What was in your risk register at ratification?

Optimistic estimates, operators feeling abandoned, and scope absorbed silently. Mitigations: confidence buffers, a priced two-quarter schedule, and the burn-up tripwire.

What was the rebuild-vs-reuse rule?

Over eight dev-weeks to rebuild — reuse; under — rebuild; rank by benefit × impact divided by dev-weeks. Agreed with product, UX and engineering per module.

Build & Test — How did you sequence in Jira?

Fifteen domains by six modules, tracked as a burn-up against planned engineering days — 55.8% to 83.2% across two quarters. New work was estimated and swapped, never absorbed.

What was your definition of done for a module?

Every work item assigned to one operator and its interactions-eliminated count recorded. The tenet: no go-live without every work item owned.

Cutover — What did each wave deliver?

MVP: 11 partners in 10 countries end-to-end on the new backend. Migration: 48 of 53 import partners within a month. Then operators onto the same chassis, then Air/LCL in 2023.

What was your rollback?

Partners could keep working in the old flow during migration, and operators were still on old screens by design. Reversibility was built into the sequencing.

Hypercare — How did you know the flip was right?

Value in six months instead of eighteen, 66% less engineering effort, about $5M cost avoidance — said once — and operators moved onto the chassis on the published schedule.

What did you track after the partners were live?

Monthly operator interactions eliminated against the 2,200-per-shipment baseline, by work item, and the burn-up on the remaining modules. Air/LCL 2023 was scheduled off that read, not off a date on a slide.

Cross-phase — Where did you trade scope for date? What did you escalate?

Operators stayed on old screens two more quarters so partners finished end-to-end in six months — priced, published, scored. Escalated nothing past my manager. I have never won a disagreement by seniority — I rank, privately first, and let the room ratify.

How did your manager react?

He asked for the estimation details, argued two points, and put his name on the flip at the onsite. That's the point of private-first.

Where this lands at Amazon: Think Big with Frugality attached — re-opening a committed plan because the goal mattered more than the plan, priced in unit economics, ratified by the owner whose name was on the original. The reversal trigger and the scope tripwire are the mechanisms that made a big swing safe.

BUSINESS IMPACT MODEL — KPI TREE, NUMBERS, THE MATH

```
~$5M cost avoidance (record, said once) = effort avoided + value pulled in
+- Engineering effort to first value -66% (record)
|    +- rebuild-vs-reuse: >8 dev-weeks -> reuse · confidence x1.0/x1.4 + 15% buffer
|    +- burn-up 55.8% -> 83.2%; scope swapped, never absorbed
+- CTS per shipment: 339.5 min x ~$0.45/min ~= $153 derived
|    +- partner-first -50% vs operator-first -25% (record, work-item grain)
|    +- shared chassis loop: partner backend -> less rework -> faster operator move
+- Time-to-value 18 -> 6 months (record); 12 months of unit saving pulled forward
+- Cost of the trade: operators on old screens 2 more quarters (priced, published)
```

DRIVER	BASELINE	AFTER	$	BASIS
Time-to-value	18 months	6 months	—	record
Engineering effort to first value	100%	34%	≈$2.9M one-time	modeled (12 eng × 18 mo × 66% × $20K/eng-mo; benchmark $15–25K)
CTS per shipment	≈$153	≈$76 (−50%)	≈$1.5M/yr	record inputs, derived $ × 20K shipments (benchmark volume)
Value pulled forward 12 months	0	12 mo × $1.5M/yr	≈$1.5M one-time	modeled
Partners migrated in a month	0	48 of 53 (90.5%)	—	record

```
model check: $2.9M eng avoided + $1.5M pulled forward = $4.4M -- within ~12% of the
  record's ~$5M avoidance; the record number is the headline, the decomposition is mine
CTS derived: 339.5 min x ~$0.45/min blended loaded rate ~= $153/shipment
run-rate = $76 saved x 20K export shipments = ~$1.5M/yr (volume illustrative)
```

If challenged. *"Is the $5M savings or avoidance?"* — "Avoidance — engineering never spent and value pulled forward — and I say it once. The recurring run-rate is a separate, smaller modeled number; I never present the avoidance as an annual reduction." *"Your manager's plan — why not escalate?"* — "Because the fastest path to a better decision was making it his decision. Private first, two days for engineering to break it, then the room ratifies. I've never won a disagreement by seniority."

THE MATH — FULL DERIVATION, STEP BY STEP

```
Baselines. Time-to-serve 339.5 min/shipment; 2,200 operator interactions per departed
shipment;
  loaded per-minute cost by pool FCL $0.56 · Network $0.61 · Data Services $0.24 (weighted ≈
  $0.45/min). Cost-to-serve per shipment CTS = Σ_pool minutes_pool × rate_pool ;
illustrative on
  the canonical inputs: 339.5 min × ~$0.45 ≈ $153/shipment (derived, labeled).
Two paths. Operator-first: 9 months to a state where operators still spend 20% of time in
the
  old app  CTS_op = −25% . Partner- first: 6 months to partners completing end-to-end in the
new
  app, and the partner backend is the chassis operators later ride CTS_partner = −50% .
  Time-to-value 18 → 6 months.
Illustrative dollarization ("not program data" beyond the canonical ~$5M). At CTS ≈
    $153 and
  −50% vs −25%: partner-first removes ≈ $76/shipment vs ≈ $38 — double the unit saving,
reached
  a year earlier. The program's canonical figure: ~$5M cost avoidance (say once); 66% less
  engineering effort. Rebuild-vs-reuse rule (per module). if dev_weeks > 8: reuse else
rebuild
    ;
  rank by BBI = benefit × impact / dev_weeks . The 8-week rule was the engineering median.
Confidence-weighted estimates. est_adj = est × CL with CL ∈ {×1.0, ×1.4} by confidence, plus
15%
  testing buffer: plan = Σ est_adj × 1.15 . Illustrative: two modules at 10 wks (high
  confidence) and 10 wks (low)  (10 + 14) × 1.15 = 27.6 wks planned vs 20 naive.
Burn-up tripwire. completed/pointed : 55.8% (Q3 8/30) → 63.3 → 66.0 → 69.3 → 76.0 → 83.2%
(Q4
  11/15); new scope estimated and swapped, never absorbed.
Result. MVP: 11 partners / 10 countries; 48 of 53 import partners (90.5%) migrated within a
  month.
Sanity. −50% ⊃ −25% because the partner backend is shared; the operator path's residual 20%
  legacy time caps its saving; buffers only inflate low-confidence items.
```

SENSITIVITY — LOW / BASE / HIGH

INPUT	LOW	BASE	HIGH
Engineers on the plan (illustrative)	10	12	15
Loaded $/eng-month (benchmark)	$15K	$20K	$25K
Eng $ avoided (one-time)	$1.8M	$2.9M	$4.5M
Export shipments/yr (benchmark)	15K	20K	30K
Unit saving at −50% ($76 × vol)	$1.1M	$1.5M	$2.3M
Value pulled forward 12 mo (one-time)	$1.1M	$1.5M	$2.3M
Decomposition total (one-time, vs record ~$5M)	$2.9M	$4.4M	$6.8M
Total modeled $/yr	$1.1M	$1.5M	$2.3M

TOTAL IMPACT STATEMENT — SAY IT LIKE THIS

"About five million of cost avoidance, and I say that once. Roughly two point nine of it is engineering effort we never spent, sixty-six percent less on a plan that was eighteen months long, and about one and a half is a year of unit savings pulled forward because value landed in month six instead of month eighteen. Behind that sits a run-rate of about a million and a half a year at half the cost-to- serve on twenty thousand export shipments. The sixty-six percent, the six versus eighteen months, and the minus fifty versus minus twenty-five are the record; the five million is the program's signed figure; the split between engineering and pulled-forward value, and the shipment volume, are my model."

If challenged — the full answers.

Twelve engineers and twenty thousand dollars a month are your numbers, not the program's.

Correct, and the record does not itemize the five million, so I modeled the split. Ten to fifteen engineers at fifteen to twenty-five thousand puts the effort piece anywhere from one point eight to four and a half million; the record's five million sits inside that band. I would firm it up with the two scoping envelopes, sixty-six point two and ninety-nine point eight engineer-months, and finance's actual loaded rate.

Where do twenty thousand shipments come from?

It is an illustrative FCL export volume, and it is the softest input: at fifteen to thirty thousand the run-rate moves from one point one to two point three million a year. The three-thirty-nine-and-a-half minutes and the per-minute rates are the time study; I would replace the volume with the departed-shipment count from the same system that produced the two thousand two hundred interactions baseline.

Are you double counting the pulled-forward value and the five million?

No. The five million is the total; the pulled- forward year is one slice of how it decomposes, and the million and a half a year is the run-rate underneath, not a second result. To make that airtight I would ask for the finance close-out that priced the avoidance and show the two slices summing to it.

S13 — Manhattan: Cyber Monday, 4 AM — allocation across 3 DCs and 100 stores when the scheduler died

Amazon LPs: Bias for Action (primary) · Customer Obsession (primary backup — the promise ranked first) · Deliver Results. **Crosswalk:** loop packet S14 — consistent. *Older story — if asked for recent, pair with the UDC robotics throttle (S2): same muscle, 2024–25. Never "$2MM baseline."*

Honesty label. OMS consultant on allocation, promising and the OMS–WMS handshake; engineering fixed the scheduler once I isolated it; the post-mortem mechanisms were mine to drive. $8M/$6M/99% are record; the margin and cost figures are my model at benchmark rates.

60-SECOND PITCH

"**Situation.** Manhattan OMS consultant on allocation and promising across 3 DCs and 100 stores; 4 AM Cyber Monday the VP called — backorders piling with inventory present, ~$6M of sales at risk against a 95% fill-rate goal. **Task.** Own the response before the root cause was known. **Action.** Discovery: the contradiction (ATP > 0, backorders rising) was the entry bar to declare an incident. Plan & Design: I ranked by promise-at-risk, not FIFO; node choice = the promise — near 0.85 days vs far 4.60. Build & Test: this was an incident — one bridge, every rebalance timestamped; reason-coded alerts I had already built. Cutover / clear: rebalanced so no DC drowned; status every two hours until 9 PM. Hypercare / close: root cause — the allocation scheduler died silently at the daylight-saving change; post-mortem shipped a heartbeat, a DST checklist, a 3σ page rule, and a rehearsed rebalance playbook. Lesson: decide reversibly, log it, find the cause later. **Result.** Primary metric: peak fill rate — 95% goal → **99% delivered**. $ impact: ~$8M of peak sales protected against ~$6M at risk (record); my model puts that at ≈$3.2M of gross margin at a 40% benchmark margin; structural stockouts −20% over the engagement."

S — OMS consultant on order allocation, promising and the OMS–WMS handshake for a strategic retailer — 3 DCs, 100 stores — on call 6 AM–9 PM for the holiday promotion. 4 AM Cyber Monday, the VP of Supply Chain on the phone: backorder lines piling up with plenty of inventory; promises aging; ~$6M of sales at risk against a 95% fill-rate goal.

T — Nobody had written a clearing rule for failure. I owned the response.

A — Five gated phases — the incident's phases (hours), plus the post-mortem program (weeks; durations illustrative):

1. **Discovery (04:00–05:00)** — triage read: symptom (allocation stalled), exposure (~$6M vs the 95% goal), what I did not know (root cause); decided to act before the cause. *Exit bar:* bridge open; exposure sized; the clearing question framed as "which orders first," not "what broke."
2. **Plan & Design (first hour)** — the clearing rule (rank by promise-at-risk, not FIFO), the rebalance plan across three DCs (near node ~0.85 d vs far ~4.60 d), the two-hourly comms cadence onto the bridge log. *Exit bar:* rule on the log; VP agrees to the ranking.
3. **Build & Test (morning)** — manual clears in promise-at-risk order; rebalanced allocation so no node drowned while others idled; read my reason-coded alerts for the cause. *Exit bar:* backorder queue draining; alerts point to the scheduler.
4. **Cutover (root-cause fix)** — handed the isolated DST scheduler death to engineering with the evidence; verified automated allocation resumed; timestamped the log. *Exit bar:* scheduler restored; manual clears stopped; fill trending to 99%.
5. **Hypercare / Close (post-mortem)** — blameless post-mortem off the log timeline; mechanisms driven: scheduler heartbeat, DST checklist, 3σ page rule, rehearsed rebalance playbook, surge-scaling rebuilt. *Exit bar:* mechanisms live before the next peak; "the next peak ran boring."

R — Primary metric: peak fill **99% vs the 95% goal**; ~$8M protected vs ~$6M at risk (record); structural stockouts −20% over the engagement.

IF PROBED — PHASE Q&A (12). Answer the phase asked, then stop.

Discovery — How did you decide without knowing the cause?

The queue was aging while inventory sat idle; waiting for a root cause would have cost promises by the hour. I ranked by promise-at-risk first and hunted the cause in parallel.

How did you size the exposure?

Backorder lines against the 95% fill-rate goal — about $6M at risk. That number went into the first status and every one after.

Plan & Design — Why promise-at-risk and not FIFO?

FIFO processes cheap orders while at-risk promises age. Promise-at-risk clears the orders whose miss costs most first — the ranking is the policy.

How did you handle the VP wanting the fastest clear?

One sentence on the difference between fastest and best, then the rule written in the log so nobody re-argued it at noon.

Build & Test — How did you rebalance without making it worse?

Checked with engineering that manual clears would not corrupt allocation, then moved load by node distance — near ~0.85 days versus far ~4.60 — logging each rebalance with a timestamp.

How did you find the root cause?

Through my own reason-coded alerts: the pattern pointed at the allocation scheduler, which had died silently at the daylight-saving change. Engineering fixed it once isolated.

Cutover — How did you know automated allocation was safely back?

Manual clears stopped, the scheduler processed a batch on its own, and fill trended to 99%. The log shows the handoff timestamp.

What was your "rollback"?

None available — the equivalent was the rehearsed rebalance playbook the post-mortem produced, so the next failure had a tested manual mode.

Hypercare — How did you run the post-mortem?

Blameless and from the log: timeline, contributing causes, mechanisms with owners — heartbeat, DST checklist, three-sigma page rule, rehearsed playbook, surge-scaling rebuilt.

How did you know the mechanisms worked?

The next peak ran boring: the heartbeat paged nothing because nothing died, the DST checklist ran before the change, and structural stockouts fell 20% over the engagement. The VP's appreciation email was nice; the boring peak was the evidence.

Cross-phase — What did you escalate and to whom?

Two-hourly status to internal and client leadership until 9 PM — same numbers each time.

This was years ago — anything more recent? · Do differently?

The UDC robotics throttle (S2) is the same muscle in 2024–25: a pre-built dial and a published gate — decide on a rule written before the crisis. Differently: the DST checklist should have existed before that Monday, and I'd page on-call engineering sooner instead of carrying the first stretch solo.

Where this lands at Amazon: peak is sacred and the scheduler always dies at the worst hour. The transferable mechanics: rank the clear by promise-at-risk, keep every intervention reversible and timestamped on one log, communicate on a fixed cadence with the same numbers — and leave behind a heartbeat so the next failure is detected in one interval, not by a VP at 4 AM.

BUSINESS IMPACT MODEL — KPI TREE, NUMBERS, THE MATH

```
Peak sales protected ~$8M vs ~$6M at risk (record pair)
+- Peak fill rate 95% goal -> 99% delivered (record)
|    +- backorder queue (stock) drained by promise-at-risk, not FIFO
|    +- node choice = the promise: near 0.85 d vs far 4.60 d (record)
|         load_n = queue_n / capacity_n -> re-point -> loads level (balancing loop)
|    +- scheduler liveness: heartbeat + DST checklist + 3-sigma page ->
|         detect in 1 interval, not by a VP at 4 AM
+- Gross margin on protected sales (= $8M x margin, modeled)
+- Structural stockouts -20% over the engagement (record) -> margin run-rate (modeled)
```

DRIVER	BASELINE	AFTER	$	BASIS
Peak fill rate	95% goal	99%	—	record
Peak sales protected	~$6M at risk	~$8M protected	one-time event	record
Gross margin protected	—	≈$3.2M	one-time	modeled ($8M × 40%; benchmark 35–45%)
Cancel / CS contacts avoided	≈40K × $6.5	0	≈$0.26M	modeled ($6M ÷ $150 AOV; CS $5–8)
Expedite premium avoided	—	near-node sourcing	≈$0.13M	modeled (benchmark $5–12/order)
Structural stockouts	—	−20%	≈$1.3M/yr	modeled margin run-rate, illustrative $400M retailer

```
margin protected = $8M x 40% = $3.2M one-time (benchmark margin 35-45%)
orders at risk = $6M / $150 AOV = ~40K -> CS avoided = 40K x $6.5 = $0.26M
node math: near 0.85 d vs far 4.60 d = -3.75 days of promise per re-pointed order
```

If challenged. *"The $8M — protected how?"* — "It's the record pair: ~$6M at risk at 4 AM, ~$8M of peak sales through the event at 99% fill. The margin translation is my model at a benchmark 40%. I never count the $8M as savings — it's sales that didn't die."

THE MATH — FULL DERIVATION, STEP BY STEP

```
Fill rate. FR = units (or lines) shipped complete by promise / units ordered ; goal 95%,
  achieved 99%. Dollars: ~$6M at risk (backorder lines × value) → ~$8M protected (orders
that
  shipped on promise through the event, canonical pair). Promise-at-risk ranking (instead of
  FIFO). For order o: risk_o = value_o × 1[promise_o − now ≤ processing_time_node] ×
  urgency(promise_o − now) ; clear in descending risk_o . FIFO processes by arrival — cheap,
  distant-promise orders first while at- risk promises age.
Node choice = promise. Ship time from node n to customer c: T_nc = handling_n + transit_nc ;
  canonical warbook illustration: near node 0.85 days vs far node 4.60 days. Assign n* =
  argmin_n T_nc s.t. ATP_n ≥ qty and node load below capacity; rebalance when any node's
    queue
  exceeds its daily capacity while others idle: load_n = queue_n / capacity_n , target max_n
  load_n − min_n load_n ≤ ε . Illustrative worked example ("not program data"). 3 DCs
    with daily
  capacity 10k / 8k / 6k orders; backlog 30k orders after the scheduler stall. Naive
allocation
  had DC1 at 22k (2.2 days), DC3 at 2k (0.33 days). Rebalance to loads 12.5k / 10k / 7.5k
each
  ≈ 1.25 days  worst-case promise slip falls from 2.2 to 1.25 days; orders with promise ≤
1.25
  days cleared first by risk_o. Three-standard-deviation page rule (post-mortem). For a
metric
  with mean μ and σ over a trailing window (e.g., allocations/hour), page if x < μ − 3σ (or
    > μ
  + 3σ). Illustrative: μ = 1,200/hr, σ = 150  page below 750/hr. A dead scheduler yields
0/hr
  pages within one interval instead of at 4 AM by a VP.
Sanity. FR ≤ 100%; protected $ ≥ at-risk $ is plausible because the clear covered orders
beyond
  the initial at-risk set through the event; loads sum to backlog after rebalance.
```

SENSITIVITY — LOW / BASE / HIGH

INPUT	LOW	BASE	HIGH
Retail gross margin (benchmark)	35%	40%	45%
Margin protected on $8M (one-time)	$2.8M	$3.2M	$3.6M
CS contact $ (benchmark)	$5	$6.5	$8
CS / cancel avoided (one-time)	$0.2M	$0.26M	$0.32M
Lost-sales-to-stockout share (benchmark)	2%	4%	6%
Illustrative annual revenue	$300M	$400M	$500M
Stockout −20% margin run-rate	$0.4M	$1.3M	$2.7M
Total modeled $/yr	$0.4M	$1.3M	$2.7M

TOTAL IMPACT STATEMENT — SAY IT LIKE THIS

"About eight million of peak sales protected against six million at risk, and ninety-nine percent fill against a ninety-five goal; those three numbers are the record. Underneath them my model says roughly three point two million of gross margin at a forty percent retail margin, about a quarter million of cancellations and service contacts we never handled, and a bit over a hundred thousand of expedite we never paid because the near node was the promise. Then the twenty percent cut in structural stockouts, which is record, models to about a million three a year of margin on an illustrative four-hundred-million retailer. So: the eight, the six, the ninety-nine and the twenty percent are measured; the margin, the contact cost and the run-rate are mine."

If challenged — the full answers.

How can you protect eight million when only six was at risk?

Because the six was the backlog on the phone at four in the morning and the eight is what shipped on promise through the whole event, including the four points above goal. Reconciling them implies about fifty million of event order value, which is my derivation, not a record number; I would confirm it against the retailer's Cyber Monday order total.

The stockout run-rate is the biggest number and the softest.

Agreed, it is the hardest one. It rests on a four percent lost-sales-to-stockout benchmark and an illustrative four-hundred-million revenue base; at two to six percent it runs from four hundred thousand to two point seven million. I would measure it from the retailer's own out-of-stock lines times unit margin before and after the playbook went in.

Forty percent margin and a hundred-fifty AOV are assumptions.

They are, and I labeled them benchmarks. Thirty-five to forty-five percent moves the protected margin from two point eight to three point six million; the AOV only moves the contact- cost line. I would swap both for the retailer's finance numbers, which is exactly what I would ask for on day one of the post- mortem.

S14 — Manhattan: the $2.5M OMS↔WMS discrepancy my manager wanted to fix quietly — transparency as the reconciliation control

Amazon LPs: Earn Trust (primary — the flagship integrity story) · Have Backbone (backup — refusing the quiet fix) · Ownership · Bias for Action (third — two days to the review). **Crosswalk:** not in the loop packet — safe anywhere. *Pair with S12 for a recent upward disagreement; S4 for the one where I was wrong. The $10M initiative is never counted as savings.*

Honesty label. The census, the risk argument, the customer pack and the war room were mine; the consultants traced the handshake, wrote and verified the per-line patch, and built the permanent fix.

60-SECOND PITCH

"**Situation.** Two days before an executive review at a Manhattan customer, OMS and WMS did not reconcile — about two hundred orders, $2.5 million — and a $10 million initiative rode on that review; my manager and the room favored a quiet back-end correction. **Task.** I was the consultant who would run whichever path was chosen. **Action.** Discovery: I built the discrepancy census by cause on a sample of the two hundred before proposing any path, and named the fear — losing the initiative — as legitimate. Plan & Design: I disagreed on risk, not principle — a cross-table script written in two days could not be verified, and an IT audit would turn a data issue into a reputation issue — and offered to run the war room personally so the cost of my position landed on me. Build & Test: customer pack with root cause, plan and timeline; a per-line, reversible patch method. Cutover: the customer prioritized their own orders; we patched that set with before/after evidence per order and briefed them before the review. Hypercare: a clean root-cause fix to the handshake and a standing reason-coded reconciliation report. Lesson: silent corrections are loans against trust at a terrible interest rate. **Result.** Primary metric: OMS↔WMS discrepancy — **~200 orders / $2.5M → zero** on the customer's prioritized set before the review, then zero through the clean fix. $ impact: the $10M initiative won (record, not counted as savings)."

S — Two days before an executive review, OMS and WMS didn't reconcile — ~200 orders / $2.5M — and a $10M initiative rode on the review. My manager and the room favored a quiet back-end correction.

T — I was the consultant who would run whichever path was chosen; I owned the recommendation and then the war room.

A — Five gated phases — two days plus the war room (durations illustrative):

1. **Discovery (day 1, morning)** — the discrepancy census by cause (which handshake step, which order states); the exposure statement (review in two days; $10M initiative); both options named honestly — quiet script vs transparency. *Exit bar:* the room agrees the facts before the path; the fear of losing the initiative named as legitimate.
2. **Plan & Design (day 1, afternoon)** — the options memo on risk, not principle: a two-day cross-table script is error-prone; an IT audit turns a data issue into a reputation issue; transparency = root cause + action plan + I run the war room. *Exit bar:* manager accepts, hesitantly; my labor attached to my position.
3. **Build & Test (day 2)** — the customer communication pack (root cause, plan, timeline — no apology-first); the war-room plan (bridge, log, cadence); the verification method for any patch (per line, reversible, before/after recorded). *Exit bar:* customer briefed before the review; war room staffed.
4. **Cutover (review + patch)** — took the customer's prioritized order list; ran the bulk patch per line with verification; scheduled the clean root-cause fix. *Exit bar:* patch verified; discrepancy to zero on prioritized orders; review held with the facts on the table.
5. **Hypercare / Close** — the permanent OMS–WMS handshake fix; the reconciliation report as a standing control read before every executive review. *Exit bar:* $10M initiative won; handshake reconciles; new discrepancies = 0 on the daily report.

R — Primary metric: discrepancy **~200 orders / $2.5M → 0**; the $10M initiative won (record, not counted as savings); the standing reconciliation control outlived the war room.

IF PROBED — PHASE Q&A (12). Answer the phase asked, then stop.

Discovery — How did you establish the facts under a two-day clock?

Reconciliation report by cause on a sample of the ~200 orders — which handshake step, which order states — before proposing any path. Facts first made the options argument about risk, not virtue.

Who did you tell first?

My manager, privately, with the census. The room heard the options after he had.

Plan & Design — What was in your risk register?

A silent fix found by IT audit, a rushed cross-table script corrupting data, and the customer being surprised at the review. Mitigations: transparency, per-line reversible patch, brief them first.

How did you disagree with your manager?

On risk: an unreliable script plus an audit exposure versus a customer told the truth with a plan. Then I offered to run the war room so the cost of my position landed on me. He accepted, hesitantly.

What if he had said no?

I would have documented the risk assessment in writing and asked for the decision to be recorded with its owner — disagree and commit, with the tripwire (any audit query on those orders) named. I would not have run the silent script without that record.

Build & Test — What was your definition of done for the patch?

Every line verified against both OMS and WMS, reversible, executed in the customer's priority order. No bulk script across related tables without per-line verification.

How did you prepare the customer conversation?

Root cause, action plan, timeline — no apology-first. The pack went to their IT and finance before the executive review, not in it.

Cutover — What did the customer do?

Prioritized their own orders, asked for a bulk patch on that list, and gave us time for a clean root-cause fix. The review happened with the facts already on the table.

What was your rollback?

Per-line reversibility on the patch. A script across every related table would have had none — that was the core of my risk argument.

Hypercare — What control outlived the war room?

The OMS↔WMS reconciliation report as a standing check, read before every executive review rather than two days before one. The handshake fix removed the cause; the report made sure the next one surfaced early.

Cross-phase — What did you refuse and what did it cost?

A silent correction. Cost: my time in a war room and the risk of being wrong in front of a customer. Benefit: a relationship that survived an audit and won the initiative.

Anything more recent where you disagreed upward?

S12 — flipping my manager's roadmap — and S4, where I was the one who was wrong. Same posture: disagree on evidence, carry the consequence.

Where this lands at Amazon: the purest Earn Trust answer in the set — trust built by telling the customer first, pricing the honest path's cost onto myself, and leaving a standing control so the courage is rarely needed again. The transferable control is not the courage; it is the reason-coded reconciliation read before every review.

BUSINESS IMPACT MODEL — KPI TREE, NUMBERS, THE MATH

```
Customer trust that survives an audit -> $10M initiative won (record)
+- OMS<->WMS discrepancy ~200 orders / $2.5M -> 0 (record)
|    +- recovery: $2.5M made consistent in the open; per-line reversible patch,
|         before/after evidence per order; customer-prioritized set first
|    +- prevention: standing reason-coded reconciliation (daily)
|         exposure at detection: $2.5M/60 d -> ~1 day (loop keeps the stock near 0)
+- Risk not taken: silent multi-table script
|    +- P(script error) x blast radius + P(audit) x reputation = avoided expected loss
+- Trust loop (+): tell first -> customer sets priority -> gives time ->
     clean fix -> initiative won -> more latitude next time
```

DRIVER	BASELINE	AFTER	$	BASIS
OMS↔WMS discrepancy	~200 orders / $2.5M	0	—	record
$10M initiative	at risk	won	—	record — **never counted as savings**
Standing reconciliation (prevention)	war rooms	daily report	≈$0.1M/yr	modeled (manual fixes + stuck cash + war rooms avoided; range $55–160K)
Silent script not run	—	—	≈$1.5M expected corruption + audit finding	modeled — avoided risk, not a saving

If challenged. *"You're claiming the $10M?"* — "No — the initiative was won in the review the facts survived; I claim the control and the posture, not the contract value. The only recurring dollar I model is the ≈$0.1M/yr the standing reconciliation saves, and it's labeled."

THE MATH — FULL DERIVATION, STEP BY STEP

```
Discrepancy. Δ = {orders whose OMS status ≠ mapped WMS status} ; canonical ~200 orders /
$2.5M
  average ≈ $12.5K per order (derived). Stakes: $10M initiative riding on the executive
review
  two days out.
Risk comparison (the argument I made). Quiet back-end correction: E[loss] = P(script error)
×
  blast radius across related tables + P(audit discovery) × reputation cost ; with a multi-
table
  script written in under two days, P(script error) is high and P(audit discovery) ≈ 1 over
an
  IT-audit horizon. Transparent path: E[loss] = P(customer displeasure now) × delay cost —
  bounded, one-time, and the customer controls it. Illustrative (not program data): if
P(script
  error) = 0.3 and a bad script corrupts 2× the affected orders, expected corrupted value ≈
0.3
  × $5M = $1.5M plus a near-certain audit finding on a $10M relationship — versus a few
days'
  delay on a subset of $2.5M.
Reconciliation KPI going forward. match_rate = orders with OMS status == map(WMS status) /
  orders in flight ; mismatch aging report with reason codes ∈ {status mapping, timing,
failed
  message, manual edit}. Σ reason classes = |Δ| — a row with no class is an investigation,
not a
  bucket.
Bulk patch scoping. Customer-prioritized set S_p ⊆ Δ ; patch applied to S_p first with
  before/after evidence per order (count of evidence rows = |S_p|); remainder fixed cleanly
  through the root cause; then assert |Δ_new| = 0 daily.
Sanity. Δ$ = Σ order values ≈ $2.5M over ~200 orders; the quiet fix has a tail the
transparent
  path does not; trust cost compounds.
```

SENSITIVITY — LOW / BASE / HIGH

INPUT	LOW	BASE	HIGH
Discrepancies arising per month (illustrative)	50	100	150
Cost per manual fix (benchmark)	$20	$30	$40
Manual-fix labor avoided	$10K	$29K	$58K
Cost of capital (benchmark)	8%	10%	12%
Order-to-cash carry avoided	$24K	$30K	$36K
Repeat war rooms avoided per year (illustrative)	1	2	3
War-room cost avoided	$22K	$43K	$65K
Total modeled $/yr	≈$55K	≈$0.10M	≈$0.16M

TOTAL IMPACT STATEMENT — SAY IT LIKE THIS

"Two and a half million of order value made consistent in the open, with before-and-after evidence on every order, and the ten-million initiative won; both of those are the record, and neither is a saving I claim. What the standing reconciliation is worth going forward is small and honest: about a hundred thousand a year in my model, roughly thirty in manual fixes we no longer do, thirty in cash that is not stuck behind mismatched orders, and forty in war rooms we do not run. The bigger number is the one we did not lose: the silent script had an expected one and a half million of corruption plus an audit finding on a ten-million relationship, and that is risk math, not money in the bank. So the two point five and the ten are measured; the hundred thousand and the one point five are modeled."

If challenged — the full answers.

A hundred thousand a year is small for a story you lead with.

Yes, and I would rather say small and true. The story is about the control and the choice, not the run-rate. The hardest input is how many discrepancies arise per month; I assumed the two hundred built up over about sixty days. I would firm it up by reading the daily match report's reason-code counts for a quarter and pricing exactly what surfaced.

You are counting the ten million as impact.

I am not. The ten million was won at the review; it is a record fact about the relationship, and I never put it in a savings column. The one point five million is the record's own illustration of the expected loss from the silent script, so it is avoided risk, not a dollar earned. To firm it up I would want the actual error rate of comparable rushed scripts from that practice.

Thirty dollars a fix and ten percent cost of capital are assumptions.

Both are benchmarks and I ranged them: twenty to forty a fix and eight to twelve percent move the total from roughly fifty-five to a hundred sixty thousand. The exception-handling minutes I would measure from the war-room log itself, which timestamped every per-line patch; the cost of capital I would take from the customer's finance team.

S15 — Sephora: converting a 20-person Product/Eng/QA team to a 4-week cadence — and the QA lead who now owns the gate

Amazon LPs: Hire and Develop the Best (primary) · Earth's Best Employer (primary — QA elevated, not blamed) · Insist on the Highest Standards (the gate). **Crosswalk:** not in the loop packet — safe anywhere. *Never invent a direct-report headcount.*

Honesty label. VP air cover, no line authority over anyone in the room; the QA lead authored the definition-of-done — I coached the frame, she owned the gate. The −50% and the six weeks are record; the dollars are my model on illustrative defect counts.

60-SECOND PITCH

"**Situation.** The VP asked me to move a 20-person Product/Eng/QA team from a six-week hybrid cadence to four-week agile; QA feared compressed test windows — and they were right. **Task.** Convert in weeks without line authority. **Action.** Discovery: one-on-ones; influencers per function. Plan & Design: the QA lead authored the definition-of-done — I coached the frame, she owned the gate. Build & Test: Jira benefit-ranking and review-status fields; a QA drop date with reason codes. Cutover: 4-week cadence live in 6 weeks. Hypercare: QA on product-business calls with pre-approved scenarios; she defended the gate without me. Lesson: development that depends on me staying is not development. **Result.** Primary metric: critical defects per release — baseline → **−50%** after the 20-person team moved to a four-week cadence in six weeks. $ impact: ≈$130K/yr modeled — the −50% and the six weeks are the record; the dollars are my model; no headcount invented."

S — Newly joined, the VP of Supply Chain asked me to move a 20-person Product/Engineering/QA team from a six-week hybrid cadence to four-week agile. QA (a contracting firm) and engineering (in-house) were heading for a standoff: QA feared compressed test windows — and they were right.

T — Convert the cadence in weeks, not quarters, without the standoff becoming the story. VP air cover, no line authority over anyone in the room.

A — Five gated phases (week markers illustrative):

1. **Discovery (weeks 1–2)** — one-on-ones with every person (objections, motivations); an influencer named per function; the problem statement — QA's fear of compressed test windows is correct — and the success criteria (four-week cadence, no rise in escaped criticals). *Exit bar:* every function's real objection heard privately; the QA lead agreed to author the gate.
2. **Plan & Design (weeks 2–3)** — the ceremony calendar for a four-week cycle; the definition-of-done *frame* (what a gate needs to survive pressure — the QA lead wrote the content); the Jira field spec (benefit ranking, review status); the user-guide outline. *Exit bar:* DoD with entry criteria written by QA and reviewed with engineering; fields live in Jira.
3. **Build & Test (weeks 3–5)** — the user guide (grooming → completion); the live dashboard; the QA drop date with reason codes; QA seated on the product-business calls with pre-approved scenarios. *Exit bar:* cycle 1 completed on the new cadence; drop-date rule in force; Sev-1 cause classification running.
4. **Cutover (week 6)** — facilitated the retro once; gave the VP a data read-out; handed the gate to the QA lead to defend herself. *Exit bar:* converted in 6 weeks; critical defects −50% under the drop-date rule.
5. **Hypercare / Close** — stepped back from the review line; neighbouring teams copied the ceremony from the user guide. *Exit bar:* the gate survived my leaving; Sev-1 trend from story-accuracy defects reversed.

R — Primary metric: critical defects per release **−50%**; 20-person team on a four-week cadence in **six weeks**; the gate defended by its owner, not by me.

IF PROBED — PHASE Q&A (14). Answer the phase asked, then stop.

Discovery — How did you know discovery was done?

When I had heard every function's real objection privately and the most skeptical person — the QA lead — had agreed to author the gate. Nothing was scheduled before it.

Who wrote the success criteria and who signed?

I wrote them with the VP: four-week cadence, no rise in escaped criticals, QA window protected by rule. The VP sponsored them; the team signed by writing the DoD themselves.

Plan & Design — What was in your risk register?

A QA–engineering standoff becoming the story, dev consuming the test window, and Sev-1s mis-attributed to QA. Mitigations: the skeptic owns the gate, a drop date with reason codes, cause classification with QA on product calls.

Why hand the load-bearing artifact to the skeptic?

A gate the skeptic owns survives pressure; one I write dies when I leave. I coached the frame — what a gate needs to say no — and she wrote the content.

How did you handle engineering objecting to the DoD?

Review, not veto — they shaped the entry criteria and I covered the low-impact stories they had to drop. Objection became co-ownership.

Build & Test — How did you sequence stories in Jira?

By the DoD: a story entered QA only when its entry criteria were met, tracked by the review-status field. Late heavy stories got a drop date with a reason code instead of a silent slide into the last two days.

What was your definition of done?

Hers, not mine — entry criteria per story type, evaluated at the Ready-for-QA transition. My frame: criteria that can say no and survive a release-day argument.

What replaced a bug bash?

The drop-date rule and demo-during-build. When dev landed heavy stories on the last two days, the reason codes made the pattern visible and criticals fell 50%.

Cutover — What made you say the conversion was done?

Two cycles on four weeks, the drop-date rule in force, criticals down 50%, and the QA lead defending the gate to engineering herself. Six weeks.

What was the rollback?

Reverting to six weeks — a two-way door I named at kickoff and never used.

Hypercare — How did you decide you could step back?

When the gate was defended without me and neighbouring teams copied the ceremony. Development that depends on me staying is not development.

What did you watch after the conversion to know it held?

Stories entering QA meeting their entry criteria, drop-date reason codes trending toward genuine re-scopes rather than late lands, and Sev-1 causes classified. Two cycles of that and I stopped attending the review line.

Cross-phase — Where did you trade scope for date? What did you escalate?

Low-impact stories dropped under the drop-date rule — dev's date protected, QA's window protected, and I covered some of the drops myself. Escalated nothing up; hard feedback went sideways through the issues-log rows — some of it turned out to be genuine overload, and the evidence got those people help.

Have you managed direct reports? · Do differently?

Yes — onshore/offshore delivery teams, SOWs and estimates earlier in my career. I have not run a large HR org with comp cycles and won't claim that muscle. Differently: go to QA first, and put her name on the review line from week one.

Where this lands at Amazon: the Hire & Develop answer with a mechanism: develop people by giving them the load-bearing artifact and the frame, not by writing it for them — and measure development by whether the gate survives your leaving. The Earth's-Best angle: QA moved from blamed afterthought to the owner of the standard, with the data protecting them.

BUSINESS IMPACT MODEL — KPI TREE, NUMBERS, THE MATH

```
Release quality at a faster cadence (escaped criticals/yr, $ avoided)
+- Releases/yr (flow): 52 / cycle weeks: 6w -> 4w = 8.7 -> 13 (+50%)
|    +- test window per cycle (stock of QA days) shrinks ->
|         DoD (QA-authored) -> stories arrive ready · drop date -> no last-2-day lands
+- Critical defects per release (record -50%)
|    +- story-accuracy Sev-1s -> QA on product calls with scenarios
+- Release-end rework (late-land re-test + hotfix) -> ~0 under the drop rule
(loop broken) Sev-1 blamed on QA -> standoff -> gate weakens -> more Sev-1s
   broken by: the skeptic owns the gate; cause classification, not blame
(loop built)  gate says no -> dev drops low-impact story -> window held ->
   fewer criticals -> gate trusted -> gate survives my leaving
```

DRIVER	BASELINE	AFTER	$/YR	BASIS
Cycle length / releases per year	6 wks / 8.7	4 wks / 13	—	record / modeled (52 ÷ weeks)
Critical defects per release	4 (illustrative)	2	—	record (−50%); count illustrative
Escaped-critical cost avoided	~$350K	~$260K	≈$90K	modeled (criticals/yr × $10K; benchmark $8–15K)
Late-land rework per release	~$4.8K	~$0	≈$40K	modeled (6 person-days × $800 × 8.7)
Total modeled	—	—	**≈$130K**	range $80–270K; no headcount

If challenged. *"Faster cadence usually means more escapes — why did yours fall?"* — "Because the gate got stronger as the window got shorter: entry criteria authored by the person with the most to lose, and a drop date that converted silent late lands into visible re-scopes. The −50% is the record; the defect counts behind the dollars are illustrative and labeled."

THE MATH — FULL DERIVATION, STEP BY STEP

```
Cadence change. Cycle length 6 weeks → 4 weeks  releases/year 8.7 → 13 (+50% release
  opportunities); test window per cycle shrinks proportionally unless the definition-of-done
  moves test left. QA's fear was arithmetic: if test needs T_test days fixed, then dev days
    per
  cycle fall from 30 − T_test to 20 − T_test (working days) — a real compression, which is
why
  the DoD and drop date exist.
QA drop date. Stories not QA-ready by day d_drop of the sprint are dropped with a reason
code;
  dropped-story share = dropped / committed trended per sprint. Critical defects −50% after
the
  drop-date rule (canonical). Sev-1 attribution. Sev1_story_accuracy / Sev1_total — the
share
  caused by wrong stories, not wrong tests; putting QA on product-business calls with
  pre-approved scenarios drove the trend reversal. Illustrative capacity check ("not program
  data"). 20 people, 4-week sprint, 15 eng × 18 working days = 270 eng-days; QA 4 × 18 = 72
  QA-days; if a story needs 3 eng-days + 1 QA-day, QA caps throughput at 72 stories and eng
at
  90  QA is the constraint — the drop date protects the constraint (theory-of-constraints
  framing from the MIT toolkit, labeled).
Conversion time. 6 weeks to convert; QA lead defending the gate by cycle 2 (≈ 8 weeks after
  start).
```

SENSITIVITY — LOW / BASE / HIGH

INPUT	LOW	BASE	HIGH
Criticals per release, baseline (illustrative)	3	4	6
Cost per escaped critical (benchmark)	$8K	$10K	$15K
Late-land rework per release (benchmark rates)	$3K	$4.8K	$8K
Escaped-critical cost avoided	~$55K	~$90K	~$200K
Late-land rework avoided	~$25K	~$40K	~$70K
Total modeled $/yr	~$80K	~$130K	~$270K

TOTAL IMPACT STATEMENT — SAY IT LIKE THIS

"Conservatively, about $130K a year, and I want to be clear that none of it is a record figure: roughly $90K is escaped-critical cost avoided, because criticals per release halved even as we shipped fifty percent more releases, and about $40K is release-end rework that the drop-date rule simply removed. What is measured is the cadence, six weeks to four, the conversion in six weeks, and the fifty percent drop in criticals per release. The dollars are my model on an illustrative four criticals per release and a benchmark ten thousand dollars per escaped critical, and I did not invent a headcount to get there."

If challenged — the full answers.

Where does four criticals per release come from?

It's illustrative, not from the record; the record gives me the fifty percent drop, not the base count. I'd firm it up by pulling the Sev-1 log for the last four six-week releases and using the actual mean.

Ten thousand dollars per critical is a soft number.

Agreed, it's a benchmark for hotfix plus regression plus incident time plus ops disruption; the range is eight to fifteen. I'd measure it by costing three real Sev-1s end to end with the team's loaded rates.

Fifty percent more releases could mean more total defects.

That's exactly why I count per year, not per release; halving per release at thirteen releases still nets nine fewer a year. I'd watch escaped criticals per quarter, which is the number the VP actually feels.

S16 — Flexport: four systems into one Snowflake pipeline — and the security-review miss I owned upward the day it surfaced

Amazon LPs: Ownership (primary — bus factor 1, and owning the miss) · Invent and Simplify (backup) · Frugality (the >$100K priced baseline) · Learn and Be Curious. **Crosswalk:** loop packet MAIN's BRD/pipeline beat + the security-review miss — consistent. *Same program family as S11 — never both to one interviewer.*

Honesty label. I wrote the BRD and owned the cutover; the infra team owned the build. The >$100K/yr is a *priced baseline* in the infra team's own loaded rates — priced, not claimed as savings. The two-week security-review miss is mine, owned upward the same day.

60-SECOND PITCH

"**Situation.** The WBR ran on four systems stitched in Sheets by me — bus factor 1, accuracy 60%, tickets with no container ID. **Task.** I wrote the BRD and owned the cutover; infra owned the build. **Action.** Discovery: froze seven metric definitions; priced the status quo at >$100K/yr in infra's own rates. Plan & Design: ETL → S3 → SQS → Snowflake → Looker; parallel-run gate within 1%. Build & Test: a cross-reference table first, then a real join key pushed upstream. Cutover: consecutive weeks inside 1%. Hypercare: my miss — I did not plan the data-access security review; it cost two weeks; I owned it upward the same day; zero review deliveries skipped. Lesson: every kickoff asks whose approval the data needs and its lead time. **Result.** Primary metric: WBR data accuracy — **60% → 90%** once four systems became one Snowflake pipeline. $ impact: the manual status quo was priced at >$100K/yr (a priced baseline, not a claimed saving); first automated catch was a 4% ticket-sync drop."

S — The Monday review ran on four systems that didn't talk, stitched in Google Sheets by one person — me: bus factor 1, accuracy 60%, 1.5–2 hours per snapshot, tickets carried no container ID. A VP was steering on a number one person could break.

T — Nobody had written requirements for a program's operating metrics. I wrote the BRD and owned the cutover; the infra team owned the build.

A — Five gated phases (week counts illustrative except the 2-week miss):

1. **Discovery** — documented the as-is (manual pulls + extracts in Sheets, 1.5–2 hrs per snapshot); named the join gap (tickets without container ID); priced the status quo at >$100K/yr in the infra team's own rates including VP decision risk. *Exit bar:* infra prioritizes it themselves off the price; scope = the seven metrics, nothing new.
2. **Plan & Design** — the BRD: seven frozen definitions, daily/hourly feed, failure alerts, a cross-reference table for the ticket join then a real key upstream; acceptance = parallel run within 1% for consecutive weeks. The RAID log — **missing the data-access security review**. *Exit bar:* BRD accepted by infra; acceptance gate published.
3. **Build & Test** — extract/load specs per system, parity checks, the parallel-run procedure; when the security review surfaced mid-build I re-sequenced and owned the miss upward the same day. *Exit bar:* review closed (2 weeks); parallel run within 1% for consecutive weeks; mismatches explained (mostly late-arriving tickets on the manual side).
4. **Cutover & Go-Live** — the cutover memo; the Monday deck moved onto the pipeline; the manual pull kept as rollback. *Exit bar:* first Monday off the pipeline, same seven numbers; zero WBR deliveries skipped through the build.
5. **Hypercare / Close** — watched the failure alerts; triaged the first automated catch (a 4% ticket-sync drop invisible in the manual era); definitions into code; institutionalized the kickoff question. *Exit bar:* accuracy 60% → 90%; the pipeline steers the review without me.

R — Primary metric: WBR accuracy **60% → 90%**; bus factor 1 → ≥2; first automated catch: a 4% ticket-sync drop. >$100K/yr priced baseline (record of the pricing).

IF PROBED — PHASE Q&A (12). Answer the phase asked, then stop.

Discovery — How did you know discovery was done?

When the as-is was documented — four systems, Sheets, 1.5–2 hours per snapshot, tickets without a container ID — and the status quo was priced in the infra team's own rates. They prioritized it off that number.

Who wrote the requirements?

I did — nobody had written requirements for a program's operating metrics. The BRD carried the seven frozen definitions and the acceptance gate.

Plan & Design — What was in your risk register at BRD acceptance — and what wasn't?

On it: the join gap, parity failure, bus factor one. Not on it: the data-access security review. That omission cost two weeks and is the item I now add at every kickoff.

How did you handle the join problem in design?

A documented cross-reference table so the pipeline could ship, then a real key pushed upstream to Support. Ship with the bridge, fix the source.

Build & Test — What was your acceptance test?

Manual versus pipeline within 1% for consecutive weeks, every mismatch explained by cause. Most mismatches turned out to be the manual side — late-arriving tickets.

How did you handle the security review mid-build?

Owned it upward the same day as a scoping miss, re-sequenced the build, and kept the WBR alive on the manual pull. Zero deliveries skipped; two weeks lost.

Cutover — What made you say go?

Parity within 1% for consecutive weeks and the review closed. The deck source switched; the VP saw the same seven numbers.

What was your rollback?

The manual pull — kept running until the pipeline had earned consecutive weeks of parity, and a week beyond.

Hypercare — How did you know it was working?

The first automated catch — a 4% ticket-sync drop invisible in the manual era — and accuracy moving from 60% to 90%. The alerts came to me, not silence.

What was your definition of "stable" after cutover?

Failure alerts arriving to me, not silence; the seven numbers matching the retired manual pull for a week beyond the parity gate; and the first real catch triaged to a source cause. Then the manual pull was retired.

Cross-phase — Where did you trade scope for date? What did you escalate?

New metrics for stability, and two weeks of date for a security review I should have planned. I would trade the second again; I would plan it now. Escalated the two-week miss, to my leadership, the day it surfaced.

Why didn't you know about the review?

I treated it as an engineering concern rather than a program dependency with a lead time. That's the category error I fixed — the standing question now: whose approval does this data need, and what is its lead time?

Where this lands at Amazon: security, privacy and data-access reviews are program dependencies with lead times — I paid for that lesson once and now ask the question at every kickoff. The rest is the Ownership pattern: price the status quo in the owning team's own rates so they prioritize it themselves, and keep the old path alive until the new one has earned parity.

BUSINESS IMPACT MODEL — KPI TREE, NUMBERS, THE MATH

```
VP steers the program on a number that is right and survives me
+- Status quo priced >$100K/yr (record; priced, not claimed)
|    +- manual hours (stock of my time) ~$30K + decision-risk term ~$70K
|         decision risk = P(wrong number) x cost of a wrong steer
+- WBR accuracy 60% -> 90% (record)
|    +- seven definitions frozen in code (no drift) · join rate -> ~100%
|    +- parity gate <1% consecutive weeks before cutover
+- Bus factor 1 -> >=2 (risk stock retired)
+- Silent data loss -> alerts; first catch: 4% ticket-sync drop
(loop broken) sheet built by hand -> only I can fix it -> more asks -> more errors
(loop held)   security review unplanned -> 2-wk slip -> WBR kept alive by hand ->
              zero skipped Mondays -> trust held -> cutover accepted
```

DRIVER	BASELINE	AFTER	$/YR	BASIS
WBR accuracy	60%	90%	—	record
Status quo priced	>$100K/yr	retired	—	record of the pricing (infra's own rates)
— hours inside that price	~4 hrs/wk	~0	≈$30K	modeled
— decision risk inside that price	P(wrong) × cost	≈$50K retired by accuracy gain	≈$70K	modeled
Bus factor	1	≥2 + alerts	≈$15K	modeled
Security-review slip	—	2 weeks	—	record (the named miss)

If challenged. *"You're claiming $100K of savings?"* — "No — pricing, not claiming. The number existed to make the infra team prioritize the build off their own rates, and it worked. The claimed record is the accuracy move and the first automated catch; the decomposition of the price is my model."

THE MATH — FULL DERIVATION, STEP BY STEP

```
Status-quo cost (priced with the infra team's own rates). C_manual = hours/week × loaded
    rate ×
  52 + decision-risk term ; canonical: > $100K/yr including VP decision risk, not just hours
  (1.5–2 hrs per snapshot, rising to ~4 hrs/wk).
Parity gate. For each frozen metric m and week w: |pipeline_mw − manual_mw| / manual_mw ≤ 1%
for
  consecutive weeks before cutover. Mismatches were mostly the manual side being wrong —
  late-arriving tickets.
Join problem. Tickets carried no container ID  join_rate = tickets with a resolvable
container /
  tickets ; interim cross- reference table (documented), then a real key pushed upstream so
  join_rate → ~100% .
Bus factor. bus_factor = 1 (one person could break the VP's number); target ≥ 2 by moving
  definitions into code with owners.
First automated catch. A 4% ticket-sync drop invisible in the manual era: sync_rate =
tickets
  landed in Snowflake / tickets in Zendesk ; alert on 1 − sync_rate > 1% (illustrative
  threshold).
Schedule impact of the miss. Security review = 2 weeks unplanned  re-sequenced; WBR
deliveries
  skipped = 0 because the parallel run kept the manual path alive. Illustrative parity check
  ("not program data"). coverage_auto manual 71.2% vs pipeline 70.9%  0.42% gap ;
    tickets 456 vs
  461  1.1% gap  → investigate: 5 late-arriving tickets missing from the manual pull  manual
  corrected; pipeline right.
Sanity. Parity measured per metric, not aggregate; two consecutive weeks minimum; a gap is
not
  "pipeline wrong" until the root cause says which side.
```

SENSITIVITY — LOW / BASE / HIGH

INPUT	LOW	BASE	HIGH
Loaded rate, hours (benchmark)	$120/hr	$150/hr	$200/hr
Cost of a wrong steer (benchmark squad-week)	$15K	$30K	$50K
P(wrong number → wrong steer) (illustrative)	5%	10%	15%
Key-person attrition (benchmark)	15%	20%	25%
Hours value	~$22K	~$27K	~$36K
Decision-error exposure retired	~$12K	~$47K	~$117K
Bus-factor exposure	~$8K	~$17K	~$31K
Total modeled $/yr	~$40K	~$90K	~$185K

TOTAL IMPACT STATEMENT — SAY IT LIKE THIS

"The record number is a price, not a saving: the manual status quo was priced at more than $100K a year in the infra team's own loaded rates, and that number is what got it prioritized. My model shows what sits inside it: about $30K is my hours, roughly four a week, and about $70K is decision risk, a VP steering on a number that was wrong forty percent of the time. Taking accuracy from sixty to ninety retires about $50K of that risk, and getting the bus factor above one is worth maybe $15K a year in expected loss. So the >$100K is record and priced; the $30K, $50K and $15K splits are modeled, and I will not call any of it realized savings."

If challenged — the full answers.

The decision-risk number is the soft part.

Yes, it rests on one wrong number in ten becoming a wrong steer worth a squad- week; the range runs from $12K to $117K. I'd firm it up by tagging every WBR action item for a quarter and counting how many were reversed when the number was corrected.

You never claimed the $100K, so why is it here?

Because it's the number the infra team used to prioritize the build; it's a priced baseline in their rates. What I'd measure now is the hours actually retired and whether any Monday number was restated after cutover, which is the honest realized version.

Bus factor is not a dollar.

Agreed, so I priced it as expected loss: twenty percent attrition times the rebuild plus blind weeks. The real test is whether the seven numbers publish when I'm on vacation, and after cutover they did.

S17 — Elementum: the semi-annual carrier bid — constraint rules and a lane master that cut the cycle from six weeks to about three (no $ claimed)

Amazon LPs: Invent and Simplify (primary) · Frugality (backup) · Dive Deep (the time audit). **Crosswalk:** loop packet S8 — consistent. *Never a dollar on the award — Transportation and Procurement ran it. Never "$500K/yr" or "−25% planning effort."*

Honesty label. I owned the requirements, the constraint rules and the parallel run — not the award; transportation, procurement and engineering did not report to me. Procurement joining late is my named miss.

60-SECOND PITCH

"**Situation.** Semi-annual carrier bid: hundreds of lanes, dozens of rate files, six weeks, mostly reconciliation because every carrier named lanes differently. **Task.** I owned requirements, constraint rules, and the parallel run — not the award. **Action.** Discovery: sat a full cycle logging hours — the problem class was reconciliation, not optimization. Plan & Design: a lane master; constraints as explicit rules; <5% mismatch vs the last two awards before the optimizer could run; scenario comparison, not a black box. Build & Test: parallel-ran one region's real bid. Cutover: weekly rate-file refresh; a stale banner past 10 days. Hypercare: other regions adopted without a mandate; my miss was procurement joining late. Lesson: diagnose the problem class from a time audit before building anything. **Result.** Primary metric: carrier-bid cycle time — **6 weeks → ~3 (~50%)**; pilot region 3 weeks early. $ impact: ≈$190K/yr modeled analyst time returned — the record has no dollar; never savings on the award."

S — The customer's transportation team ran a carrier bid twice a year: hundreds of lanes, dozens of carrier rate files, constraints like incumbency floors and volume commitments — six weeks per cycle, mostly reconciliation, because origin/destination naming and mode codes differed per carrier. Everyone assumed it was an optimization problem.

T — Own the requirements, the constraint rules and the parallel run; transportation, procurement and engineering did not report to me.

A — Five gated phases:

1. **Discovery (one full bid cycle)** — sat through a whole cycle and built the time audit by activity: the six weeks went to reconciling lane names and mode codes, not analysis; wrote the problem-class statement. *Exit bar:* Transportation agrees the cost is reconciliation. (Procurement not yet in the room — the miss.)
2. **Plan & Design** — the lane-master spec (origin/destination/mode normalization); constraints written one per rule (volume commitments, incumbency floors, mode rules); the validation gate (<5% mismatch vs the last two awards); scenario comparison required as first-class output; the black-box solver rejected in writing. *Exit bar:* rules signed by Transportation; gate published before any optimizer run.
3. **Build & Test** — held engineering to the spec; back-test against the two historical awards; the 10-day stale-data banner required. *Exit bar:* mismatch <5% against both awards; scenarios explainable to an incumbent.
4. **Cutover & Go-Live (pilot region)** — the parallel run — tool vs manual on one region's real bid; acceptance = the tool matches their defensible choices and beats cycle time. *Exit bar:* pilot region finished 3 weeks early; award defensible in carrier meetings.
5. **Hypercare / Close → adoption** — weekly rate-file refresh cadence; Procurement brought into the constraint definitions (late — my miss); the tool handed over as a standing global tool. *Exit bar:* other regions adopted without a mandate; cycle ~3 weeks; procurement in discovery for the next cycle.

R — Primary metric: cycle **6 weeks → ~3 (~50%)**; pilot 3 weeks early; adopted without a mandate. ≈$190K/yr modeled analyst hours returned; **no dollar on the award**.

IF PROBED — PHASE Q&A (12). Answer the phase asked, then stop.

Discovery — How did you know discovery was done?

When the time audit from a full cycle showed the six weeks were reconciliation — lane names and mode codes differing per carrier — not analysis, and transportation agreed. It was not fully done: procurement was missing, which I learned later.

Why sit through a whole cycle instead of interviewing?

Interviews tell you what people think they do. Logging hours told me where the time actually went, and the answer contradicted the room.

Plan & Design — What was in your risk register at design sign-off?

Constraint semantics differing across functions, a black-box award, and stale rates. The first one fired — "incumbency floor" meant different things to transportation and the category manager — because I hadn't put procurement in the room.

How did you handle engineering objecting to the design?

They wanted a cheapest-lane solver. I made scenario comparison a first-class output and required <5% mismatch against past awards before the optimizer could run. Explainability was the spec.

Build & Test — What was your definition of done for the model?

<5% mismatch against the last two award cycles, every scenario explainable in one sentence, and a stale-data banner past 10 days. Below that bar the optimizer did not run.

Describe your bug-bash equivalent.

The back-test read-out: every mismatch against two historical awards classified — naming, mode code, constraint semantics — in one session, with owners. That is where most of the reconciliation logic got fixed.

Cutover — What was your go criterion for the pilot?

The tool matched the regional team's defensible choices on their real bid and beat the cycle time. Their judgment was the acceptance test, not mine.

What was the rollback?

The manual process ran in parallel through the pilot. If the tool had disagreed indefensibly, they would have awarded by hand — never needed.

Hypercare — How did other regions come on?

Without a mandate — the pilot finished three weeks early and the tool carried each region's own constraints as rules. Adoption by envy; the refresh cadence kept it honest.

What kept the tool honest after the pilot?

The weekly rate-file refresh with the stale banner past 10 days, and constraint definitions now written with procurement in the room. A model with stale rates or unowned rules decays into a spreadsheet with a logo.

Cross-phase — Where did you trade scope for date? What did you refuse?

Nowhere on the bid itself. The trade was mine: a slower first cycle to write and validate constraints, for a tool that halved every cycle after. Refused: a black-box solver — and a dollar figure; transportation and procurement ran the award.

What did it save? · Do differently?

I don't have a dollar figure and won't invent one; the cycle halved and the pilot finished three weeks early — the savings claim belonged to the negotiation. Differently: procurement in discovery on day one.

Where this lands at Amazon: Invent and Simplify with the diagnosis first — a time audit that contradicted the room ("it's an optimization problem") and a simplification (one lane master + explicit rules) that beat the fancier tool. The refusal to claim the award dollar is the credibility move interviewers remember.

BUSINESS IMPACT MODEL — KPI TREE, NUMBERS, THE MATH

```
Defensible carrier award, delivered in half the cycle (no $ on the award)
+- Cycle time 6 wks -> ~3 (record); pilot region 3 weeks early (record)
+- Reconciliation hours (the time audit's finding) -> lane master + rules
|    +- back-test gate <5% vs last two awards before the optimizer runs
|    +- scenario comparison first-class -> award survives the incumbent meeting
+- Freshness loop: weekly rate-file refresh; stale banner past 10 days
+- Adoption loop: pilot 3 wks early -> regions adopt without mandate ->
     each region's constraints as rules -> tool stays theirs
```

DRIVER	BASELINE	AFTER	$/YR	BASIS
Carrier-bid cycle time	6 weeks	~3 weeks	—	record
Back-test mismatch	—	<5% vs two awards	—	record (the published gate)
Analyst hours returned	—	≈2,200 hrs/yr	≈$190K	modeled (illustrative team and region counts; range $35–420K)
Award savings	—	—	**none claimed**	the award belonged to Transportation and Procurement

If challenged. *"Surely the better tool saved freight dollars?"* — "Possibly — and that claim belongs to the people who ran the award. What I can defend is the cycle halving, the <5% back-test, and the analyst hours returned, modeled and labeled. Refusing the dollar I can't defend is what makes the ones I do defend credible."

THE MATH — FULL DERIVATION, STEP BY STEP

```
Bid model (constraint-aware; engineering built it to the rules). Decision variable x_{l,c} ∈
  [0,1] = share of lane l awarded to carrier c. min Σ_l Σ_c rate_{l,c} × volume_l × x_{l,c}
  subject to: Σ_c x_{l,c} = 1 ∀ l (every lane covered); volume commitments Σ_l volume_l ×
  x_{l,c} ≥ V_min,c for committed carriers; incumbency floors x_{l,inc(l)} ≥ f_l where the
rule
  applies; mode rules x_{l,c} = 0 if carrier c not eligible for mode(l). Scenario output:
Δcost
  = cost(all-cheapest) − cost(constrained) = the price of service/relationship risk, shown
as a
  number.
Historical mismatch gate. mismatch = lanes where the model's award ≠ the actual award in the
  last two cycles / lanes evaluated ; the optimizer runs only if mismatch < 5% . The gate
  validates the rules (constraints) against defensible past decisions, not the prices.
Time audit. Cycle = 6 weeks; measured composition (my hour log): reconciliation of carrier
files
  (naming, mode codes) dominated; analysis was the minority. Post: ~3 weeks (~50%); pilot
region
  3 weeks early. Illustrative worked example ("not program data"). Three lanes, two
    carriers.
  Lane volumes 100/200/300 loads; rates (A,B): L1 (900, 950), L2 (1,100, 1,000), L3 (800,
850).
  All-cheapest: L1→A, L2→B, L3→A  90,000 + 200,000 + 240,000 = $530,000.
Constraint: B has a volume commitment ≥ 250 loads  must also take ≥ 50 loads of L1 or L3;
  cheapest way: 50 loads of L3 to B at +$50 each = +$2,500  $532,500. Scenario line:
    "honoring
  B's commitment costs $2,500 (0.47%)." The award survives the room because the delta is
  explicit.
Stale-data banner. age = today − rate_file_date ; banner if age > 10 days ; weekly refresh.
Sanity. Σ shares = 1 per lane; constrained cost ≥ unconstrained cost always; mismatch gate
on
  rules never admits an unexplainable award.
```

SENSITIVITY — LOW / BASE / HIGH

INPUT	LOW	BASE	HIGH
Analysts per regional bid (illustrative)	2	3	4
Regions on the tool (illustrative)	1	3	4
Loaded analyst rate (benchmark)	$70/hr	$90/hr	$110/hr
Hours returned per year	~480	~2,160	~3,840
Total modeled $/yr	~$35K	~$190K	~$420K

TOTAL IMPACT STATEMENT — SAY IT LIKE THIS

"I don't have a dollar figure for the award and I won't invent one; transportation and procurement ran it. What I can model is the time: the cycle went from six weeks to about three, twice a year, and on an illustrative three analysts and three regions that is roughly 2,200 analyst hours a year, about $190K of capacity at benchmark rates. The cycle time, the pilot finishing three weeks early and the five percent gate are the record; the analyst count, the region count and the dollars are my model, and none of it is savings on the award."

If challenged — the full answers.

Three analysts and three regions are guesses.

They are, which is why the range runs from about $35K to about $420K. I'd replace them with the actual bid roster per region and the hour log I kept in discovery, which is the same audit that found the problem class.

Returned hours are not cash.

Correct, nobody was let go; the hours became analysis and onboarding the next region. I'd measure it as analyst-weeks per bid, before and after, which is the number transportation itself would recognize.

Surely the faster bid saved freight money?

Maybe, but I did not run the award and the record has no figure, so I don't claim one. The scenario delta, constrained cost minus cheapest, is shown in every award pack; the negotiation owns whatever it saves.

S18 — Sephora: RP → JDA WMS dual-DC migration — 0.5% tolerance, three mocks, and the RF-device miss (the last story)

Amazon LPs: Insist on the Highest Standards (backup — the published tolerance) · Deliver Results (backup) · Bias for Action (the rehearsed fallback fired) · Learn and Be Curious (never run a WMS migration before). **Crosswalk:** loop packet S15 — **§1.3 CONFLICT on the budget**: this record says ~$1.8M, ~3% under (~$54K), and ~$2.5M/*month* of slip exposure; the loop packet says ~$800K and ~$2.5M/*week*. This page is this packet's canon. *Never "400 devices failed" — ~100 of ~400.*

Honesty label. Full lifecycle owner, accountable to sponsor and Finance; I had never run a WMS migration and learned the floor in month one. Month markers are illustrative unless stated; the RF miss is mine — I scored the task by effort when impact was what mattered.

60-SECOND PITCH

"**Situation.** Two DCs on a legacy WMS, no documentation, an eight-month deadline, ~$1.8M budget, a $20M ROI pipeline gated behind go-live — about $2.5M a month of slip; I had never run a WMS migration. **Task.** Full lifecycle, accountable to sponsor and Finance. **Action.** Discovery: wrote the success criteria in week one; learned the floor in month one. Plan & Design: interface inventory with an owner and a test cell per contract; the 0.5% reconciliation tolerance published with Finance *before* mock 1. Build & Test: three mocks — 7% (units of measure, location names, order-status vocabularies) → 1.8% (in-transit timing) → 0.3%. Cutover: a 72-hour in-transit freeze; the legacy system read-only for 30 days; ~100 of ~400 RF devices failed the conversion — I invoked the rehearsed paper-pick fallback rather than roll back, and told leadership the training criterion slips 1.5 weeks. Hypercare: exit week 3 at under two blockers a day. Lesson: single-point dependencies get a what-if plan regardless of size. **Result.** Primary metric: inventory reconciliation variance at cutover — **7% → 1.8% → 0.3% against the 0.5% tolerance published with Finance**. $ impact: on time, ~3% under the ~$1.8M budget (~$54K), and the $20M ROI pipeline unblocked the next month — slip avoided, not banked."

S — Two DCs on a legacy WMS; zero inherited documentation; a hard eight-month deadline with a $20M ROI pipeline of four programs gated behind go-live (~$2.5M/month of slip); budget ~$1.8M; and I had never run a WMS migration. Inventory accuracy across two DCs could not wobble at cutover.

T — Own the full lifecycle — discovery charter, design sign-off, the ERP ↔ WMS ↔ 3PL ↔ carrier interface inventory, UAT, cutover, hypercare — accountable to the sponsor and Finance.

A — Five gated phases (team 14 core + 6 DC Ops; month markers illustrative unless stated):

1. **Discovery (month 1)** — sponsor charter written in week one (objectives, assumptions, success criteria); phase entrance/exit criteria; interface inventory v0 from message logs; the systems-impacted table with LOE per owner. *Exit bar:* charter signed by sponsor and Finance; every interface has a named owner; I could walk receive → pick → ship unaided.
2. **Plan & Design (months 2–3)** — design sign-off pack; interface inventory v1 with a test cell per contract; the conversion approach (masters → open orders → on-hand → in-transit as its own batch); the mapping-table standard (data, not code); **the 0.5% tolerance published with Finance before mock 1**. *Exit bar:* council sign-off; tolerance signed; stories slotted by business criticality × effort.
3. **Build & Test (months 4–7)** — build tracking; the QA demo schedule to DC Ops; the UAT entrance memo (function testing done, conversion build done, test data handed off, pre-prod + perf ready); three mock cutovers with written root causes; after the RF miss, the training plan and device readiness table. *Exit bar:* mock 3 ≤ 0.5% (0.3%) → written go-live recommendation; training sign-off (slipped 1.5 weeks, date held).
4. **Cutover & Go-Live (month 8)** — the cutover runbook; the 72-hour transfer freeze; conversion batches; post-load reconciliation; the legacy read-only switch; the go/no-go call. When ~100 of ~400 RF devices failed the

conversion script, I invoked the rehearsed paper-pick fallback rather than roll back. *Exit bar:* post-load mismatch inside tolerance; ship confirms posting; zero accuracy incidents.

5. **Hypercare / Close (4 weeks planned)** — blockers/day read daily; the 30-day failback watch; budget closed; the exit memo with the single-point-dependency lesson. *Exit bar:* <2 blockers/day sustained → exit week 3; ~3% under budget; pipeline programs started the next month.

R — Primary metric: mock variance **7% → 1.8% → 0.3% vs the 0.5% tolerance**; on time; ~3% under ~$1.8M (~$54K); the $20M pipeline unblocked the next month.

IF PROBED — PHASE Q&A (13). Answer the phase asked, then stop.

Discovery — You had never run a WMS migration; how did you know discovery was done?

When I could walk receive → put-away → pick → ship on the floor unaided and every interface I saw in the message logs had a row, an owner and a test cell. That took month one; the charter had been written week one.

Who wrote the success criteria and who signed?

I wrote them in week one — on the date, inside a published reconciliation tolerance, zero accuracy incidents, hypercare exit under two blockers a day — and the sponsor and Finance signed the charter.

Plan & Design — What was in your risk register at design sign-off?

Undocumented interface surprises, conversion mismatch above tolerance, and 3PL/carrier freeze non-compliance — each with a trigger, a data-first mitigation and an owner. RF conversion was on the log as a task, not a risk; that was my miss.

Why publish the tolerance before you knew you could hit it?

So the number belonged to Finance and me before the test, not after. Mock 1's 7% was the price; mock 3's 0.3% was worth it.

Build & Test — How were UAT entrance criteria set and who could fail a build?

Function testing done, conversion build done, test data handed off, pre-prod and perf ready — written in the charter. QA could fail a build; DC Ops confirmed in UAT because QA had demoed to them during build.

Describe your bug-bash equivalent.

The three mocks. Everyone in one room reading the mismatch by cause class — units of measure, location drift, status vocabularies, then in-transit timing. Nothing left the room without a class, an owner and a data fix.

How did you read a mock?

Never by the headline number alone. Each delta got a root-cause class; if the total shrank without a named cause, the mock was not evidence. 7 → 1.8 → 0.3 each removed a category.

Cutover — What made you say go?

A written recommendation off mock 3 at 0.3% against 0.5%, the freeze agreed in writing, failback tested, training signed off — 1.5 weeks late, inside the window.

What were your rollback triggers and did any fire?

Post-go-live mismatch above 0.5% for two days, or ship-confirm posting failing beyond two hours — rollback per interface, the legacy system read-only for 30 days as the failback. Rehearsed; never used. The paper-pick contingency, also rehearsed, did fire — for the RF miss.

Hypercare — How did you decide hypercare was over?

Blockers per day under two, sustained — exited week 3 of 4. The number was in the charter before go-live, so nobody negotiated the exit.

What did you hand over at hypercare exit?

The interface inventory with owners and test cells, the mapping tables owned by DC Ops and ERP, the failback plan still armed for the rest of the 30 days, and the device readiness table. The RF lesson went into the sponsor read-out with my name on it.

Cross-phase — Where did you trade scope for date? What did you escalate?

Nowhere on scope — I traded sequence: training slipped 1.5 weeks so the go-live date held, and I told leadership the same day with the slip priced. Escalated the RF miss as a priced decision with my name on the cause. The rule I left:

> any single-point dependency in front of an exit criterion gets a what-if plan, regardless of size.
>
> **You'd never run a WMS migration — why you? · Do differently?**
>
> I wrote the success criteria before anyone asked and learned the floor and the vendor docs in month one; the mocks were where I proved it. Differently: dual-path device conversion on day one; a device model × OS × script pass/fail table before training week.

Where this lands at Amazon: the cutover discipline in its most literal form — a tolerance published with Finance before the first rehearsal, mocks read by cause class rather than headline, a freeze, a tested failback, and a numeric hypercare exit nobody can negotiate. The RF miss is the honest beat: a rehearsed contingency fired, the date held, and the lesson became a rule.

BUSINESS IMPACT MODEL — KPI TREE, NUMBERS, THE MATH

```
$20M ROI pipeline starts on time (record: ~$2.5M/month of slip at stake)
+- Go-live date held (8 months)
|    +- cutover variance 7% -> 1.8% -> 0.3% vs 0.5% tolerance
|    |    +- mapping tables as data (UoM, location, status) · 72-h freeze ·
|    |       in-transit as its own batch · tolerance signed BEFORE mock 1
|    +- RF miss ~100 of ~400 -> paper-pick fallback, not rollback;
|         training criterion slips 1.5 wk; the date does not move
+- Budget ~$1.8M, ~3% under (~$54K) (record)
+- Inventory $ in dispute at cutover = on-hand $ x variance (modeled)
(loop) mock read by cause -> data fix -> next mock lower -> Finance trust up ->
       tolerance never renegotiated
(loop) slip priced per day (~$83K) -> escalations decided same day ->
       critical path protected -> pipeline on time
```

DRIVER	BASELINE	AFTER	$	BASIS
Cutover variance (three mocks)	7%	1.8% → 0.3% vs 0.5%	—	record
Budget	~$1.8M	~3% under (~$54K)	—	record
$20M pipeline	gated	unblocked next month	~$875K critical-path slip avoided	record math (~$2.5M/mo) — avoided, not banked
Inventory in dispute at cutover	0.5% of $50M	0.3%	≈$0.1M one-time	modeled (illustrative $50M on-hand)
Paper-pick fallback cost	—	paid to hold the date	≈$45K	modeled

```
slip price = $2.5M/month / 30 ~ $83K/day -> every escalation decided same day
dispute delta = (0.5% - 0.3%) x $50M illustrative on-hand = ~$0.1M less in dispute
fallback = ~$45K paid deliberately: ~20x cheaper than the ~$875K critical-path slip it
  protected (and ~55x cheaper than a full month of slip)
```

If challenged. *"Why not roll back when the RF devices failed?"* — "Because the fallback was rehearsed and the rollback wasn't needed: inventory was inside tolerance; the failure was device conversion, not data. I paid ~$45K of paper-picking to protect ~$2.5M a month of pipeline — and took the miss upstairs the same afternoon with my name on the cause."

THE MATH — FULL DERIVATION, STEP BY STEP

```
Reconciliation tolerance. Per SKU-location j: Δ_j = WMS_onhand_j − ERP_onhand_j . Aggregate
  mismatch M = Σ_j |Δ_j| / Σ_j ERP_onhand_j ; gate M ≤ 0.5% plus a per-SKU band ( |Δ_j| /
ERP_j
  ≤ b or |Δ_j| ≤ k units for low-count SKUs). Canonical trajectory: mock 1 = 7% → mock 2 =
1.8%
  → mock 3 = 0.3% vs 0.5%.
Worked example (illustrative units on canonical percentages). Total ERP on-hand across two
DCs =
  2,000,000 units. Mock 1: Σ|
Δ| = 140,000 → 7.0% (UoM mappings: a case-vs-each factor of 12 on 1% of SKUs alone creates
11 ×
  those units of error). Mock 2: 36,000 → 1.8% (in-transit snapshot timing). Mock 3: 6,000 →
  0.3% ≤ 0.5%  go-live recommendation. Sanity: each fix class must remove a category of Δ;
if Δ
  shrinks without a named root cause, the mock is not evidence.
In-transit freeze logic. Transfers in flight at snapshot time t_s are counted at neither end
or
  both unless frozen: freeze window F = 72 h ≥ max lane transit  in-transit set is static;
  converted as its own batch with Σ in-transit_legacy = Σ in-transit_JDA .
Slip economics. Pipeline $20M gated  ~$2.5M/month of slip  ~$83K/day; a 1.5-week training
slip
  that does not move go-live costs 0 pipeline dollars; the same slip on the critical path
would
  have cost ≈ $875K. Budget: ~$1.8M, ~3% under ≈ $54K.
Hypercare exit. blockers/day < 2 sustained  exit; exited week 3 of 4.
RF-device miss. Failure rate on newer hardware ≈ 100/400 = 25%; effort-scored "3–4
    days" vs
  impact = a go-live exit criterion (training sign-off). Rule left behind: any single-point
  dependency in front of an exit criterion gets a what-if plan regardless of estimated
effort.
```

SENSITIVITY — LOW / BASE / HIGH

INPUT	LOW	BASE	HIGH
Average unit cost (illustrative)	$15	$25	$40
Paper-pick slowdown (benchmark)	15%	25%	40%
DC labor rate (benchmark)	$25/hr	$30/hr	$35/hr
Inventory dispute avoided (0.5% → 0.3%)	~$60K	~$100K	~$160K
Paper-pick fallback cost	~$23K	~$45K	~$84K
Total modeled $/yr	~$40K net	~$55K net	~$75K net

TOTAL IMPACT STATEMENT — SAY IT LIKE THIS

"The record numbers first: on time, about $54K under a $1.8M budget, and the $20M pipeline started the next month, where every month of slip was worth about $2.5M. When a quarter of the RF devices failed, the same slip on the critical path would have cost about $875K, so I paid roughly $45K of paper-pick productivity instead and told leadership the training criterion moved, not the date. The modeled part is smaller and I'll label it: landing at 0.3 percent against a 0.5 tolerance meant about $100K less inventory in dispute at cutover on an illustrative $50M on-hand, and at mock one's 7 percent that would have been $3.5M. The date, the budget, the variance and the pipeline are measured; the on-hand value and the fallback cost are my model."

If challenged — the full answers.

Where does $50M of on-hand come from?

It's illustrative, two million units at $25, chosen so the percentages become dollars; the record gives me 0.3 percent, not the balance sheet. I'd firm it up with the actual ERP on-hand valuation at the freeze snapshot, which Finance already had for the reconciliation.

The $875K avoided slip is a counterfactual.

Yes, it's record arithmetic on a slip that did not happen, so I never bank it; it's the price that made the fallback the obvious call. What I'd measure is the pipeline's first program start date against plan, which was the next month.

Paper-pick cost is a guess.

It's a benchmark, 15 to 40 percent slower on a hundred devices for a week and a half, so somewhere between $23K and $84K. I'd measure it from the WMS: picks per labor hour on the paper lanes versus the RF lanes during those ten days.

3. The design round — twelve designs from the Elementum years, each with the math and the probes

The GFiber packet's Part 3 carried twelve system-design prompts. Here they are told as what they were underneath: **design engagements from the Elementum client work** — supply-chain design for manufacturers and network operators running SAP, 3PLs, EDI and field fleets. Use these for Amazon's functional and "how would you design / approach…" questions; they are design stories, not behavioral stories. Every design is answered in the same six moves, said out loud so the interviewer can follow the map: **(a) stocks and flows** — every bucket a unit can sit in, every flow between them, with the driver and the document that records it; **(b) first principles** — the invariants (every unit is in exactly one stock; every flow has exactly one document; every number has exactly one owner); **(c) the design** — components, the data model at its true grain, integration events, controls and kill switches; **(d) sequencing** — phases ranked Reach × Impact × Confidence ÷ Effort, with the invariants deciding what may ship at all (shown in full on E1–E2; folded into the design elsewhere); **(e) the deep-dive probes**; **(f) one metric with a dollar** — illustrative, labeled. *Every dollar in this section is illustrative — a sizing method with stated assumptions. Where a pattern wasn't run end-to-end in production, the honest line says so.*

3.0 — How I'd run programs at Amazon (the one shared section — methodology lives here, never inside a story)

STEP	WHAT IT IS	THE GATE IT CREATES	FROM THE RECORD
1. Design doc	Context · goals · non-goals · the stocks-and-flows map · data model at grain · integration events · alternatives considered · cross-functional dependencies · risks	Architecture and the business owners sign the policy before any configuration starts	The FSD signed by three orgs before config (S1, S5); "a no that lives in a document survives"
2. Goals / metrics	One objective per program; 3 key results that are counts or rates, each with an owner and a denominator	A KR without a denominator is not a KR	Seven frozen metrics, honest denominators (S11); MAPE/bias published from month 2 (S6)
3. Milestones with exit criteria	Discovery → Plan & Design → Build & Test → Cutover → Hypercare, each with entry / graduating / exit stated in writing before the phase starts	A phase graduates on counted rows, never on confidence	Every story's Action in §2; the 0.5% tolerance published before mock 1 (S18)
4. Launch readiness review	Checklist: reconciliation clean N days running · monitors live · rollback rehearsed · control owners named · comms drafted	Go/no-go is a recommendation read from the checklist, taken by the forum as a decision	Go/no-go packs (S1, S2, S18); six graduation criteria published before wave 1 (S11)
5. Staged rollout + rollback	Canary (one site / one partner / one node) → wave → full; rollback trigger written as a number; kill switch per feed	A wave that misses a criterion does not board — publicly	Wave 3 held (S11); robotics throttle to ~30% (S2); RF paper-pick fallback (S18)
6. Blameless postmortem	Timeline · contributing causes · mechanisms that outlive the incident · owner per action	Every Sev produces at least one mechanism, not a reminder	Scheduler heartbeat + DST checklist + 3σ page rule (S13); the in-transit back-test rule (S4)
7. Release cadence	Fixed cadence; a regression matrix at the true grain is the gate; freeze around peak / close; emergency change with retrofit, never a manual table edit	A rule change rides the train or does not ship	4-week cadence with the QA lead owning the gate (S15); the policy fix released on cadence (S4)

Sequencing convention used throughout (illustrative): Reach = units, orders or people touched per month; Impact on the 0.25 / 0.5 / 1 / 2 / 3 scale; Confidence 50–100%; Effort in person-months. Score = Reach × Impact × Confidence ÷ Effort. **The score ranks; the invariants decide what is allowed to ship at all.** If interrupted at minute two: clarify the book of record and the primary metric in twenty seconds, then draw the stocks.

E1 — Elementum design engagement: contractor / truck-stock inventory for a field-install fleet

Use for: "design field/mobile inventory," "stock you own but don't hold," ambiguous-ownership questions. **Record behind it:** the owner-partitioned pool and audited acts (S1, S2), lower-of-two when two books disagree (S1, S18), reason-coded reconciliation (S14).

> **Honest line.** A design engagement for a network-operator client whose contractors' trucks carried client-owned install equipment; I owned the inventory model, the event contract and the controls — the client's SAP team configured. I have not run a full van program end-to-end; the volumes here are illustrative.

The problem as I restate it. Contractors' trucks carry client-owned equipment and consumables; issues from the 3PL are posted, installs post late or not at all, counts are sporadic, and Finance sees "unaccounted" growing. Design the inventory model, the events, and the controls. Book of record: the client's SAP inventory. Primary metric: unaccounted units per contractor per month, and their value.

(A) STOCKS AND FLOWS · (B) FIRST PRINCIPLES

```
[3PL SLoc] --issue (311, scanned)--> [Van SLoc] --install (601 by serial)--> [Installed]
                 |                          |-- swap-out (311) --> [Returns-untested]
                 |                          |-- van-to-van (311, both scan)
            weekly count --> [Adjust 701/702 + reason] --> [Chargeback only from a posted
count]
Unaccounted(van) = issued - installed - returned - counted_on_hand
Classify like out-of-balance: timing -> unposted install -> serial mismatch -> true loss
```

Invariants. Conservation: every owned unit is in exactly one stock at every instant — today the van is an *unmeasured* stock; "unaccounted" is not theft yet, it is a missing document. One document per flow: issue = scanned transfer; install = work-order close that posts the goods issue; count = a physical-inventory document — if a flow has no document, it does not exist and the design adds one before adding a report. One owner per number: Inventory Control owns the book; Field Ops owns counts; Finance owns the chargeback method; the contractor owns the physical; I own the definitions and gates.

(c) Design. A van is a storage location under the market plant (not a spreadsheet); serialized issues with forced scan on every flow; min/max replenishment per van by tech type, released only when last week's count is posted (**count-as-gate**); a reconciliation service computing issued − installed − returned − counted per van, reason-coded; governed screens for count, adjustment (role + reason) and chargeback review; the CRM reads only the assigned tech's van availability. Data grain: SKU × plant × SLoc(van) × serial × state, with an append-only serial-history ledger; every event idempotent on document + line, replays never double-post, reversals reference the original. Controls: no new issue to a van with an unposted weekly count (hard gate); days-of-cover cap with audited override; a serial issued to van A cannot be installed from van B without a posted transfer — the install is held, not lost; two weeks over tolerance ⇒ count-every-issue mode; kill switch = freeze issues to a contractor, 3PL fulfills directly.

(d) Sequencing (illustrative). P1 van-as-SLoc + serialized issue + count-as-gate (score 300) → P2 field-app install/swap events posting by serial (175) → P3 reason-coded reconciliation + Finance chargeback method (267 — ranks above P2 but needs its events; the invariant sequences it after) → P4 min/max + cap → P5 CRM same-day availability read. Rejected: a spreadsheet outside SAP — a second book.

(f) Metric. Unaccounted units per contractor per month → near zero, from posted documents only, published weekly per van with the reason mix. Illustrative dollar: 500 vans × 2 unaccounted/month × $150 average unit ≈ $150K/month ≈ $1.8M/yr of stock with no owner; the design targets the unexplained share — Finance sets the chargeback threshold, not me. Supporting: install events posted same-day ≥ 99%; vans with a posted weekly count = 100%; "van said yes, book said no" reschedules → 0.

Deep-dive probes.

Why a storage location and not consignment or subcontracting special stock?

The client owns the unit until install, so it is their stock at the contractor's location — a storage location under the market plant. Subcontracting stock is a manufacturing pattern (components consumed into a finished good); customer consignment implies the customer holds it. Neither describes a truck.

Movement types?

311 for 3PL → van (scanned both ends), 601 on the install (or 201 to a cost object if not order-based — decide once), 311 van → RMA for swap-outs, 701/702 for count differences with a reason code. 561 only for the initial load — never for "fixing" a count.

The field app is offline half the day — how does the book stay right?

Capture with a client-generated idempotency key and timestamp; replay in order per serial; reconciliation counts "captured, not posted" as its own reason code so timing is never read as loss.

Failure modes?

Install posted from the wrong van — hold, never auto-transfer. Duplicate install replay — idempotency key. Count with a phantom serial — quarantine the line. Chargeback from an unposted count — impossible by design; chargebacks read only physical-inventory documents.

Trade-offs?

Count-as-gate slows issues in week one and finds every uncounted van — accepted. Per-serial postings raise volume — sized. The days-of-cover cap frustrates long routes — override with a reason code, audited.

What if a contractor has no scanner?

Stage 1: SLoc + signed paper count uploaded; stage 2: scanning. Never skip the SLoc for the scanner.

SOLUTION FLOW — THE SYSTEM MAP (PROPOSED)

```
[Salesforce [BoR customer/order]]        [BI / BigQuery: van recon, aging]
  WorkOrder close (install/swap)                ^  MATDOC + install/count
  | event, real-time                           |  events, hourly
  v                                           |
[Field app: tech / van / install_event / return_event / count]
  offline capture, client idempotency key, replay in order per serial
  | install event -> 601 (or 201 cost ctr) by serial, real-time
  | swap event    -> 311 van -> RMA SLoc by serial, real-time
  | van count     -> PI document -> 701/702 + reason, weekly
  v
[SAP CPI / Integration Suite]  --retry / DLQ per hop, idempotent-->
  v
[SAP S/4HANA [BoR stock + finance]]
  plant = market · SLoc per contractor x truck · serials (MARC-SERNP)
  311 3PL SLoc -> van SLoc · 601/201 install · 651 customer return
  701/702 count adjust -> chargeback report (Finance)
  ^
  | 940 pick / 945 ship-confirm for the van issue, event
[3PL WMS [BoR bin]]  -- replenishment to van min/max, weekly wave
```

E2 — Elementum design engagement: serial-number tracking, vendor → warehouse → technician → customer → return

Use for: traceability, serialized inventory, state-machine design questions. **Record behind it:** state machines for inventory buckets with one owning system per transition (S1, S2); two books reconciled to a published tolerance (S18).

> **Honest line.** I designed and governed the state model and reconciliation; the client's SAP team configured the serial profile — I own the state list and which transitions are legal, not the IMG screens. Volumes illustrative.

The problem as I restate it. Every serialized unit the client ships — network hardware to customer premises — must be traceable by serial across its whole life, in SAP and in every system that touches it (3PL WMS, field app, CRM asset, RMA/refurb). Book of record for the serial state: SAP, with the 3PL and field app as event sources. Primary metric: serial-state accuracy — serials whose SAP state matches a physical or system observation.

(A) STOCKS AND FLOWS · (B) FIRST PRINCIPLES

```
[Vendor FG] --856 ASN w/ serial list--> [In-transit] --receipt by serial (101)-->
[3PL unrestricted] --issue (311)--> [Van] --install (601 + CRM asset)--> [Installed base]
[Installed] --return/disconnect--> [Returns-untested] --test--> refurb / RTV / scrap (E10)
```

Invariants. A serial never appears from nowhere — its first legal appearance is the ASN; a receipt for a serial the ASN did not carry is an exception, not a receipt; an install for a serial not in that van is a hold. Every flow has a reversal that references the original, so a replay or correction never manufactures a phantom serial. Minimal design forced: serial profile with stock check; the ASN carries serials; every downstream event carries the serial; one append-only serial-history ledger; a daily three-way reconciliation (SAP vs 3PL vs field app/CRM asset).

(c) Design. Serial profile with stock check on the serialized materials, so a goods issue for a serial not in that location fails loudly instead of posting silently; EDI/ASN ingest validating serial lists against the PO; receipts by serial; install/swap events by serial creating the CRM asset; an RMA module with test-result events and grade — refurb keeps the *same serial, same material*, with a grade attribute: never re-number, so the asset history stays continuous. Grain: serial × SKU × state × location × owning-system × as-of, plus the serial-history ledger (one row per transition: document, source, actor, timestamp, idempotency key). Controls: ASN serial count = PO quantity or the ASN quarantines; a receipt for an unknown serial goes to an exception queue with a 24-hour vendor SLA — never auto-create; installed serials with no CRM asset after 24 h alert; corrections only through a governed screen with role + reason — *never* a manual table edit, and never "fix SAP to match the 3PL" — lower-of-two for promising while the cause is found.

(d) Sequencing (illustrative). P1 serial profile + ASN serials + receipt by serial (~50K serials/mo, score 24,000) → P2 install/swap by serial + CRM asset (10,500) → P3 three-way daily reconciliation + governed correction screen (28,333 — after P1/P2; needs the events) → P4 RMA/refurb states → P5 scrap/RTV with certificates.

(f) Metric. Serial-state accuracy ≥ 99.5%, measured by the daily three-way reconciliation with a reason-coded exception list. Illustrative dollar: each mis-stated serial costs a truck roll or a lost unit; at 200K installed serials, 97% → 99.5% removes ~5,000 mis-states/yr × ~$120 blended ≈ $600K/yr — before warranty-recovery value (in-warranty failures returned to vendor by serial). Supporting: ASN-to-receipt serial match ≥ 99.5%; installs with an asset record within 24 h = 100%.

Deep-dive probes.

How do serials travel on EDI?

The 856 ASN carries serials in its hierarchical loops at item/pack level; the 945 ship advice carries serials for the van issue; adjustments must carry serials or they quarantine.

Idempotency?

Key = document + line + serial; the ledger rejects a duplicate transition and logs it; reversals reference the original document. A replayed install never creates a second asset.

The 3PL receives a serial the ASN didn't list?

Exception queue, 24-hour SLA to the vendor; the unit sits blocked and unpromisable until resolved. Never auto-create the serial.

Refurb identity — same serial, new material?

Same serial, same material, a grade attribute or refurb location — never re-number; the asset history stays continuous.

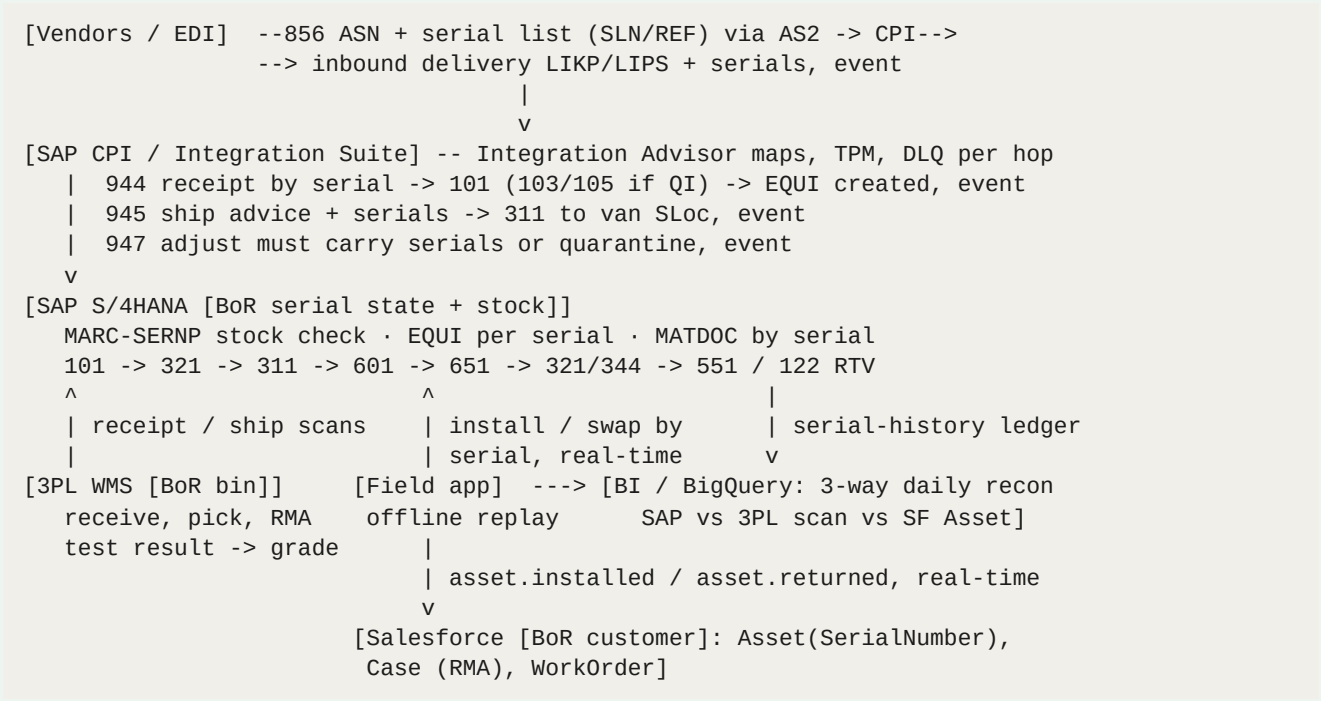
Failure modes?

Duplicate serials from a vendor — block at ASN validation. Typo serial from the field — scan-only with check digits. State fixed by a table edit — forbidden; correction screen with reason and role. Reconciliation that "fixes" SAP to match the 3PL — forbidden; lower-of-two while the cause is found.

Trade-offs?

Serialization slows receiving (a scan per unit) — offset by ASN pre-load. Three return states add screens — but an untested return that is promisable is a truck roll.

SOLUTION FLOW — THE SYSTEM MAP (PROPOSED)



E3 — Elementum design engagement: vendor EDI + VMI for long-lead network equipment

Use for: vendor integration, consignment/VMI, inbound-reliability design questions. **Record behind it:** 597 lanes of X12 EDI behind a ≥95% valid-ASN cohort gate (S9); lead times measured as distributions (S8).

> **Honest line.** The EDI onboarding is run experience (S9); the VMI/consignment mechanics are the SAP pattern, designed in the engagement but not run by me as a program — labeled.

The problem as I restate it. Long-lead equipment from a few strategic vendors; the client wants EDI for POs/ASNs/invoices and a VMI arrangement where the vendor replenishes to agreed min/max at the central 3PL. Design the transaction set, the stock-ownership model, and the controls. Book of record: SAP MM (PO, goods receipt, consignment stock). Primary metric: confirmed-date reliability — the vendor's receipts measured against its own confirmations — and valid-ASN rate.

(A) STOCKS AND FLOWS · (B) FIRST PRINCIPLES

```
[Vendor inventory] --850 PO out / 852 product activity (VMI)--> [Vendor committed (855 ack)]
  --ship (856 ASN; 214 status)--> [In-transit] --receive (101 K if consignment / 101 if
own)-->
[Consignment K at 3PL] --consume/transfer (411 K)--> [Own unrestricted] -> settlement (MRKO)
Reorder point ~ demand over p90(LT) + z x sigma_L  -- from measured distributions (S8),
  refreshed monthly with a drift flag; brochure lead times are inputs to nothing
```

Invariants. A confirmation without a date is not a confirmation. A receipt without an ASN is an exception, not a receipt. Ownership is explicit at every instant: vendor-owned consignment on our floor is special stock K, visible to planning, settled at consumption — or the vendor replenishes our own stock under blanket POs; who carries the working capital is a Finance decision, modeled both ways, not a default.

(c) Design. Transaction set: 850 PO out · 855 acknowledgment (confirmation, with date and quantity) · 856 ASN · 810 invoice · 997 functional acknowledgment on every interchange · 852 product activity for VMI · 861 receiving advice for discrepancies. Confirmation discipline: the 855's date becomes the planning date; "acknowledged" with no date = unconfirmed, a planner exception. ASN gate per vendor: ≥95% valid ASN before activation (the S9 gate, reused); vendors with weak data onboard on portal templates and graduate. Min/max set from measured lead-time distributions per vendor × lane, refreshed monthly, drift-flagged. Controls: gateway de-duplication on interchange control numbers; document idempotency on PO + line + confirmation sequence; out-of-order 855 after 856 — the ASN wins quantity, the ack updates only the date; a mis-parsing pack hierarchy quarantines, never partially posts.

(f) Metric. Confirmed-date reliability ≥ 95% — receipts on or before the vendor's own confirmed date, measured from month one (baseline unknown, so the metric starts honest). Illustrative dollar: if late equipment idles a build crew at ~$50K/week and confirmation discipline avoids ~20 such weeks/yr ≈ $1M/yr; plus working capital if consignment carries ~$2M at the vendor's expense (≈$160K/yr at 8%). Supporting: valid-ASN ≥95% per vendor before activation; receipts without ASN → 0.

Deep-dive probes.

Consignment vs own stock for VMI?

Special stock K: vendor-owned on our floor, visible to planning, settled at consumption. Alternative: vendor replenishes our own stock under blanket POs — simpler accounting, but we carry the inventory. Choose by who should carry the working capital; I model both and let Finance decide.

Idempotency and ordering?

Control numbers de-duplicate at the gateway; document-level idempotency on PO + line + sequence; an ASN received twice creates one inbound delivery.

Failure modes?

An 855 with "acknowledged" but no date — treat as unconfirmed. An ASN with an unparseable pack hierarchy — quarantine, not partial post. Unit-of-measure mismatch — a conversion table at the map (the classic mock-1 defect from S18). A looping 997 on a failed interchange — dead-letter after three.

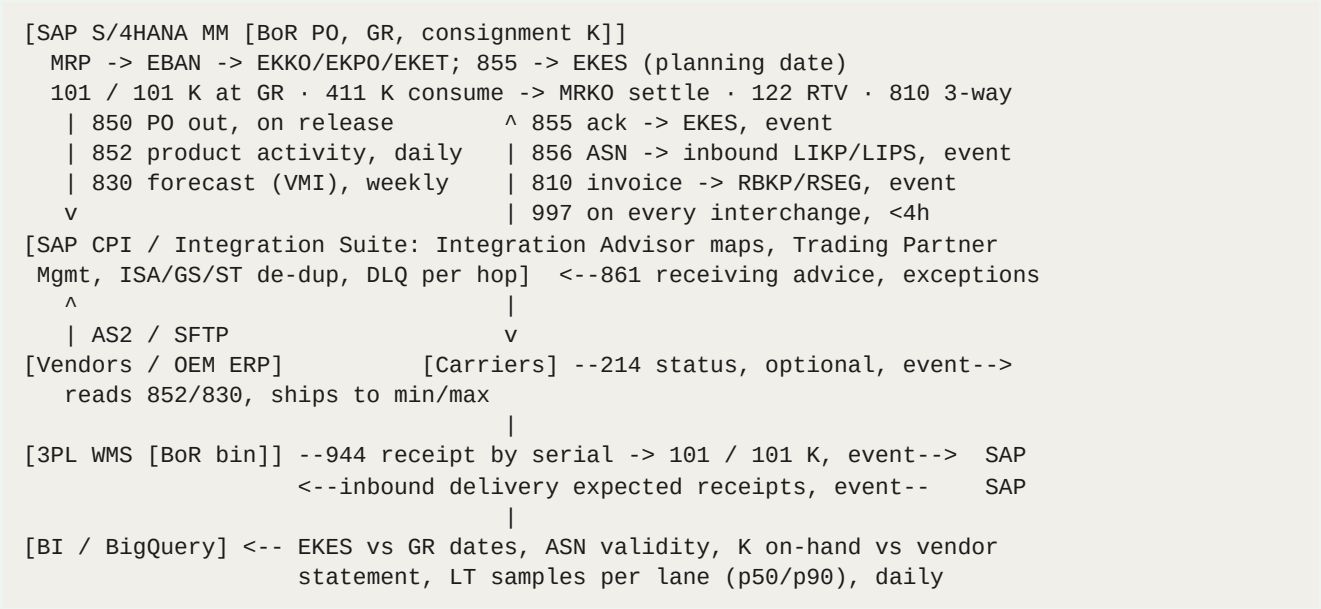
How do you set min/max?

From measured lead-time distributions per vendor × lane: reorder point ≈ demand over the p90 lead time plus safety stock from σ_L, refreshed monthly with a drift flag. Brochure lead times are inputs to nothing.

Trade-offs?

Confirmation control adds planner exceptions in month one — that is the point; it reveals the unreliable vendors. The ASN gate delays go-live for weak-data vendors — they onboard on the portal floor and graduate, which keeps the long tail inside the standard.

SOLUTION FLOW — THE SYSTEM MAP (PROPOSED)



E4 — Elementum design engagement: CRM order → SAP fulfillment integration with status sync

Use for: order-orchestration, system-integration, "orders are dropping" design questions. **Record behind it:** the OMS held to "read published availability, never compute" and the event chain across WMS/OMS/POS/S4 (S1, S3); scheduler and status-mapping failures lived through (S13, S14, and a ~$2.8M status-mapping root-cause at Flexport).

> **Honest line.** I have not implemented the client's CRM; I owned the integration contract — the event chain, idempotency, status truth and reconciliation — and the client's teams built both ends.

The problem as I restate it. A customer signs up in the client's CRM; the order must create the SAP fulfillment (a kit delivery from the 3PL or van), reserve stock, and keep status flowing back so the CRM, the technician and the customer see the same state — without orders dropping at month-end or double-shipping on retries. Book of record: CRM for the customer order; SAP for stock and fulfillment; **status truth per hop is the system that performed the hop**. Primary metric: orders reconciled across every hop — sent = created = delivered = status returned — 100% daily.

> **(A) STOCKS AND FLOWS · (B) FIRST PRINCIPLES**
>
> ```
> [CRM order created] --order.created (correlation id)--> [SAP order created]
>   --ATP against PUBLISHED availability (never computed in the CRM)--> [Reserved]
>   --install date - lead--> [Delivery created] --pick/ship/PGI--> [Shipped]
>   --install event--> [Complete] ... status flows BACK per hop, one writer per field
> ```
>
> **Invariants.** One creation path: deliveries are created only from the SAP order, never from the inbound event — a duplicate event cannot make a second delivery. One availability truth: the CRM reads published availability; it never computes its own. One writer per status field; a status the map does not know defaults to "unknown — investigate," never to "shipped." Every message carries a correlation id that lives on the SAP order, so a retry finds its own order.
>
> **(c) Design.** Durable messaging with retry and dead-letter queues per hop; an outbox table on the CRM side so order-create is at-least-once with no lost orders; idempotency at create (correlation id unique on the order — a second create returns the first order); version checks so an update arriving before its predecessor holds rather than overwrites; availability published to a cache with a staleness flag ("hold last-good, flag it") so CRM promising survives an SAP outage; daily hop-count reconciliation with an unreconciled queue cleared by 9 AM; a heartbeat on every scheduler in the chain (the S13 lesson — detect in one interval, not by a VP at 4 AM).
>
> **(f) Metric.** Orders reconciled across every hop daily → 100%, unreconciled count → 0 by 9 AM. Illustrative dollar: if 0.5% of 50K orders/month stall and each stalled install costs ~$75 in rescheduling plus a day of deferred revenue, ≈$19K/month ≈ $225K/yr direct — before churn on a bad first impression, which is the number that matters and which Finance, not I, should size. Supporting: dead-letter age p95 < 1 h; duplicate deliveries = 0; status latency p95 < 5 min.

> **Deep-dive probes.**
>
> **Durable documents or APIs?**
>
> Durable, monitorable documents (IDoc-class) for order create/change — retry with visible status; APIs for status back where latency matters. Either way the correlation id lives in a reference field on the order so a retry finds its own order.
>
> **How do you stop a retry from double-shipping?**
>
> Idempotency at create — the correlation id is unique on the order, so the second create returns the first order — and deliveries are created only from the SAP order, never from the inbound event.
>
> **Orders drop at month-end — where do you look?**
>
> Count reconciliation across hops for the last three month-ends; the message monitor and dead-letter queue by error class; job overlaps and lock waits at the drop times — close jobs locking customer or material masters, availability runs in the same window; the release calendar. Fix: retry with backoff plus idempotency immediately; re-sequence or throttle the contention as the cure; counters on a dashboard so it is visible before close.
>
> **Status mapping — what breaks?**

A new partner status not in the map (default to "unknown — investigate," never "shipped"); a mapping changed without regression (release train only); two hops writing the same CRM field (one writer per field). A ~$2.8M root-cause I ran at Flexport was exactly a status-mapping defect.

Failure modes?

Version 3 arrives before version 2 — version check holds the older. SAP created but the confirmation back was lost — reconciliation finds "created, unconfirmed" and re-emits. A ship-confirm for a cancelled delivery — exception; physical stock to be found. A scheduler dies silently — heartbeat page.

Trade-offs?

The CRM reading SAP availability adds an uptime dependency — mitigated by the published cache with a staleness flag. The outbox adds a table and a relay — the price of at-least-once with no lost orders.

SOLUTION FLOW — THE SYSTEM MAP (PROPOSED)

```
[Salesforce [BoR customer + order]: Account, Order/OrderItem, WorkOrder,
 Asset, status field (one writer per field)]
   | order.created/changed/cancelled via outbox, at-least-once, real-time
   | correlation_id = SF order + line + version
   v                                    ^ order.confirmed (SAP VBELN)
[SAP CPI / event bus]                   | status.changed(hop, ts, corr id)
   ORDERS/ORDCHG IDoc or API ·          | p95 < 5 min, per hop
   DLQ per hop · per-hop counters ·     | availability read (never
   status_map table (versioned) -------+ computed), cached + stale flag
   v
[SAP S/4HANA [BoR stock + fulfillment]]
   VBAK/VBAP (corr id in reference field) -> ATP -> VBBE requirement
   -> LIKP/LIPS delivery (install date - lead time) -> PGI 601 (MATDOC)
   | 940 pick, daily wave        ^ 945 ship confirm -> 601, event
   v                             |
[3PL WMS [BoR bin]] --or-- [Field app: van pick / job.completed]
                              | job.completed -> SAP install posting
                              | + SF Asset(SerialNumber), real-time
[BI / BigQuery] <-- hop_counter, orders-by-state aging, DLQ age, daily
   received = processed + failed + in_flight per hop, balanced by 9 AM
```

E5 — Elementum design engagement: 3PL inventory reconciliation — two books, one truth, Finance protected

Use for: reconciliation design, "two systems disagree," inventory-integrity questions. **Record behind it:** the lower-of-two basis and out-of-balance sign rule (S1), the 0.5% tolerance published with Finance and read by cause (S18), reason-coded reconciliation as a standing control (S14).

> **Honest line.** This is the seam I have owned repeatedly; the design is the generalized version of run experience. Volumes illustrative.

The problem as I restate it. The 3PL's WMS and SAP disagree on on-hand by SKU (and by serial); the delta drifts, promising reads SAP, Finance closes on SAP, and adjustments are posted ad hoc. Design the reconciliation so the two books converge, the customer is protected while they disagree, and Finance can trust the close. Books: SAP at storage-location grain; the 3PL WMS at bin/serial grain. Primary metric: reconciled delta — rows outside tolerance after reason-coding, trending to inside tolerance.

(A) STOCKS AND FLOWS · (B) FIRST PRINCIPLES

```
[In-transit (ASN)] --receive (944 -> 101)--> [3PL bin] = [SAP unrestricted]
[SAP unrestricted] --pick/ship (940 -> 945 -> PGI 601)--> [Shipped]
[WMS bin moves] -- internal, MUST NOT change SAP qty
[Damage/hold] --947 status adjust--> [SAP blocked] · [Count diff] --947 w/ id--> [701/702]
Daily: snapshot(WMS by bin, rolled to SLoc) vs MB52(SAP) per SKU x SLoc x state
Delta classified: timing (in-flight ledger) | mapping | status | physical -> only
  "physical" ever becomes an adjustment
```

Invariants. Neither side wins automatically: SAP is the book of record for Finance and promising; the 3PL is the physical truth; promising reads **the lower of the two** until the reason is known. Grain alignment is a contract: bins roll up to the storage location; WMS statuses map to unrestricted/QI/blocked through one owned table; units of measure and pack factors per SKU; serials compared as sets — any row that cannot be aligned is a mapping defect, not a variance. Adjustments only from documents: a 947 with an adjustment id, posting a referenced 701/702 — never a retroactive snapshot that rewrites history.

(c) Design. A daily snapshot comparison against the in-flight ledger (so timing never reads as shrink), reason-coded into timing / mapping / status / physical with an owner per class; a tolerance per value band published with Finance *before* the first comparison; serialized equipment at zero tolerance — every unexplained serial is a unit whose state is unknown, which is exactly the work; idempotent snapshots (date + warehouse) and adjustment ids so replays never post twice; the delta report read before every close, trending from "pipe" causes to "physical" causes as the pipes get fixed.

(f) Metric. Reconciled delta: SKU × location rows outside tolerance after reason-coding → 0 on consecutive days; serialized equipment: 0 unexplained serials. Illustrative dollar: with ~$20M of equipment at 3PLs and a 2% unexplained delta, ~$400K of inventory has no reliable owner; converging to 0.5% frees ~$300K of buffer Finance no longer reserves against — before the truck rolls avoided by not promising phantoms. Supporting: unposted ship-confirms >48 h → 0; adjustments with reason code = 100%; month-end close with zero silent true-ups.

Deep-dive probes.

Which side wins?

Neither, automatically. SAP is the book for Finance and promising; the 3PL is the physical truth. Promising reads the lower until the reason is known; Finance sees the reason-coded delta; only the "physical" class becomes an adjustment.

How do you align grains?

Bins roll to the storage location; statuses map through one owned table; unit-of-measure and pack factors per SKU — the classic mock-1 defect class; serials as sets. Unalignable rows are mapping defects, not variances.

Idempotency?

Snapshots idempotent on date + warehouse; adjustments carry the 3PL's id — a replay never posts twice; the book adjustment references the 3PL document.

Failure modes?

An inventory snapshot arrives before overnight ship-confirms post — timing; compare against the in-flight ledger, not zero. A status the map does not know — "unknown, hold," never unrestricted. The 3PL "fixes" a count by re-sending corrected history — reject retroactive snapshots; adjustments go through documents.

Trade-offs?

Lower-of-two under-promises while the delta stands — a planning variance accepted over a customer defect. Zero tolerance on serials creates more exceptions — each is a unit whose state is unknown, which is exactly the work.

SOLUTION FLOW — THE SYSTEM MAP (PROPOSED)

```
[Vendors / EDI] --856 ASN -> CPI -> inbound delivery LIKP/LIPS, event-->
                                                                  |
[3PL WMS [BoR bin / serial scan]]                                 v
   bins, WMS status, cycle counts, holds      [SAP S/4HANA [BoR SLoc stock
   | 944 receipt -> 101, event                  + finance]]  (#2: IM grain)
   | 945 ship confirm -> PGI 601 + serials       MARD / MCHB / MSKA by
   | 947 adjust (qty or status) -> quarantined    MATNR x WERKS x LGORT
   |     until reason-coded -> 701/702 or 344     x stock type; EQUI serials
   | 846 stock snapshot at SKU x whse x status    MATDOC movement trail
   |     x serial, daily by 06:00 local           ^ 940 pick, daily wave
   v                                              |
[SAP CPI / Integration Suite]  ---- IDoc status WE02 (51 / 64) --------+
   idempotent on date + warehouse (846), 947 adjustment id, 945 id
   |                                              |
   v                                              v
[BI / BigQuery: recon_fact] <-- MARD/MCHB + MATDOC + EQUI, daily 06:30 --
   align bin->SLoc, status->stock type, UoM/pack; Δ by reason;
   in-flight ledger (940 w/o 945, ASN w/o 944, 945 not posted)
   | min(SAP, WMS) per SKU x node while Δ != 0, hourly
   v
[Salesforce / promising] reads the lower basis  ·  [Finance] reads Δ list
```

E6 — Elementum design engagement: PLM ECN → SAP material/BOM handoff, with obsolete-part disposition

Use for: engineering-change, BOM-integrity, obsolescence questions. **Record behind it:** the contract manufacturer's BOM made safe to plan on — phantoms, revision overlaps, one supply equation, per-site reconcile gate (S7) — the same problem from the SAP side.

> **Honest line.** I have not implemented a PLM system; I have run the part of the PLM→ERP handoff that hurts — effectivity, phantoms and where-used are what break planning. Volumes illustrative.

The problem as I restate it. Engineering releases change notices in the client's PLM (new revision, substitute component, end-of-life). SAP must get the material-master and BOM changes with correct effectivity; planning must not buy the obsolete part; stock of the obsolete part — say 5,000 units — needs a disposition; and the field must know which revision to install. Books: PLM for engineering design; SAP for the planning/costing BOM and inventory. Primary metric: ECN-driven exceptions — POs placed for obsolete parts, and revision overlaps in the planning BOM → 0.

(A) STOCKS AND FLOWS · (B) FIRST PRINCIPLES

```
[PLM in-work] --ECN released--> [Released revision] --transfer (change master,
  valid-from)--> [SAP material + BOM, new rev] --effectivity switch--> [MRP re-plans]
[Old-part stock, 5,000 units] --disposition--> run-out | rework | return-to-vendor | scrap
Purchasing BLOCKED at ECN approval, not at effectivity -- the gap between the two is
  where obsolete buys happen
```

Invariants. Two revisions are never valid on the same date — reject at transfer. Effectivity is never set in the past — planning would re-plan history. The BOM never arrives before its material — order the transfer or stage the BOM. A phantom assembly explodes through to components, so the reconcile gate must explode through phantoms or it will pass a wrong structure (the S7 lesson). Disposition is a signed sequence — Engineering → Planning → Finance — and purchasing is blocked at approval while issuing continues for run-out.

(c) Design. The ECN transfer carries the change master with valid-from; date effectivity for planning (MRP needs one switch), serial or lot effectivity for the field only where revisions are not interchangeable; a where-used check drives the disposition decision tree: still consumable in a valid assembly → run-out (push valid-from to the run-out date, if not safety-related); small modification → costed rework order; vendor accepts → return against the PO; otherwise scrap with Finance's write-down — approved *before* effectivity, with the workflow posting the status change itself so "approved but not blocked" cannot exist. Field protection: asset-record effectivity plus a bulletin so an old revision cannot be installed after the switch.

(f) Metric. ECN exceptions — POs placed for obsolete parts plus revision overlaps in the planning BOM → 0 per month, measured from open-PO data against material status and from the reconcile-gate log. Illustrative dollar: one avoided obsolete buy of 5,000 units × $40 = $200K; at ~10 obsoletes/yr with even 20% previously mis-bought ≈ $400K/yr, plus the write-down Finance no longer takes by surprise. Supporting: ECN → SAP lead time ≤ 1 business day; dispositions approved before effectivity = 100%; field installs of a superseded revision → 0.

Deep-dive probes.

Date vs serial effectivity?

Date for planning — MRP needs one switch; serial or lot for the field when old and new revisions must be distinguished at install. Both recorded on the ECN; the planning BOM uses the date.

5,000 units on hand — how do you decide?

Where-used first: is the part still consumable in any valid assembly? Run-out if yes and the change is not safety-related; rework if it's a small costed modification; return if the vendor accepts; scrap with the write-down otherwise. The signature order is Engineering → Planning → Finance, and purchasing is blocked at approval, not at effectivity.

How do phantoms behave?

A phantom explodes through to its components in planning; an ECN on a phantom changes the parent's effective components — the reconcile gate must explode through phantoms or it passes a wrong structure.

Failure modes?

Two revisions valid the same date — reject at transfer. Effectivity in the past — reject; planning would re-plan history. BOM before material — stage it. Disposition approved but status unchanged — the workflow posts the status itself. Field installs the old revision after effectivity — asset effectivity plus the bulletin.

Trade-offs?

Blocking purchasing at approval can starve a run-out if Planning misjudged — mitigate by allowing issue while blocking purchase, with a dated review. Serial effectivity costs field data discipline — use it only where revisions are not interchangeable.

SOLUTION FLOW — THE SYSTEM MAP (PROPOSED)

```
[Siemens Teamcenter [BoR engineering design]]
   Item / ItemRevision / BOMView / ChangeNotice (ECN), release status
   | ECN released -> T4S (or CPI/API, #4): MATMAS then BOMMAT, ordered,
   | with change master AENR + valid-from; per ECN, <= 1 business day
   v
[T4S / SAP CPI]  -- staging queue: hold BOM until material exists;
                    reject two revisions valid on one date; idem = ECN id
   v
[SAP S/4HANA [BoR planning BOM, stock, cost]]
   MARA/MARC (MMSTA/MSTAE status) · MAST/STKO/STPO under AENR
   CS15 where-used · MD04/MD06 MRP · EKPO open POs · MARD on-hand
   | where_used.snapshot per ECN (BOMs, POs, on-hand, installed), event
   | disposition approved (Eng -> Planning -> Finance) -> status change
   | PO cancel / convert -> 850 change -> 855 from vendor, per ECN
   | 122 RTV · 551 scrap · rework order · run-out (valid-from pushed)
   v                                    v
[Vendors / EDI] <- 850/855 ->     [Field app + Salesforce Asset]
                                    serial effectivity -> which rev to
                                    install; service bulletin, per ECN
[BI / BigQuery] <-- ECN log, reconcile-gate results, EKPO vs status,
                    revision overlaps, daily
```

E7 — Elementum design engagement: equipment supply & demand forecasting for a new-market build

Use for: forecast-to-buy, cold-start demand, launch-planning design questions. **Record behind it:** forecast-accuracy measurement and input integrity (MAPE/bias, 70→85%, the both-errors release rule, S6); lead-time distributions (S8); need floors so one channel cannot starve another (S1).

> **Honest line.** I have not owned a market-launch forecast end to end; the forecast number belongs to Planning — I own the equation, the measurement and the triggers. Newsvendor and safety-stock formulas are the MIT MicroMasters toolkit, labeled. Volumes illustrative.

The problem as I restate it. The client launches market N: build sites ramp by month from the build plan; demand follows adoption curves with no local history; kits and long-lead equipment have vendor lead times of weeks to months. Design the forecast-to-buy chain so installs are never delayed by stockouts and stranded stock stays inside a band. Books: the project system for the build plan; Planning for the adoption curve; SAP MRP for supply. Primary metric: install delays caused by stockouts → 0, with days-of-cover inside band.

(A) STOCKS AND FLOWS · (B) THE MATH

```
[Build plan] --milestones--> [Sites passed] --adoption curve x marketing-->
[Install demand] --kit BOM explosion--> [Component need by SKU x week, as PIRs]
--MRP (measured lead times, confirmed dates)--> [Planned orders -> PRs -> POs]
--856/101--> [Central 3PL pool] --STOs as sell-through proves the curve--> [Markets]
Newsvendor first buy: CR = Cu / (Cu + Co)
  missed install ~ $300 (re-roll + deferred revenue); holding ~ $3/mo
  CR ~ 0.99 -> order the 99th percentile of month 1-3 demand for the core unit;
  lower-Cu extras sit at the 70th (illustrative inputs; the method is the toolkit)
Re-forecast trigger: actual take-rate vs curve +/-20% for two consecutive weeks ->
  new version, re-explode kits, MRP on confirmed dates; bias sign published weekly
```

Invariants. Demand enters planning exactly once, through the engine — manually loaded requirements beside it are blocked. Lead times are measured distributions, never brochures (the S8 rule): the planned delivery time is the measured p50 with a drift flag, and the *confirmed* date drives the receipt, not the requested date. The first buy is held central: under maximum uncertainty one pool covers several markets' variance (risk pooling); stock forward-deploys only as sell-through proves the curve. Launch reserves carry need floors with hold-until dates — one market cannot steal another's launch; a floor breach is a governed screen with a reason code, not an email.

(f) Metric. Install delays caused by stockouts → 0 in the first 90 days of a launch, measured from reschedule reasons joined to the supply view; days-of-cover inside band per SKU × market; MAPE and bias published from month 2. Illustrative dollar: 2,000 installs/month × 3% stockout-caused reschedules × ~$300 ≈ $18K/month ≈ $216K/yr per market, plus stranded-stock avoidance — holding 20% of a $500K first buy central rather than in-market keeps ≈$100K from stranding. The client's take-rate and unit costs belong to its Planning and Finance — never invented.

Deep-dive probes.

Why central-first?

The first buy happens under maximum uncertainty; holding it central collapses the distribution — one pool covers several markets' variance — and stock transfers forward-deploy as the curve proves out.

Show the newsvendor logic.

Critical ratio = underage cost over underage plus overage. A missed install at ~$300 against ~$3/month of holding gives ~0.99 — order the 99th percentile of early-month demand for the core unit; accessories with a lower miss cost sit at the 70th. Illustrative inputs; the method is the toolkit, labeled.

Re-forecast trigger?

Actual vs curve ±20% for two consecutive weeks → new forecast version, kits re-exploded, planning on confirmed dates. Bias sign publishes weekly so Planning sees direction before magnitude.

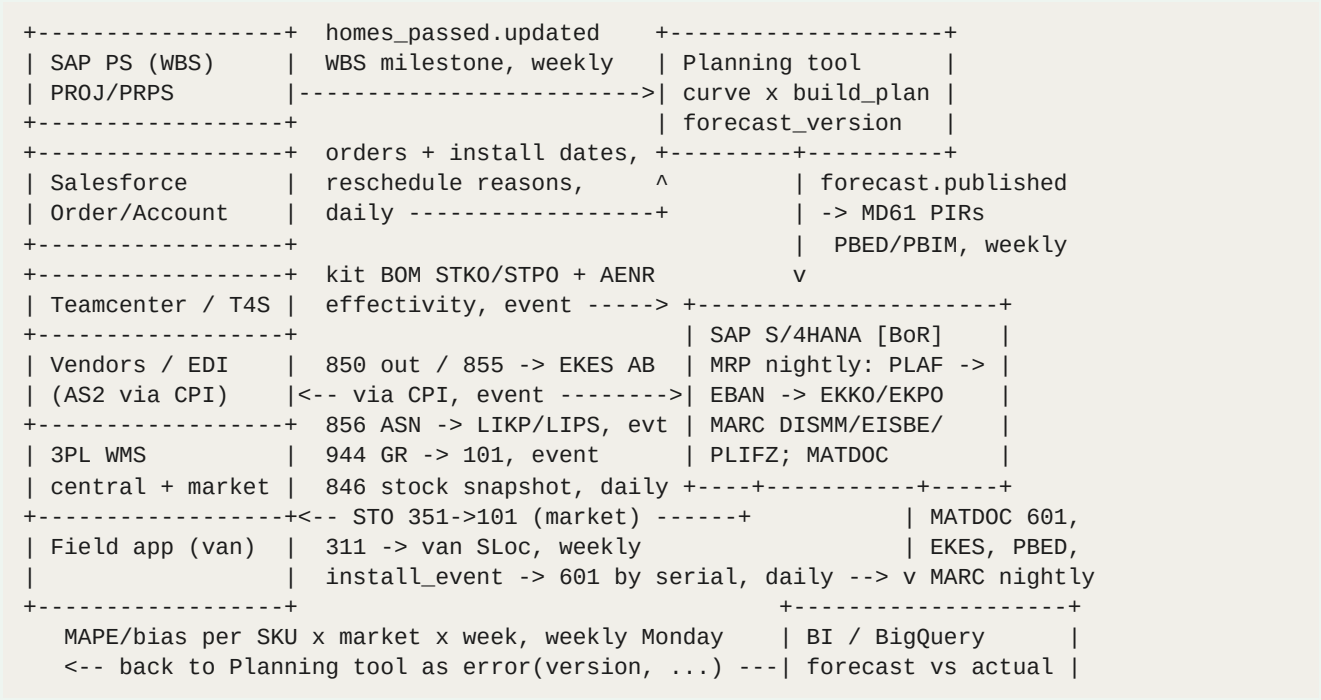
How do you stop one market stealing another's units?

Need floors and hold-until dates on launch reserves; only the uncommitted pool is reallocatable; a floor breach is a screen with a reason code.

Failure modes?

Requirements loaded manually beside the engine — block. A brochure lead time in the master — replace with measured p50, flag drift. Kit BOM effectivity undated — the ECN process (E6). An adoption curve borrowed from a non-analog market — archetype review with Marketing.

SOLUTION FLOW — THE SYSTEM MAP (PROPOSED)



E8 — Elementum design engagement: the monthly release / regression strategy for SAP + integrations

Use for: release management, regression strategy, "a release broke production" questions. **Record behind it:** run experience — the 4-week cadence with the QA lead owning the gate (S15); node × channel × transition regression matrices as the gate (S1); rule changes riding the cadence with a kill switch (S4).

> **Honest line.** This one is experience generalized, not adjacency. Volumes illustrative.

The problem as I restate it. SAP configuration, custom objects, integration maps, the CRM, the 3PL interface and the field app all change; a release broke MRP last month. Design the release train and the regression strategy so changes ship monthly with no surprise on the floor. Book of record: the transport/change log and the release manifest. Primary metric: release-caused Sev 1–2 incidents per release → 0, with the cadence held.

(A) THE TRAIN · (B) FIRST PRINCIPLES

```
[Backlog] --commit (freeze T-15d; definition of ready cites a regression cell)-->
[Release scope] --build--> [DEV transports/packages] --promote T-10d--> [QA integrated]
--regression matrix + E2E chain, T-5d--> [QA verified] --go/no-go T-2d (QA lead owns
the gate; Blocker+Critical = 0; rollback plan; freeze check)--> [PROD] --hypercare,
numeric exit--> [closed]
Rules: scope swapped, never absorbed · config and code in separate transports ·
emergency changes counted and 100% retrofitted · freeze around peak and close
```

Invariants. A change rides the train or does not ship. The regression matrix at the true grain is the gate, and every cell has an owner. The manifest is frozen at T−15; new work is estimated and swapped, never absorbed. Transports import to QA in the same sequence as production; nothing goes "just this once" outside the manifest. Hypercare exits on a number, never a calendar.

(c) Design. The MRP-protection cell: run MRP in QA on a production-like snapshot before and after the transport, diff the exception counts and planned-order totals by plant — unexplained deltas fail the build. Integration maps regression-tested with golden files per transaction set replayed against the new map version, payloads asserted field by field, contract tests against the partner's staging endpoint, dead-letter queue empty after replay. The CRM and field app ride the same manifest and the same go/no-go; if an app-store lag decouples them, SAP-side changes deploy dark and enable when the app reaches the field.

(f) Metric. Release-caused Sev 1–2 per release → 0, cadence held (12 trains/yr, no skipped month), measured from the incident ledger joined to the deploy log by object. Illustrative dollar: one MRP-breaking release that mis-plans a week of equipment can idle a build crew (~$50K/week) and strand or delay ~$200K of POs; avoiding two such incidents a year ≈ $300–500K/yr — before the trust cost of a floor that stops believing the calendar. Supporting: emergency changes ≤1 per release, 100% retrofitted; hypercare exit ≤ T+5; regression cells owned = 100%.

Deep-dive probes.

Why did the release break MRP, and what cell catches it?

Typical causes: a changed MRP type, lot size or planning strategy; a BOM effectivity landing early; a planning-file change; a job variant. The cell: run MRP in QA on a production-like snapshot before and after, diff exception counts and planned-order totals by plant; unexplained deltas fail the build.

How do you regression-test integration maps?

Golden files per transaction set replayed against the new map version; the produced payload asserted field by field; contract tests against the partner's staging endpoint; the dead-letter queue empty after replay.

Transport discipline?

A managed change list; QA imports in production sequence; configuration and code in separate transports so a config rollback doesn't drag code.

Failure modes?

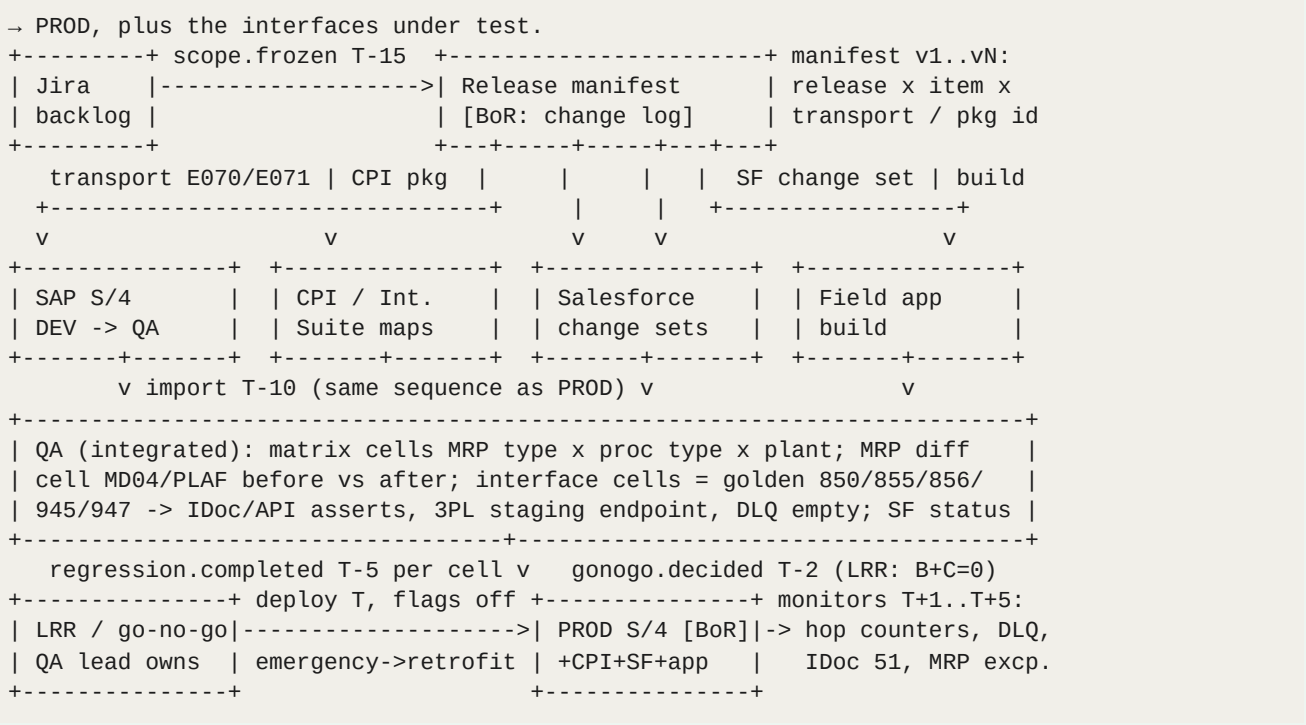
Scope creep after freeze — swap, never absorb. Regression passed on synthetic data missing the failing case — production-like snapshot, anonymized. A transport imported out of order — sequence check. Hypercare exited on the calendar — numeric exit only. Emergency changes not retrofitted — counted in the postmortem, retrofit ticket mandatory.

Trade-offs?

A 4-week train delays small fixes — the emergency path exists and is counted. Automation is expensive up front — build the manual matrix first, automate the cells that fail most.

SOLUTION FLOW — THE SYSTEM MAP (PROPOSED)

The "data" here is the release artefact: transport → regression cell → interface test cell → go/no-go (LRR)



E9 — Elementum design engagement: the ERP cutover from a legacy WMS — tolerance, mocks, freeze, one promise publisher

Use for: cutover/migration design questions — the designed generalization of S18's run experience. **Record behind it:** the 0.5% tolerance published before mock 1, mocks 7 → 1.8 → 0.3 read by cause, the 72-hour freeze, the RF fallback (S18); zero-then-publish (S2); book-of-record discipline (S1, S3).

> **Honest line.** The design is the generalized playbook of a migration I ran; mechanics from the record, volumes illustrative.

The problem as I restate it. A DC (or a 3PL site) runs a legacy WMS with its own inventory book; the client wants the ERP as book of record, with no double-promising during the switch and a Finance-grade reconciliation at go-live. Book of record: the legacy WMS until the switch instant; the ERP after. Primary metric: cutover reconciliation variance vs a published tolerance, and zero double-promises.

(A) STOCKS AND FLOWS · (B) FIRST PRINCIPLES

```
[Legacy on-hand, bin x status] --extract + map (UoM, location, status vocab)-->
[Staging, SKU x SLoc x stock type x serial] --load (561 / migration cockpit,
control totals)--> [ERP initial stock]
[In-flight receipts/shipments] --72-h freeze + their own batch--> [ERP in-transit]
[Open orders] --convert / re-create at cut-off--> [ERP deliveries]
[Both books] --reconcile by cause vs published tolerance--> go/no-go
Promise switch: zero the legacy availability FIRST, then publish the ERP's --
  never the reverse; consumers read one publisher at a time
```

Invariants. The tolerance is published with Finance before the first mock, so "good enough" is arithmetic, not negotiation. A mock is read by cause class, never by headline — a total that shrinks without a named cause is not evidence. The initial load has one door (migration postings with control totals); nothing is "fixed" afterwards outside the governed path. In-transit gets its own batch behind the freeze — the mock-2 lesson. Serials load only after their masters exist, at zero tolerance.

(c) Design. Three mocks on full production data, each a rehearsal of the real cutover including the failback; mapping sets as data (units of measure, location names, status vocabularies — the mock-1 defect classes); the cut-off list that catches late receipts posted against legacy orders; device readiness as a program row with a rehearsed manual fallback (the S18 RF lesson); hypercare with a numeric exit (<2 blockers/day sustained). If the 3PL keeps its WMS, the ERP integrates at storage-location grain over the standard document set (E5); if the client runs the floor, an embedded warehouse module with bin mapping — decided by who runs the floor, not by preference.

(f) Metric. Cutover reconciliation variance vs the published tolerance — e.g., 0.3% against 0.5% at go-live, 0 unexplained serials — with zero double-promises during the switch, measured by the reconciliation-by-cause report at mock 3 and at T+1. Illustrative dollar: the cost of missing the date is the program's own number — in my record it was a $20M pipeline at ~$2.5M/month of slip; for a new client I ask Finance for the equivalent rather than invent it. Supporting: hypercare exit ≤ T+3 weeks; manual stock corrections post go-live outside the governed path = 0.

Deep-dive probes.

Load mechanics?

Migration cockpit or initial-entry postings with control totals; batch and serial masters pre-loaded before balances; value flows to the migration clearing account and Finance reconciles it. The migration posting is the only door — never "fix" with count adjustments afterwards without a cause.

What produced the 7% in mock 1 and how did it get to 0.3%?

Units of measure, location names that mapped to nothing, and order-status vocabulary — fixed in the mapping set (1.8%); then in-transit timing — fixed with the 72-hour freeze and a separate batch (0.3%). Each cause had an owner; the number was read by cause every time.

How do you avoid double-promising during the switch?

Zero the legacy promise source first — legacy availability reads zero to the order systems — then publish the ERP's availability. Never the reverse; consumers read one publisher at a time.

Failure modes?

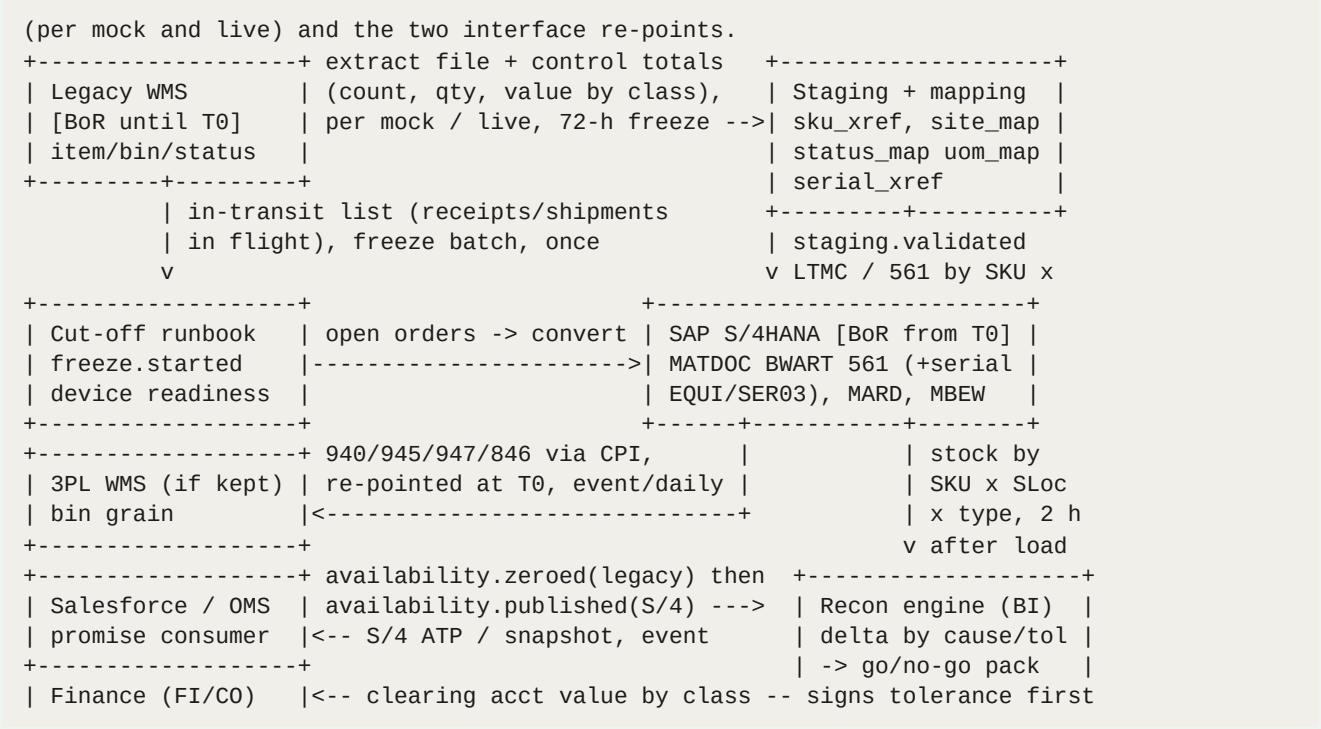
Control totals match but value doesn't — valuation/price mapping, the Finance row. Late receipts posted against legacy orders — the cut-off list. Serials without masters — pre-load. Devices on old screens — readiness row, rehearsed fallback. The one extract expert on vacation — documented, rehearsed by a second.

Trade-offs?

Three mocks cost weeks — they are the weeks that would otherwise be spent live. A 72-hour freeze delays receipts — priced with operations, scheduled off-peak. Zero tolerance on serials produces exceptions — each is a real unit whose state is unknown.

SOLUTION FLOW — THE SYSTEM MAP (PROPOSED)

Before the switch instant the legacy WMS is [BoR] ; after it, S/4 is. The diagram shows the load path once



E10 — Elementum design engagement: the returns / RMA flow — premise → test → refurb / return-to-vendor / scrap

Use for: reverse-logistics, warranty-recovery, disposition design questions. **Record behind it:** bucket state machines with one owning system per transition (S1, S2), reason-coded reconciliation (S14), serial-grain reconciliation at cutover (S18).

> **Honest line.** No RMA program run end to end; the design generalizes the serialized state-machine discipline. Volumes illustrative.

The problem as I restate it. Customers disconnect or swap; techs bring units back; some go straight to a returns bin and are never tested; refurb yield is unknown; warranty recovery from vendors is leaking. Design the reverse flow so every returned serial is tested, graded and dispositioned, and in-warranty failures go back to the vendor. Books: SAP for serial state and stock; the RMA system for reason and test result. Primary metric: returned units dispositioned within SLA, and the warranty recovery rate.

(A) STOCKS AND FLOWS · (B) FIRST PRINCIPLES

```
[Installed base] --disconnect/swap (RMA w/ reason code)--> [Return-in-progress]
--pickup / tech return (651 by serial)--> [Returns-untested, market RMA SLoc]
--weekly consolidation (STO)--> [Central test center] --test-->
  [Refurb-good A/B (321)] | [Repairable] | [Failed (344 blocked)]
Failed + in-warranty --> RTV w/ vendor RMA number (122 / return PO) -> credit
Failed + out-of-warranty --> scrap (551, with certificate)
No test, no grade -- an untested return that is promisable is a truck roll
```

Invariants. Every returned serial passes through test — no grade, no availability. Refurb identity: same serial, same material, a grade attribute — never re-numbered, so the asset history stays continuous. Warranty eligibility computes from the serial's own receipt or install event per the vendor's contract term — never from a person's memory of a PO date. A return-to-vendor without the vendor's RMA number is blocked — the credit would be denied. Scrap of an in-warranty serial is prevented by rule; the exception needs Finance approval.

(c) Design. Return order with reason code at initiation; receipt by serial into returns-untested; weekly consolidation to one central test bench — one grading standard beats faster-but-unreliable grading on the truck; test-result events drive the state machine; graded availability published with the grade so the order system can decide which customers receive grade B (policy, not technology); the RTV credit reconciliation as a standing report. Idempotency: serial + event type + document; a repeated test result replaces only if newer and flags the earlier; a repeated return receipt for the same RMA is rejected.

(f) Metric. Dispositioned within SLA ≥ 95% within 14 days; warranty recovery on in-warranty failures ≥ 80%. Illustrative dollar: 48K returns/yr × 75% refurb yield × $120 replacement cost avoided ≈ $4.3M/yr of new-unit purchases displaced (net of ~$25 test cost on all returns ≈ $3.1M net); warranty recovery lift 40% → 80% on 8K failures × 60% in-warranty × $150 ≈ $290K/yr. Supporting: returns-untested aging p95 < 14 days; refurb field-failure rate ≤ new-unit rate + margin; scrap with certificate = 100%.

Deep-dive probes.

Movement types and documents?

Return delivery to returns stock (651; to QI if inspecting at receipt — never straight to unrestricted); QI → refurb location on pass; unrestricted → blocked for fails; stock transfers for consolidation; return against the original PO for RTV; scrap with reason; a write-off for a non-return only with a case reference.

Warranty start — ship or install?

Contract-dependent; store the term and start event per vendor and compute eligibility from the serial's receipt or install event. Never a person's memory.

Failure modes?

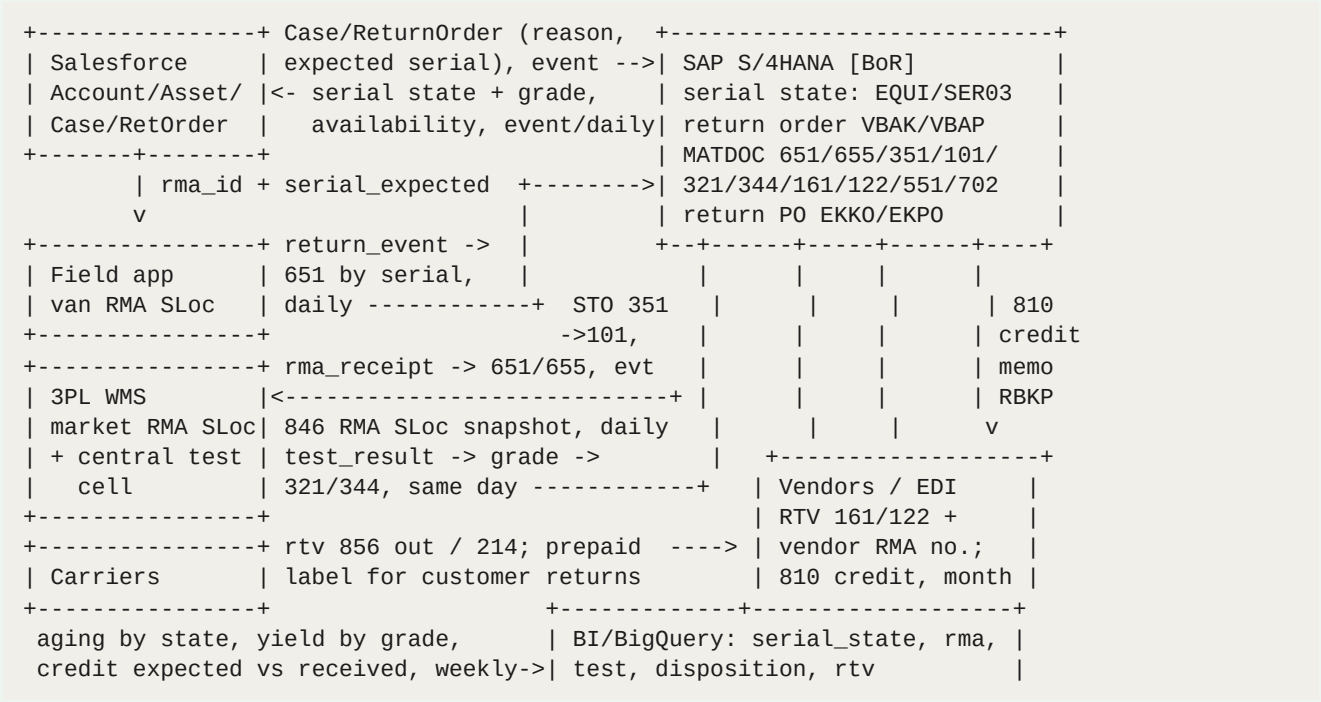
A tech "returns" a unit that never posts — that's E1's unaccounted; the return is a swap event on the van. A test result for a serial not in returns-untested — hold; the state machine refuses. RTV shipped without a vendor RMA number —

blocked. Scrap posted for an in-warranty serial — rule prevents it.

Trade-offs?

Centralized testing adds a hop and days — but one grading standard. "No test, no grade" means refurb stock lags during a surge — better than a truck roll on a bad unit.

SOLUTION FLOW — THE SYSTEM MAP (PROPOSED)



E11 — Elementum design engagement: direct procurement of long-lead equipment — PO lifecycle, confirmations, ranked expediting

Use for: procurement design, vendor-reliability, long-lead-time questions. **Record behind it:** SAP purchasing data domains and EDI confirmations (S6, S9); lead-time distributions (S8); the carrier bid's constraint rules (S17).

Honest line. The indirect-procurement platform boundary is a knowledge pattern — I have not configured one; the boundary design is how I would frame the decision, not a claim. Volumes illustrative.

The problem as I restate it. Core equipment carries 12–40-week lead times, allocation risk, and multi-site delivery; today POs are placed off spreadsheets, confirmations live in email, and expediting is heroic. Design the procure-to-receive lifecycle in SAP for direct materials, define the boundary with the indirect-spend platform, and make confirmed-date reliability a managed metric. Primary metric: on-time-to-confirmed-date receipts, and PO lines without a vendor confirmation → 0.

(A) STOCKS AND FLOWS · (B) FIRST PRINCIPLES

```
[Demand: build plan / MRP need by week] --requisition (release strategy)--> [PR approved]
--source (source list / info record / contract)--> [PO sent, 850 out]
--confirm <= 48 h (855 -> confirmation record, MRP-relevant)--> [PO confirmed]
--at risk? expedite ranked by milestone-at-risk, not by who shouts--> [re-confirmed]
--ship (856) / receive (101) / invoice (810, three-way match)--> [closed]
OTIF measured BOTH ways: to-request (our planning discipline) and to-confirmation
  (vendor reliability) -- managed to confirmation; the gap published separately
```

Invariants. A confirmation without a date is not a confirmation — after 48 hours the line is a planner exception. Planning runs on the *confirmed* date, not the requested one, so the plan reflects reality rather than hope. Every urgent buy goes through a requisition — spreadsheets creeping back is a failure mode, blocked by the release strategy. A PO changed after confirmation without re-confirmation is flagged.

(c) Design. Confirmation control made MRP-relevant so the vendor's date replaces the schedule line in planning; expedite list ranked by the milestone at risk × value, refreshed weekly; for allocated long-lead items: outline agreements with scheduled releases, dual sourcing where qualified, and a strategic buffer sized by the p99 lead-time tail against the build plan's variance (toolkit, labeled). The direct/indirect boundary as a category table owned with Procurement and Finance: items that become stock or project stock → SAP MM; services, consumables expensed at receipt, tools, software → the indirect platform, which posts to the GL via the standard integration and never creates stock.

(f) Metric. Receipts on or before the vendor's confirmed date ≥ 95%; PO lines lacking a confirmation after 48 h → 0 — measured per line, monthly per vendor. Illustrative dollar: a late shipment that idles a build week costs ~$50K in crew and deferred activations; confirmation discipline plus ranked expediting avoiding ~15 such weeks/yr ≈ $750K/yr; the measured-lead-time buffer that prevents one allocation stockout protects a quarter's plan — the client Finance's number, not mine. Supporting: requested-vs-confirmed gap published; receipts without ASN → 0 for EDI vendors; spreadsheet POs → 0.

Deep-dive probes.

Confirmation control — how does it change planning?

With an MRP-relevant confirmation category, the confirmed date and quantity replace the schedule line for planning; the supply view shows the confirmed receipt. A PO with no confirmation shows the requested date — which is why the 48-hour exception exists.

OTIF to confirmation or to request?

Measure both, manage to confirmation: it isolates vendor reliability from our own over-asking. The requested-vs-confirmed gap is its own planning-discipline metric.

Allocated items with 40-week lead times — what do you do?

Outline agreements with scheduled releases; dual source where qualified; a strategic buffer sized from the p99 lead-time tail and the build plan's variance — toolkit, labeled; expediting ranked by the milestone at risk, not by who shouts.

Where exactly is the direct/indirect boundary?

Items that become stock or project stock live in SAP MM; services, expensed consumables, tools and software live on the indirect platform, which posts accruals to the GL and never creates stock. The boundary is a category table owned with Procurement and Finance — decided once.

Failure modes?

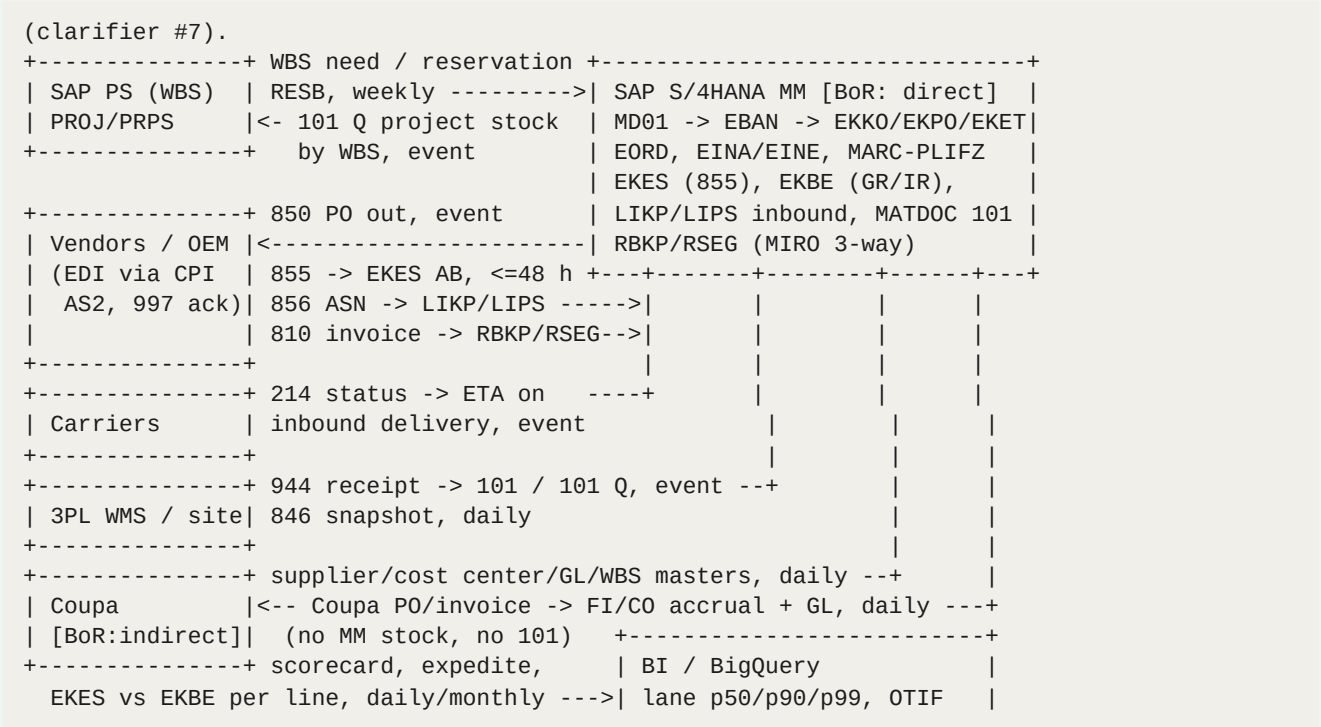
"Acknowledged" with no date — unconfirmed. A PO changed after confirmation without re-confirmation — block or flag. A receipt posted to the wrong plant or project — account-assignment check. A confirmation for a closed PO — exception. Spreadsheets creeping back for urgent buys — the release strategy requires a requisition.

Trade-offs?

Confirmation discipline creates exceptions in month one — that is the diagnostic; it reveals the unreliable vendors. Project stock improves site accountability but fragments the pool — use it only when the site is fixed.

SOLUTION FLOW — THE SYSTEM MAP (PROPOSED)

Direct (becomes MM or project stock) lives entirely in S/4; Coupa handles indirect and never creates stock



E12 — Elementum design engagement: smart vending machines — the consignment ledger, and why a tech van is a vending machine

Use for: open-ended system design, event-ledger design, consignment/working-capital questions — the showpiece design.
Record behind it: the real-time decrement ledger with idempotent events and daily reconciliation to a settlement book (S1); one unit in one bucket, counts change the book only through posted documents (S1, S14); lower-of-two when ledger and physical disagree (S1, S18).

> **Honest line.** I have not built a vending network or run a consignment program; the par-level and safety-stock formulas are the MicroMasters toolkit, labeled; every number is illustrative.

The problem as I restate it. A network of connected vending machines sells product with no human at the counter: slots with a planogram, dispenses on payment, trucks on refill routes, machines sometimes offline. Design the inventory ledger, the payment flow, the replenishment and the reconciliation so the operator, the supplier and Finance agree on what was sold, what is on the shelf, and who owns it. Books: the dispense-event ledger for units, the ERP as the settlement book; the payment processor for money. Primary metric: fill rate (dispenses served / attempted) with shrink % inside tolerance.

Why this design matters beyond vending. It is the consignment question in its purest form: if the supplier owns the product on our shelf until it is dispensed, that is special stock K — receipt creates custody, not liability; the dispense is the consumption that creates the liability; month-end is a settlement, not an invoice. **And a field tech's van is a vending machine**: the van is the machine, the install is the dispense, the weekly restock is the refill, the van kit is the planogram. E1 is the own-stock version of this design; E12 is the version where the vendor owns the unit until the dispense — for a capital-hungry operator, a working-capital lever: hundreds of vans of equipment stop sitting on the balance sheet until the day of install.

(A) CLARIFIERS · STOCKS AND FLOWS · (B) THE MATH

```
Clarifiers first (five): scale (default 500 machines x 40 slots x ~12 dispenses/day
  ~ 6,000 events/day) · ownership (own stock vs consignment K -- K is the interesting
  case) · connectivity (assume a machine can be dark for hours) · payment (offline auth
  allowed to a small cap) · replenishment (our drivers, fixed weekly routes, handheld)
[Depot (own or K)] --load route truck--> [In-transit per route x stop] --refill event
(counted_before + qty_added + planogram version)--> [Slot on-hand] --dispense (payment
auth -> motor -> capture; event_id = machine x slot x seq)--> [Sold / consumed (411 K)]
Monthly: settlement = Sigma consumption x agreed price (MRKO -- a settlement, not an
  invoice), three-way matched: consumption vs delivery notes vs refill counts
Par per slot = mu_d*(R + L) + z*sigma_d*sqrt(R + L), capped at slot capacity C
  e.g. mu=1.2/day, sigma=0.8, R=7, L=1, z=1.65 -> 9.6 + 3.7 ~ 13; capacity 15 holds;
  if capacity were 10, the slot is capacity-bound -> the ROUTE INTERVAL moves, not the par
Refill trigger: on_hand < Par - mu_d x (days to next scheduled visit)
Route ranking: Sigma_slots P(stockout before next visit) x margin per dispense
Shrink_slot = ledger_before_refill - counted_before; shrink % reason-coded per route
Worked network: 20,000 slots · ~6,000 events/day (~250/hr, 10x lunch peak ~2,500/hr --
  correctness is the hard part, not throughput) · weekly routes -> ~71 visits/day ->
  3 trucks x ~25 stops · shrink 1.5% of ~42,000 refilled units/wk x $1.50 ~ $50K/yr
```

Invariants. Every unit is in exactly one stock; the dispense event is minted at the edge (journaled before the motor turns) so a crash between dispense and send still produces exactly one event. Money is stricter than units: auth is synchronous or offline-capped; capture is at-least-once with idempotent ids; a refund is a compensating event referencing the original — never an edit. The count changes the book only through a posted refill count; the settlement is reproducible from the event store or it is not trusted.

(f) Metric. Fill rate ≥ 98% with stockout slot-hours published per route; shrink ≤ 1.5% (illustrative tolerance). Illustrative dollars: 500 machines × 12 dispenses × $2.00 ≈ $12K/day of revenue; a 2-point fill-rate lift on a network stocking out 5% of the time recovers ≈ $85K/yr; the van-consignment analogue defers cash on ~500 vans × 30 units × $150 ≈

$2.25M of stock off the balance sheet until install — Finance's number to confirm, not mine. Supporting: dispense → ledger latency p95 < 60 s online, backlog drained < 10 min after reconnect; payment success ≥ 99%; offline exposure loss < the margin offline enables.

Deep-dive probes.

Event ordering with clock skew?

Order by the machine's monotonic sequence number per machine × slot, never by device wall-clock; carry both device and received timestamps. Cross-machine order doesn't matter — each machine is its own stream; device time is corrected from the heartbeat offset for reporting only.

Idempotency key design?

machine × slot × sequence, minted at the edge and persisted in the journal before the motor turns — a crash after the dispense and before the send still produces one id. The server keeps a per-machine high-water mark plus a short dedupe window.

Offline authorization limit?

Per machine: a cap on count and amount per offline hour, decaying as the outage lengthens; per card: one offline auth per machine per day; a floor limit on price. Set the cap where expected decline loss < the margin the offline window enables.

How do you detect a jammed slot from data?

Auth without a capture on the same slot; dispense events with a jammed outcome; a slot whose ledger never moves while neighbours do; a refill count equal to the ledger plus a run of voided auths. Two together raise the alert before the customer calls.

Reconciliation cadence?

Per dispense: auth ↔ capture ↔ event, continuous. Per refill: count vs ledger per slot — the physical inventory. Daily: captures vs the processor settlement file. Monthly: consumption vs supplier delivery notes vs refill counts — the three-way match behind the settlement. Unit-ledger tolerance zero; shrink tolerance negotiated in the consignment agreement.

Consistency model and scale?

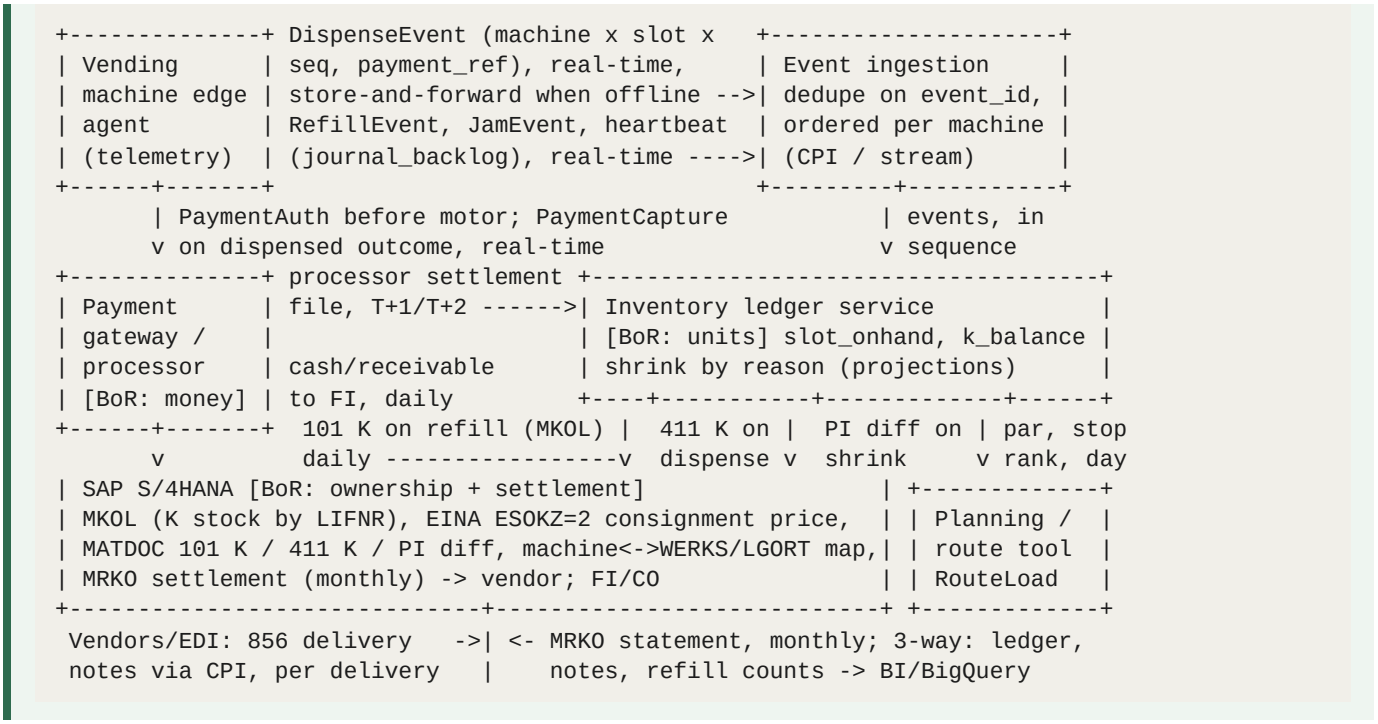
Eventually consistent projections over a strongly ordered per-machine event log; partition by machine id; consumers idempotent on event id; projections rebuilt from the log on demand. Six thousand events a day is small — the design holds the same shape at 50,000 machines; only the partition key count changes.

Fraud?

Repeated offline auths on one card across machines; refund-outcome clusters on one machine; shrink concentrated on one route or driver; door-open events without a refill event. Controls: offline caps, remote lock, driver-level shrink reporting, refund rate per machine on the daily dashboard.

SOLUTION FLOW — THE SYSTEM MAP (PROPOSED)

and settlement (special stock K, MRKO), the payment processor is [BoR] for money. In the GFiber analogue the machine is the tech van and the dispense is the install.



3.99 — Cross-cutting: how I open any "design X" prompt (say this in the first minute)

1. **Book of record and metric first** — "SAP is the book; the metric is unaccounted units per contractor per month — agree?"
2. **Stocks on the whiteboard** — every bucket a unit can sit in; then the flows, each with the document that records it.
3. **The invariant that is broken today** — a unit in two stocks, a flow with no document, a number with two owners.
4. **Minimal design the invariants force** — then components, grain, events, controls, NFRs.
5. **Sequence the phases** (Reach × Impact × Confidence ÷ Effort) — and say which invariant overrides the score.
6. **One metric, one dollar** — illustrative, labeled, measured from posted documents.
7. **How I'd run it** — point to §3.0 once; never re-explain the method per prompt.

4. Numbers and never-say — one canonical value per metric

Carried from the GFiber v8 ledger, kept as-is per the transform instruction. A record dollar is said flat; a modeled dollar carries the word "modeled" in the same breath and its basis lives in the story's Business impact model. Where §1.3 marks a conflict with the loop packet, this table is this document's canon.

4.1 One canonical value per metric

METRIC	VALUE TO SAY	DO NOT SAY	MODELED IMPACT (BASIS IN THE STORY)
Sephora ISM spec (S1)	FSD v1.1 (3/5/25) → v1.2 (5/16/25) → signed 6/5/25 by Supply Chain Strategy, Inventory Management, FSC Inventory Accounting	—	S1: ≈$2.5M/yr = $1M MCS support retired (user anchor) + ≈$1.5M modeled (oversell ≈$0.9M, recon labor ≈$0.33M, change premium ≈$0.3M); range $1.7–5.2M; upside not in headline: shrink ≈$0.3M, time-to-market ≈$0.2M
UDC (S2, S5)	Zero customer-facing incidents; +15% processing efficiency; 836-row issues log 85.4% closed; robotics gate 98%, week 2 at 94%, ~30% throttle one week; a dozen core systems, 20+ touched; 60+ stakeholders (~80 names across ~15 functions); council 5/15 → sign-off 7/31 → FSD 8/14; rollback triggers sync <99% for 4 h / p95 > 2× for 1 h; cutover 343 tasks / ~815 hrs	"12 systems" alone or "20 systems" alone; a single stakeholder count	S2: ≈$1.6M/yr modeled + ≈$2.2M one-time √n release · S5: ≈$0.5M one-time modeled · S3: ≈$0.6M/yr modeled · S4: ≈$0.5M/yr modeled, oversell +$0
RP → JDA (S18)	8 months; ~$1.8M, ~3% under (~$54K); mocks 7% → 1.8% → 0.3% vs 0.5% tolerance; $20M pipeline (~$2.5M/month); 72-hour freeze; legacy read-only 30 days; hypercare exit week 3 (<2 blockers/day); RF ~100 of ~400 devices; 1.5-week training slip	"400 devices failed"	S18: record stays headline; modeled ≈$0.1M less inventory in dispute (0.3% vs 0.5% on illustrative $50M), ≈$45K fallback cost; ~$875K critical-path slip avoided is record math, not banked
Supply/demand view (S6)	Forecast accuracy 70% → 85% on scoped class-A SKUs; $2M safety stock released; shortage alert live in 3 months; planner effort −50%; eight domains	Pasting "3 months" onto the six-site ingest	$2M is record (one-time); modeled recurring ≈$0.8M/yr (carry ≈$0.2M, expedites ≈$0.3M, planner capacity ≈$0.3M; range $0.5–1.2M)
Six-site CM ingest (S7)	Six sites, three waves, 5 months; hardest site 6 weeks vs 3-week average; zero auto-loaded revision overlaps	"6 NA sites"; "500+ product families"; 60→95% inside this story	≈$0.3M/yr modeled (recon labor, phantom expedites, overlap rework; range $0.2–0.7M) — $1.5M/580K stays with S8
Lead-time program (S8)	200 lanes, 99% lead-time accuracy, 580K units / ~$1.5M released	Adding it to the $2M into one number	$1.5M is record (one-time); modeled recurring ≈$0.6M/yr (carry ≈$0.3M, expedites ≈$0.3M; range $0.3–1.4M)

METRIC	VALUE TO SAY	DO NOT SAY	MODELED IMPACT (BASIS IN THE STORY)
EDI (S9)	1,000 routes asked → 597 first = 80% of spend; completeness 60% → 95%; adoption 8×; 100% visibility on high-value lanes; freight −10% ≈ $2M in 6 months; ≥95% valid-ASN cohort gate; ~200 next wave	Cohort sizes or the number of cohorts	$2M is record; modeled decomposition compliance ≈$0.8M / mode ≈$0.6M / expedite-detention ≈$0.6M (they SUM to the record); modeled add-on ≈$0.4M/yr document labor (range $0.2–0.9M)
Austria routing guide (S10)	True DQ ~55% (reported 40%) → ~99% in 3 months; manual 8–10 → 1–2 hrs/wk; $500K/yr (10% of ~$5M); 25 EU sites / 400+ lanes ≈ $8M; ~8% of lanes need Incoterm in the key	"$1M"; "99% in two months"; 1–2 hrs as the before	$500K → ~$8M is record; roll-up shown (~$320K/site); modeled add-on ≈$0.5M/yr analyst capacity (range $0.2–0.7M)
Carrier bid (S17)	6 weeks → ~3 (~50%); pilot 3 weeks early; <5% mismatch gate; 10-day stale banner; no $ figure	"$500K/yr"; "−25% planning effort"; any award dollar	≈$0.19M/yr modeled analyst hours returned (range $35–420K) — never award savings
Manhattan (S13)	3 DCs + 100 stores; ~$6M at risk → ~$8M protected; 99% fill vs 95% goal; stockouts −20%	"$2MM baseline"	$8M / $6M / 99% are record; modeled ≈$3.2M margin (40% GM), ≈$0.26M CS/cancel, ≈$0.13M expedite (one-time); stockouts −20% ≈ $1.3M/yr margin (range $0.4–2.7M)
Manhattan discrepancy (S14)	~200 orders / $2.5M; $10M initiative won	The $10M counted as savings	Modeled prevention run-rate ≈$0.1M/yr (range $55–160K); the silent script's expected ≈$1.5M corruption is avoided risk, not a saving
Flexport WBR (S11)	Automation 40% → 90%; CTS $0.66 → $0.03 (−95.5%); accuracy 60% → 90%; rollout unlocked 2 months early; 1.5–2 → ~4 hrs/wk by hand; 99.4% week discarded; customers 7 → 68 (9.7×); 170 MAU target	"$1.00 → $0.03"; "70% baseline"; "10×"	CTS record; modeled direct ≈$50K/yr at 32K shipments (range $30–80K) + ≈$0.25M one-time pull-forward — the value is scale without headcount; ~$1.5M pre-prod is a separate canonical
Pre-prod measurement (S11)	300 containers, ±1.6% at 95%; accuracy 85% → ~98%; rollout 1.5 months vs 6; ~$1.5M saved; 500 containers / 20 min / 99.6%	Merging the $1.5M with CTS	—
Roadmap flip (S12)	18 months → value in 6; 9 vs 6 months re-estimates; −25% vs −50% CTS; 66% less effort; ~$5M cost avoidance (say once); 339.5 min/shipment; 2,200 interactions; 48 of 53 migrated	"$5M annual cost reduction" in the same breath as the avoidance	~$5M avoidance is record (said once); modeled ≈$2.9M eng avoided + ≈$1.5M pulled forward (≈$4.4M, within ~12%); run-rate ≈$1.5M/yr (range $1.1–2.3M)
Snowflake (S16)	4 systems → 1; >$100K/yr priced; parity within 1%; 2 weeks lost to the security review; accuracy 60% → 90%; 4% ticket-sync drop caught	The $100K as claimed savings	>$100K/yr is a priced baseline (record of the pricing); modeled ≈$90K/yr inside it (hours ≈$30K, decision risk ≈$70K → ≈$50K retired, bus factor ≈$15K)
Agile conversion (S15)	20-person team; 6 weeks; critical defects −50%	A direct-report headcount	≈$0.13M/yr modeled (escaped criticals ≈$90K, late-land rework ≈$40K; range $80–270K); no headcount

METRIC	VALUE TO SAY	DO NOT SAY	MODELED IMPACT (BASIS IN THE STORY)
SAP tenure	S/4HANA integration program lead 3+ years (Apr 2023 – present); client-SAP data integration at Elementum 2017–19; 15 years full-lifecycle SCM systems	"10 years of SAP configuration"; "SAP functional owner"	—
Quant toolkit	"MIT Supply Chain MicroMasters + Supply Chain Network Design certificate — acquired so I could design what I'd been implementing"	"I build supply chain models for a living"; any story where modeling is the day job	The modeling formulas across §2–3 are this toolkit, labeled in the cover voice block and the §3 preface, and per story where used

4.2 Never-say list (one glance before every conversation)

- Never "I built the model" as a day job — "I owned the requirement/definition/rule; the data team or engineers built; the toolkit is from MIT."
- Never "I configured S/4," "I deploy SAP," or "I wrote ABAP" — "I led the integration program and owned the data contract and definitions; the SAP team / SI configured and coded to the spec I signed." The only platform I configured myself is Manhattan DOM.
- Never claim Salesforce, Coupa, Teamcenter, VMI or serial-profile implementation experience — "adjacent, bridge honestly," and name the pattern. The §3 designs carry their honest lines for exactly this reason.
- Never: "6 NA sites" · "500+ product families" · 60→95% inside the CM-ingest story · a $ figure for the carrier bid · "$1M routing guide" · "$1.00 → $0.03" · "10×" · "400 devices failed" · cohort sizes · $2M + $1.5M added together · $5M "avoidance" and "annual reduction" as two results.
- Never present a modeled number as record: the SNB drift dollars (S3), the bid cycle-time value (S17), the S1 oversell dollars, the S2 labor dollars are modeled — say "modeled from these inputs" and give the basis. Numbers with no basis stay uninvented: direct-report headcounts, carrier-bid award savings, realized dollars on the $10M initiative. Say "I don't have a canonical figure and won't invent one; what I can defend is…"
- Never "we" for an action that was mine; name the team's part once, explicitly.
- Never volunteer only successes — S4 (the rule I got wrong), S18 (the RF miss), S16 (the security-review miss) and the UAT-crash bench story are the failure stories; Amazon explicitly probes for lessons from mistakes.
- **Amazon-specific:** never tell the same story to two interviewers — the §1.1 map exists so each LP has a primary, backup and third. Never carry a §1.3-conflicted number into the room before resolving which document is right. Every modeled dollar gets its basis label out loud; the loop packet's "my model, Finance's rates" discipline applies to every figure here.

4.3 Bench stories (not core headings — pull only if asked for "another example")

BENCH STORY	WHERE IT LIVES
Status-mapping root cause, ~$2.8M (Flexport)	Referenced in E4's probes — the status-map defect class
UAT crashed under load; numeric entrance criteria ever since	The second failure story if S4/S16/S18 are spent (loop packet S16)
110%-capacity disagree-and-commit with a tripwire	Backbone backup — disagree, commit, name the tripwire
Two programs, one team, war room	Prioritization under conflict

BENCH STORY	WHERE IT LIVES
"Too tactical" → fever chart, 40 integration points (NA scope, 1 month)	Never "25 integration points"
LLM triage agent — eval set first	Recent AI judgment; unit economics before scale (loop packet S13)
Controlled release bench: 30% of lanes ≈ 90% volume; lead times 7–30 days; 98% ASN first 3 days; APAC ≈ $2M	Never "LATAM"; never "20–30 days"

> **Final rule of the packet.** This document and the loop packet are one system: this one carries the depth (gated STARs, phase probes, impact models, the design round); the loop packet carries the loop mechanics (interviewer tracker, the 128-question AIQB bank, writing defense, video execution). Rehearse from both; resolve the §1.3 conflicts before the loop; and in the room, say less than you know — the pitch ends on the primary metric, and the probes are how the interviewer fills their worksheet in your favor.